Quantum Computation and Information

Saavan Shah

July 2026

Contents

1 Introduction **4**

2 An Abridged History of Quantum Computing **4**

3 Building a Quantum Computer **7**
3.1 Qubits . . . 7
3.2 Quantum Gates . . . 8
3.2.1 Single Qubit Gates . . . 8
3.2.2 Multi-Qubit Gates . . . 9
3.3 Let's Talk Initialization and Measurement . . . 10
3.4 Entanglement and Quantum Teleportation . . . 11
3.4.1 Creating the Bell State . . . 11
3.4.2 Quantum Teleportation . . . 11
3.4.3 Superdense Coding . . . 12
3.4.4 Quantum Key Distribution and a First Look at Quantum Communication . . . 13
3.5 Quantum Logic . . . 14
3.5.1 Superposition and Linearity . . . 14
3.5.2 Interference . . . 14
3.5.3 Entanglement . . . 15

4 A First Look at Quantum Algorithms and Quantum Information Theory **16**
4.1 Classical Computations on a Quantum Computer . . . 16
4.2 Quantum Parallelism and the n-Hadamard . . . 16
4.3 The Deutsch-Jozsa Algorithm . . . 17
4.3.1 The Bernstein-Vazirani Algorithm . . . 18
4.4 A Summary of Quantum Algorithms . . . 19
4.5 Complexity Classes . . . 20

5 Quantum Circuits **21**
5.1 Summary of Quantum Operations, Thus Far . . . 21
5.1.1 Single Qubit Gates . . . 21
5.1.2 Controlled Gates (Multiqubit Gates) . . . 22
5.1.3 Measurement . . . 23
5.2 Universal Quantum Gates . . . 23
5.2.1 Single-Qubit + cNOT Gates Form a Universal . . . 24
5.2.2 Clifford vs. Non-Clifford Gates . . . 24
5.2.3 The Search for a Discrete Universal Set . . . 25
5.3 Another Look at the Embedding of Computational Classes . . . 26
5.4 A Note on Quantum Simulations . . . 27

6 The Quantum Fourier Transform and Quantum Search Algorithms 28
6.1 Talking About the Quantum Fourier Transform and its Applications . . . 28
6.1.1 The Quantum Fourier Transform . . . 28
6.1.2 Phase Estimation . . . 29
6.2 Period Finding and Shor's Algorithm . . . 30
6.2.1 Period Finding . . . 30
6.2.2 Order-Finding and Shor's Algorithm . . . 31
6.3 The Hidden Subgroup Problem . . . 32
6.4 A Quantum Search using Grover's Algorithm . . . 33
6.4.1 The Oracle . . . 33
6.4.2 How to Search . . . 34
6.4.3 Geometric Visualization . . . 35
6.4.4 Constructing the Oracle and Phase Unitaries . . . 36
6.5 Quantum Counting . . . 37
6.6 A Few Closing Words . . . 38

7 Physical Realizations of Quantum Computers 39
7.1 What Makes a Good Quantum Computer . . . 39
7.2 Harmonic Oscillator Quantum Computer . . . 40
7.3 Optical Photon Quantum Computer . . . 41
7.4 Ion Trap Qubits . . . 43
7.5 Superconducting Qubits and Planar Hardware . . . 45
7.6 Neutral Atoms and Rydberg Arrays . . . 47
7.7 A Farewell to Quantum Hardware . . . 48

8 Quantum Error Correction 50
8.1 Understanding Quantum Channels and Single Qubit Errors . . . 50
8.1.1 Quantum Channels . . . 50
8.1.2 Multi-Qubit Noise and t-Qubit Errors . . . 52
8.2 Classical Error Correction Codes . . . 53
8.2.1 First Look at Coding Distance . . . 54
8.2.2 Syndrome Decoding . . . 54
8.2.3 General Linear Codes . . . 54
8.3 First Look at a Quantum Error Correction Code: the Shor Code . . . 55
8.3.1 The 9-Qubit Code . . . 55
8.3.2 Understanding Code Distance . . . 57
8.4 Quantum Error Correction and CSS Codes . . . 57
8.4.1 Revisiting the Shor Code . . . 59
8.4.2 The Steane Code . . . 60
8.4.3 CSS Parity Check Matrices . . . 61
8.4.4 Quantum Hamming Bound . . . 61
8.5 Stabilizer Codes . . . 61
8.6 Quantum Error Correction as Topology and Fault Tolerance . . . 63
8.6.1 What Is Fault Tolerance? . . . 63
8.6.2 Why Topology? . . . 64
8.6.3 The Toric Code . . . 65
8.7 A Golden Standard of QEC: the Surface Code . . . 68
8.7.1 Cutting the Donut . . . 69
8.7.2 The Finer, Logical Details . . . 69
8.7.3 Syndrome Extraction and Decoding . . . 70
8.7.4 The Rotated Surface Code . . . 70
8.7.5 How Good is This? . . . 71
8.8 Lattice Surgery, Logical cNOTs, and the Logical Hadamard . . . 71
8.8.1 Merging and then Separating Two Patches . . . 72
8.8.2 Building a cNOT Out of Two Measurements . . . 73
8.8.3 The Cost of Lattice Surgery . . . 74
8.8.4 The Logical Hadamard . . . 74

8.9 The Logical S- and T-Gate and the Need for Magical State Distillation 75
8.9.1 A Close Look at the Logical S-Gate 75
8.9.2 The Logical T-Gate 76
8.9.3 Magical State Distillation 77
8.10 A Farewell to Quantum Error Correction 79

9 Conclusion **79**

1 Introduction

Now that we have discussed Quantum Mechanics and Linear Algebra, we can truly understand Quantum Computing. Here, we will explore the fundamental algorithms, hardware, and information theory at the heart of this burgeoning field. Additionally, with all the "hype" surrounding quantum computers, it is important to discern fact from fiction. Thus, I want to help convince the reader what quantum computing is and, more importantly, isn't.

2 An Abridged History of Quantum Computing

- The history of quantum computing must begin with that of quantum mechanics (1900-1930).
  - **The Ultraviolet Catastrophe** (1900) was a physical conclusion that betrayed all of classical mechanics: an ideal black box at thermal equilibrium would continuously emit infinite amounts of energy at high frequencies (in the UV spectrum), which violates the Conservation of energy.
  - Max Planck (1900) resolved this issue with the idea that energy is quantized (there is a "smallest" notion of light). This is highly controversial, but later proven to be true.
  - One of the staunchest adversaries of Quantum Mechanics, Albert Einstein, gave a key piece of evidence in support of the idea of quantized energy with his experiments on the Photoelectric effect (1905).
  - Werner Heisenberg and Erwin Schrödinger (1925-1926) gave us the modern formulations of quantum mechanics. Names very familiar within the field. Heisenberg would then go on to "solve" the hydrogen atom.
  - Paul Dirac introduced bra-ket notation and much of the mathematical framework (identifying it as linear algebra) still used today. (1928)
  - At this stage, quantum mechanics was a purely theoretical framework describing nature (not a computational model).
- As the field of quantum mechanics proliferated and was proven, the idea of quantum computing emerged because simulating quantum systems on classical computers became exponentially intractable (1980-1982).
  - Paul Benioff (1980) proposed a quantum version of the classical Turing Machine. Then, in 1981, Richard Feynman argued that classical computers cannot efficiently simulate quantum physics and suggested building computers that themselves obey quantum mechanics.
  - Independently, in 1982, Yuri Manin expressed similar ideas.
  - The consensus was that, if Nature is quantum, so perhaps computers should be quantum too.
- **The Ising Model:** one of the simplest and most important models in statistical physics. It was originally introduced to study ferromagnetism, but it has since become central to condensed matter physics, phase transitions, optimization, and quantum computing.
  - Setup: Imagine a lattice (a line, square grid, or 3D crystal), where each vertex contains a tiny magnetic moment (spin 1/2 particle). Thus, the spin state of each particle must be either up or down spin. We want to determine the ground state (lowest-energy configuration) of all the spins.
  - The Hamiltonian of a spin configuration is

$$H = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i$$

  - Here J is the interaction strength between neighboring spins, h is an external magnetic field, $\langle i, j \rangle$ means the sum runs over neighboring lattice sites.

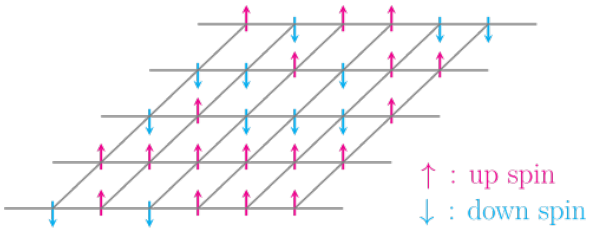


Figure 1: 2D Ising Model on Lattice

  - If $J > 0$ (ferromagnetism), neighboring spins "prefer" to align. Thus, the ground state will have each spin align with the magnetic field (or generally in the same direction for no magnetic field).
  - If $J < 0$ (antiferromagnetism), neighboring spins prefer to point in opposite directions (thus we get a pseudo-checkerboard), but when we apply a (inhomogeneous) magnetic field the checkerboard breaks, and we can predict chaotic patterns.
  - The classical Ising model uses binary variables to represent spin states, but we know that spins (when not observed) can exist in a superposition of states. Our Hamiltonian here becomes
$$H = -J \sum_{\langle i,j \rangle} Z_i Z_j - h_z \sum_i Z_i - h_x \sum_i X_i$$
  - Here, we break the magnetic field into two orientations (for a lattice), the z-direction (h_z) and the x-direction (h_x). However, as we previously noted, observables in different spin bases act linearly on their eigenstates, so the X-term adds quantum effects such as superposition and entangling behavior to our system.
  - This is too much for a classical computer to keep track of, so we require a quantum computer to simulate the evolution of the system (especially as it evolves in time).

- David Deutsch introduced the universal quantum computer (1985). He showed that a finite set of quantum gates could perform arbitrary quantum (and classical) computations, much like classical logic gates. This was the first understanding that quantum logic was a legitimate computational model.

- While we understood quantum logic to be a valid logical system, the question now became: Could quantum computers solve problems faster than classical ones?(1990s).

  - The first important answer came from David Deutsch and Richard Jozsa with the Deutsch-Jozsa Algorithm.
  - They were followed by Ethan Bernstein and Umesh Vazirani, who established much of quantum complexity theory.
  - These algorithms demonstrated that quantum computers could outperform classical computers for carefully chosen problems.

- In 1994, Shor's Algorithm was unveiled as perhaps the single most important moment in the field. Peter Shor formulated a polynomial-time quantum algorithm for integer factorization.

  - You may ask: Why is this THE significant discovery? Prime numbers, after all, are a mathematician's plaything, and don't tell us much about physics or nature.
  - Mathematicians built modern cryptography off of keys and passcodes constructed from primes. After all, primes are easy to create, and (classical) computers have a difficult time breaking large numbers into their prime factors.
  - Much like a lock, it's hard to crack unless you know the password. The issue is we now have a jackhammer, trivializing the problem.
  - So what? We can just create a quantum-resistant encryption algorithm. While yes, some algorithms show progress in this regard. They are far from being implementable.

  - Thus, every text message you've sent, every piece of bank information, and every state secret becomes public with a good enough computer. The good thing is that, likely, no average person will have the funds to build one. The bad news is that corporations, multi-billionaires, and governments can.
  - This transformed quantum computing from a theoretical curiosity into a subject of a world-wide arms race.
  - But this is far from the only application, as realizable quantum computation is a gateway to advances in medicine, chemistry, and physics.

- Grover's Algorithm (1996) is a quantum search algorithm developed by Lov Grover. Instead of requiring $O(N)$ search steps, it only takes $O(\sqrt{N})$ steps. While not an exponential speedup, the algorithm applies broadly to unstructured search problems.

- We now have a set of quantum algorithms that show promise. The issue is that we now have to make a quantum computer. The biggest obstacle in the way is the qubit (quantum bit, or the data of our algorithm). Unfortunately, by virtue of quantum mechanics and decoherence, quantum states are extremely fragile. We began the quest for quantum error-correction models (Mid 1990s).

  - Many believed quantum computation was impossible because of noise. Then, Shor, Steane, and Laflamme developed quantum error-correction codes.
  - Soon after, the **threshold theorem** showed that if physical error rates are below a certain threshold, arbitrarily long quantum computations are theoretically possible using fault-tolerant error correction.

- Going into the 2000s, attention soon shifted from the development of algorithms to experimental realization. Researchers developed multiple hardware platforms, including: superconducting circuits, trapped ions, neutral atoms, photonic, and semiconductor spin qubits. Small demonstrations of quantum algorithms became possible.

- By the 2010s, quantum computations became a source of corporate interest, so major tech companies began investing heavily. Examples include: IBM, Google, Microsoft, Intel, and more. With this, cloud-accessible quantum processors became available, allowing researchers worldwide to run experiments.

- In the 2020s and looking forward, current research focuses on improving qubit fidelity, scaling computing size, implementing quantum error correction, and finding practical algorithms. Researchers are transitioning from small noisy devices (often called NISQ devices) toward fault-tolerant quantum computers capable of running deep, reliable quantum algorithms.

3 Building a Quantum Computer

3.1 Qubits

- The **bit** is the foundational piece of data in the classical computation world. It represents a binary variable with a "1" state and a "0" state. With a sequence of these binary variables, we can theoretically encode any piece of (finite) information (with enough bits).
- The **qubit** is simply quantum computing's analog. It represents a piece of quantum information, and to no one's surprise, a qubit can host a superposition of a "$|1\rangle$" eigenstate and a "$|0\rangle$" eigenstate: $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle\,;\ \alpha, \beta \in \mathbb{C}, |\alpha|^2 + |\beta|^2 = 1$. We call the basis expressed by the eigenstates **the computational basis**.
- This is not dissimilar to another two-level system we have seen, the spin state of an electron. In fact, under certain spin-state qubits, we say that $|+\rangle_z = |0\rangle$ and $|-\rangle_z = |1\rangle$.
- Another thing one can likely assume is that, like in quantum mechanics. We cannot directly "look into" the superposition, meaning when we measure the qubit, the superposition collapses into an eigenstate with the probability given by the Born rule.
- From our previous sets of notes, we know we can represent an arbitrary two-level system with the following superposition:

$$|\psi\rangle = e^{i\gamma}(cos(\frac{\theta}{2})|0\rangle + e^{i\varphi}sin(\frac{\theta}{2})|1\rangle)$$

- Again, we call the $e^{i\gamma}$ term the global phase (which is entirely ignorable) and the $e^{i\varphi}$ term the relative phase (which very much isn't).
- By familiarity with spherical coordinates, we can represent the state of a single qubit with a vector on the Bloch sphere, where θ controls the vertical angle and φ controls the azimuthal angle (Note: the $\theta/2$ ensures we don't double-cover the sphere, as if $0 \leq \theta \leq \pi/2$, we sweep out the image of a semi-circle, which is then rotated 2π to sweep out a sphere).

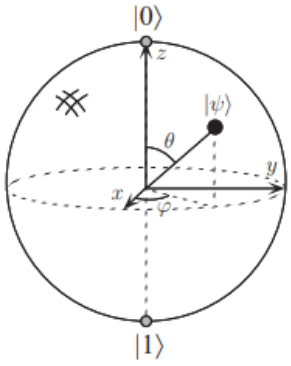


Figure 2: Bloch sphere representation of a qubit

- A qubit, while its superposition holds a continuous range, only represents finite information because, again, we can't peek into the superposition. This is the same reason why measuring a state vector in quantum mechanics doesn't yield multiple energies, but a single value.
- There exists another quantum data item in the **qudit**. Instead of being in a binary superposition, it is in an n-dimensional superposition. Often, we stick to qubits because they are easier to work with, control, and manufacture. However, we use them when we need increased information density.

- Suppose we now have two (non-entangled qubits); the number of possible states doubled: 00, 01, 10, 11. This is explicitly stated as the Composition postulate of quantum mechanics. With an n number of qubits, the number of states we can represent grows as 2^n. At $n = 500$, the dimensionality of the Hilbert space becomes larger than the number of atoms in the universe. Meaning on a classical computer, the size of the universe, it would be impossible to store all the coefficients of a 500-qubit superposition.

- Suppose we want to measure a single qubit in a two-qubit system (easily generalizable to any number of qubits). Given a generalized quantum state (with associated probabilities), our post-measurement state is simply a projection into the eigenspace of the total Hilbert space associated with that measurement; e.g., for the first qubit measuring 1: $|\psi'\rangle = (\alpha_{00}|00\rangle + \alpha_{01}|01\rangle)/(\sqrt{|\alpha_{00}|^2 + |\alpha_{01}|^2})$

- An important two-qubit state is the Bell state or the EPR pair. We have already discussed in depth the significance of this state: it is a correlated entangled state. However, in quantum computing it is necessarily one of the main places that quantum logic diverges from classical logic. It is also the key ingredient of quantum teleportation and superdense coding.

- With the scale of multiple-qubit systems and entanglement, we have an enormous computational potential. But before we have a complete picture, we must ask: How can we do computations with our qubits?

3.2 Quantum Gates

- Continuing the analogy of a classical computer, let's talk about our quantum gates. There are many important gates, such as the Pauli, CNOT, Clifford, and T gates. We will first explore single-qubit gates, then discuss many-qubit gates, and finally explain what a universal gate set is.

3.2.1 Single Qubit Gates

- Classically, the vast majority of important gates in standard computing act on two bits. There only exists one (non-trivial) single-bit gate: the NOT gate. In fact, we don't even need gates that act on more than two bits, by the **universality** of the NAND (and NOR) gate(s), meaning all logical operations can be replicated using a series of just this gate.

- Can we say the same for quantum computing? Yes, but instead of universality only needing a single gate, we require a set of them. More on that in a (qu)bit.

- Let us first consider lifting the NOT gate from classical logic into the quantum world. A gate that maps ones to zeros and zeros to ones linearly. Written as a matrix in the computational basis, it looks like

$$NOT(|0\rangle) = |1\rangle\,,\ NOT(|1\rangle) = |0\rangle \implies NOT = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

- For those of you who are quantum mechanists, this precisely looks like the Pauli-x matrix, which is precisely why we call a **quantum-NOT** (or a **Bit-flip**) gate an **X-gate**. Additionally, we can clearly see that applying two successive X gates yields an identity transform on the system.

- In general, a condition we should want for our quantum gates is that they should be norm-preserving, meaning that the probability of measuring any given state by the qubit should be one before and after applying a gate. This corresponds to the requirement that each quantum gate is a unitary operator. This is the only true requirement (other than adherence to the postulates of quantum mechanics).

- This forms another distinction between the classical and quantum worlds: there are many non-trivial single-qubit gates. We have already introduced the X-gate, but truly we can transform one state on the Bloch sphere to any other state on the Bloch sphere with a series of unitary operations.

- Now, let's ask ourselves the reasonable question: what do the Z and the Y gates do:

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}; \quad Y = iXZ = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$$

- Since we can decompose a Y-gate into a constant factor, an X-gate, and a Z-gate. We can answer that question after determining the effect of the Z-gate. Because we write the matrices in the computational basis, we see that the Z-matrix is the diagonal matrix corresponding to our eigenstates: $|0\rangle \mapsto |0\rangle$ and $|1\rangle \mapsto -|1\rangle$. Therefore, we call the **Z-gate** a **phase flip gate**.

- A Y-gate is simply a phase flip then a bit flip times a constant (sometimes we define Y=XZ, but I will avoid trying to confuse you).

- If we visualize these transformations on the Bloch sphere, we see that each of these transforms corresponds to a 180° rotation along the corresponding axis

- Another very important single qubit gate is the **Hadamard gate**:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

- Upon inspection, we can see that the gate maps $|0\rangle \mapsto (1/\sqrt{2})(|0\rangle + |1\rangle)$ and $|1\rangle \mapsto (1/\sqrt{2})(|0\rangle - |1\rangle)$. This is very much the same as a transformation of spin bases in quantum mechanics from 1/2-z spin to 1/2-x spin (and this becomes an essential understanding in the field). Often, we use this to create an equally random state from a definite state. Also, the Hadamard transform can simply be expressed as $H = (X + Z)/\sqrt{2}$

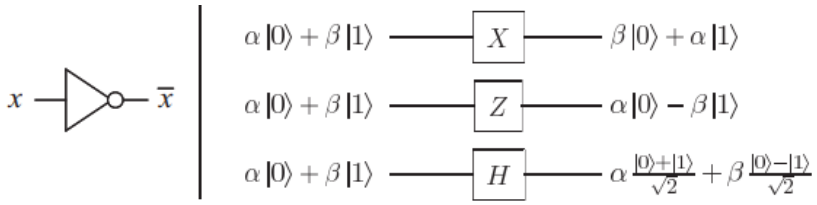


Figure 3: Single Bit vs Qubit Logic Gates

- In general, there are an infinite number of arbitrary unitary transforms, but we can decompose any given unitary transform as:

$$U = e^{i\alpha} \begin{pmatrix} e^{-i\beta/2} & 0 \\ 0 & e^{i\beta/2} \end{pmatrix} \begin{pmatrix} cos\frac{\gamma}{2} & -sin\frac{\gamma}{2} \\ sin\frac{\gamma}{2} & cos\frac{\gamma}{2} \end{pmatrix} \begin{pmatrix} e^{-i\delta/2} & 0 \\ 0 & e^{i\delta/2} \end{pmatrix}$$

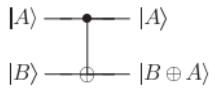
- This is precisely the Euler angle decomposition (often used in rotational kinematics). This is simply a rotation about the Z-axis δ, then a rotation about the y-axis by γ, then another rotation about the z-axis by β. This can be used to continuously parameterize a map of one orthonormal basis onto another orthonormal basis on the Bloch sphere.

3.2.2 Multi-Qubit Gates

- Obviously, single-qubit gates aren't enough to do computations, so we must add multi-qubit gates.

- As usual, let us first begin in the classical world. We have a few widely identifiable logical gates: AND, OR, XOR, NAND, and NOR gates. These are all two-bit gates, and, as we have already stated, NAND and NOR are universal.

- Alternatively, we have the controlled-NOT gate (or cNOT) as the prototypical multi-qubit quantum logic gate. This gate has two input qubits: the control (top) and the target (bottom) qubits.



- We can define the action of this gate (like with classical two-bit gates) as an "if" statement: If the control is set to 0, then the target qubit is left alone, but if the control is 1, then an X-gate is applied to the target qubit. Like all quantum gates, this is applied linearly. Hence, we get the following linear transform.

$$cNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

- In effect, the cNOT's action eigenstates take the sum of both the labels (0 or 1) and take the remainder (modulo) 2, applying it to the target qubit. As is a common trend (but not law), the cNOT is clearly an involution. This also marks another difference in the classical and quantum world: by virtue of being unitary, each quantum gate must be invertible, but that is not shared for classical computing.

- There are, of course, more than one multi-qubit gate, such as the Toffoli gate, but we can show that any multi-qubit logic gate may be composed of a cNOT and single-qubit gates.

- The swap gate can be expressed with a sequence of three cNOT gates. You may be asking, isn't a cNOT its own inverse, so wouldn't they cancel? That is a good question; however, the target qubit flips each time, so it doesn't cancel

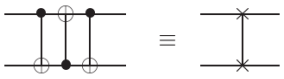


Figure 4: SWAP gate

- The mapping of the swap gate is given by $|a, b\rangle \mapsto |b, a\rangle$. Broken down step by step, $|a, b\rangle \mapsto |a, b \oplus a\rangle \mapsto |a \oplus (a \oplus b), a \oplus b\rangle = |b, a \oplus b\rangle \mapsto |b, (a \oplus b) \oplus b\rangle = |b, a\rangle$

- As cNOTs are controlled NOT gates, we can easily write an arbitrary controlled unitary gate, with the lower right block matrix as any unitary operator.

3.3 Let's Talk Initialization and Measurement

- **Initialization:** Before computation begins, all qubits are prepared in a well-known state, typically $|0\rangle^{\otimes n}$ (all qubits initialized in the $|0\rangle$ state). Reliable initialization provides a well-defined starting point for quantum algorithms. Also, we can quite easily put this into a superposition state by using a Hadamard gate to add "quantumness" into our state.

- **State Preparation**: Quantum gates are applied to the initialized qubits to create the desired input state, often involving superposition and entanglement that encode the problem to be solved.

- **Measurement**: At the end of a computation, qubits are measured in a chosen basis $\{|0\rangle, |1\rangle\}$, causing the quantum state to collapse into one of the basis states.

- The result of a measurement is inherently probabilistic. Because a single measurement yields one random sample from the output distribution, quantum algorithms are often executed many times to estimate/infer the correct computational result with high confidence.

- Note: while we have wires to move electrical signals between gates in classical computing. In quantum computing, it's often much a much more complicated to operate (and move) qubits. So, think of the wires in a quantum circuit diagram as more of a flow chart than a physical schematic of the procedure.

- Now, we have everything we need to build a quantum circuit. A data object (qubits), a start point (initialization), an endpoint (measurement), and steps to go from beginning to end (gates).
- Before we move on, we should cover a key theorem of quantum mechanics (and computing).
- **No Cloning Theorem:** an arbitrary unknown quantum state cannot be copied exactly by any quantum operation, preventing the duplication of qubits during computation. This stems from the fact that cloning is a nonlinear operation, but all physically allowed quantum evolutions are linear (unitary).
- This is a massive roadblock and motivator for quantum error correction, as we can't save "copies" of a state. Therefore, we have to find a way to actively revise our state and correct for errors.

3.4 Entanglement and Quantum Teleportation

3.4.1 Creating the Bell State

- As we discussed previously, one of the most important aspects of quantum mechanics arises from entanglement. Can we replicate this phenomenon in quantum computing?
- Consider the circuit shown below: it is a Hadamard gate followed by a cNOT gate. This basic quantum circuit produces Bell states. The quintessential entanglement state:

In	Out
$\|00\rangle$	$(\|00\rangle + \|11\rangle)/\sqrt{2} \equiv \|\beta_{00}\rangle$
$\|01\rangle$	$(\|01\rangle + \|10\rangle)/\sqrt{2} \equiv \|\beta_{01}\rangle$
$\|10\rangle$	$(\|00\rangle - \|11\rangle)/\sqrt{2} \equiv \|\beta_{10}\rangle$
$\|11\rangle$	$(\|01\rangle - \|10\rangle)/\sqrt{2} \equiv \|\beta_{11}\rangle$

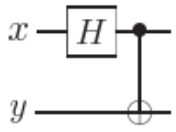


Figure 5: Quantum Circuit Used to Create Bell States

- These states are the foundations of many quantum algorithms/processes. Let's take a look at an example below.

3.4.2 Quantum Teleportation

- Quantum teleportation works by transferring the precise quantum state of one particle to another distant particle, using a shared entangled pair and classical communication. It uses one shared Bell pair and two classical bits of information (as well as the state you want to encode).

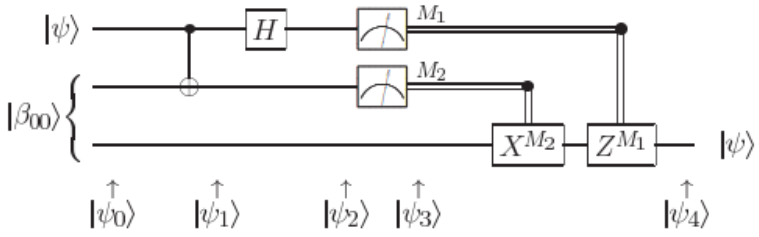


Figure 6: Quantum Teleportation Circuit

- Teleportation begins with Alice and Bob already sharing a Bell pair. Usually, it is $|\beta_{00}\rangle = (1/\sqrt{2})(|00\rangle + |11\rangle)$. Alice owns an unknown qubit and one half of the Bell pair; Bob owns the other half. Thus, the initial state of our system is represented by $|\psi_0\rangle = |\psi\rangle \otimes |\beta_{00}\rangle$

- Then, Alice performs a cNOT with the unknown pair as the control and her Bell pair as the target. Thus, we start "mixing" the unknown information into the shared entangled pair, meaning entanglement spreads information over several particles.

$$|\psi_1\rangle = \frac{1}{\sqrt{2}}[\alpha\,|0\rangle\,(|00\rangle + |11\rangle) + \beta\,|1\rangle\,(|10\rangle + |01\rangle)]$$

- Next, Alice applies a Hadamard on the unknown qubit, yielding:

$$|\psi_2\rangle = \frac{1}{2}[\alpha(|0\rangle + |1\rangle)(|00\rangle + |11\rangle) + \beta(|0\rangle - |1\rangle)(|10\rangle + |01\rangle)]$$

$$= \frac{1}{2}[|00\rangle\,(\alpha\,|0\rangle + \beta\,|1\rangle) + |01\rangle\,(\alpha\,|1\rangle + \beta\,|0\rangle) + |10\rangle\,(\alpha\,|0\rangle - \beta\,|1\rangle) + |11\rangle\,(\alpha\,|1\rangle - \beta\,|0\rangle)]$$

- This expression breaks down into four terms. The first term has Alice's qubits in the state $|00\rangle$, and Bob's qubit in the state $\alpha\,|0\rangle + \beta\,|1\rangle$ (the original state $|\psi\rangle$). If Alice performs a measurement and obtains the result 00, then Bob's system will be in the state $|\psi\rangle$. Given the result of Alice's measurement, we obtain:

$$00 \mapsto |\psi_3(00)\rangle = \alpha\,|0\rangle + \beta\,|1\rangle\,;\ 01 \mapsto |\psi_3(01)\rangle = \alpha\,|1\rangle + \beta\,|0\rangle\,;$$

$$10 \mapsto |\psi_3(11)\rangle = \alpha\,|0\rangle - \beta\,|1;\ 11 \mapsto |\psi_3(11)\rangle = \alpha\,|1\rangle - \beta\,|0\rangle\rangle$$

- Depending on the result of Alice's measurement, Bob's qubit will end up in one of these states. Of course, to know which, Bob must know the result of Alice's measurement. Thus, preventing the spread of quantum information by requiring the spread of classical information.

- To replicate the original state, Bob (knowing Alice's measurement) must either use the identity, an X gate, a Z gate, or an X then Z gate. Thus, we have "teleported" a state from one observer to another.

- This doesn't violate the no-cloning theorem, since Alice must measure her superposition to "teleport" the information to Bob, so there only ever exists one such superposition state.

3.4.3 Superdense Coding

- Superdense coding is quite effectively the reverse protocol. Instead of wanting to teleport a qubit, Alice wants to send two classical bits. Normally, this requires sending two physical bits, but quantum mechanics can allow her to do it in one (hence the name superdense).

- Again, Alice and Bob start with the Bell state $|\beta_{00}\rangle = (1/\sqrt{2})(|00\rangle + |11\rangle)$. Alice possesses one qubit, and Bob possesses the other.

- Alice wants to send some message encoded in two bits (00, 01, 10, 11). She then applies the respective (to Alice's measurement in quantum teleportation) inverse of the gates Bob applied to decode the previous superposition.

- These will put the Bell pair into the four orthogonal Bell states: $I \mapsto |\beta_{00}\rangle\,,\ X \mapsto |\beta_{01}\rangle\,,\ Z \mapsto |\beta_{10}\rangle\,,\ XZ \mapsto |\beta_{11}\rangle$. Thus, Alice has encoded two classical bits into one qubit of an entangled pair.

- Alice then sends her qubit to Bob (only one qubit travels), but transported at the classical limit. Bob, with both halves, performs a CNOT, then a Hadamard (inverse of the Bell state), and the four Bell states become the definite states $|00\rangle\,,\ |01\rangle\,,\ |10\rangle\,,\ |11\rangle$. Once Bob measures, then he knows Alice's two classical bits.

- We can double-stack the amount of information sent due to the long-range capabilities of entanglement. We have two quantum bits as well; one was already delivered before they even started.

3.4.4 Quantum Key Distribution and a First Look at Quantum Communication

- We have already explained why quantum computing is scary. Nearly every secure communication relies on encryption methods that use the factoring of large numbers or discrete logarithms, and quantum computing turns these difficult problems into a cakewalk. A current subject of interest to many in the field is constructing quantum-resistant encryption.
- Let's, as always, understand how classical communication occurs in the age of the internet:
  - Encryption generally consists of two parts: generating a secret key and using that key to encrypt/decrypt messages.
  - There are two main forms of Classical key distribution: symmetric cryptography and public-key cryptography. Symmetric cryptography holds that Alice and Bob already share a secret key that they predetermined before communicating (e.g., they met in person and agreed on a valid key for their conversation).
  - Public-key cryptography includes algorithms like RSA (Rivest-Shamir-Adleman), Diffie-Hellman, and Elliptic Curve Cryptography. For almost everything on the internet, we rely on public-key encryption.
- While strong, this classical encryption method would be broken if we develop a sufficiently capable quantum computer. Though the answer may be connected to the very cause of our issue. How can we use quantum mechanics to ensure we don't have any bad actors (given a few caveats)? The tools we have at our disposal are: measurement collapses our system, the no-cloning theorem, and that non-orthogonal states cannot be perfectly distinguished.
- The good thing is that public-key methods of encryption are vulnerable. Symmetric key distribution isn't. Thus, we only need to replace the key exchange stage with QKD; all else remains the same. The only issue is how we create an algorithmic passcode that is quantum-(and classically)-resistant.
- QKD requires two channels: a classical authenticated channel (a classical channel where people can't pose as different people) and a quantum channel (carries qubits).
- The first QKD protocol was introduced in 1948 (by Bennett and Brassard), and it's called the BB84 protocol.
- For this algorithm, Alice randomly chooses a string of bits expressed randomly in either the Z (computational) or X (Hadamard) bases. Then, she sends each photon to Bob. For every photon, Bob guesses the basis Alice encoded the state in.
- The key is encoded in the fact that some of the guesses are correct and some of the guesses are incorrect. After transmission, on the classical channel, Alice declares which basis she used for each photon (not the bit, only X or Z), and Bob declares which basis he measured in. Whenever the bases disagree, they discard that photon. This process is called **sifting**. We get that roughly half of the photons survive. Now, the remaining bits become the raw key that both Bob and Alice know of. Then, we simply use symmetric key distribution.
- So this is clearly quantum-proof since they both use guessing to create a key. However, we want to know why this is a safe platform (i.e., what prevents others from peeking)?
- Suppose Eve intercepts every photon. She doesn't know what basis Alice used, so she must guess (half correct, half incorrect). She has destroyed the original state.
- E.g., suppose Alice sends $|+\rangle$ and Eve incorrectly guesses the Z-basis measurement; she collapses the superposition. If Bob then tries to measure correctly in the X basis, the statistics are altered, and errors appear. The probability of revealing the tampering is 1/8 (1/2 chance Eve measures wrong basis, 1/2 chance Bob guesses the correct basis, 1/2 chance we get the wrong bit).
- Now, if Alice and Bob compare a random subset of their bits, and the error rate exceeds a certain threshold (put in place since quantum channels are noisy), they abort and assume the channel has been compromised.

- For added privacy, suppose they apply a universal hash function (a technique where, instead of using one fixed formula to sort data, you pick a formula at random from a pre-approved "family" of formulas). They compress 1000 bits to 256 bits of information. The resulting key is uniformly random and essentially unknown to Eve.

- So have we solved it? No, not quite. While this method is very powerful, it doesn't work if the users are unsure if they are talking directly to one another. If Eve intercepts the communication of Alice and Bob and starts two independent QKD sessions. Alice and Bob think they are talking without realizing there is a man in the middle.

3.5 Quantum Logic

- Now that we have gotten a first look at our tools in quantum computing, let us ask ourselves a question: what makes quantum computing better, as we claim, at certain tasks?

- Let's first make note of the fact that, while quantum computers are very skilled at certain tasks, they tend to be much slower at others, such as deterministic computation (binary math). This is due to both hardware and the nature of the computing method. Quantum computing isn't a "faster" computer; it leverages a different logic system, one that is better optimized for certain problems. This is why quantum computers are always paired with a classical computer, so we get the best of both worlds.

- Moreover, we do not know that there doesn't exist a classical algorithm that can do "exponential problems" (like factoring primes) in polynomial time. There may be one developed in the future, but we do not know of it or its existence. So, quantum computing is the only known way to do certain subsets of these problems in polynomial time (e.g. Shor's algorithm).

- To answer the question posed, the reason why quantum logic outperforms (in some regards), classical logic rests on three structural features of quantum mechanics cooperating. None of the three is sufficient alone, and each one closes off a different route back to classical tractability: Superpositions & Linearity, Interference, and Entanglement.

3.5.1 Superposition and Linearity

- As we have discussed, a classical register of n bits occupies one of 2^n states at a given time. A quantum register of n qubits allows us to now be in a superposition of all 2^n basis states at once, with a given complex amplitude associated with each: $|\psi\rangle = \sum c_x |x\rangle$

- Quantum gates are also unitary, and thus of course linear operators. Applying a gate to a superposition acts on every branch (or eigenstate) at once. This is loosely called quantum parallelism; one circuit application evaluates a function on all 2^n inputs in a superposition.

- This alone does not give a speedup because measurement will only return a single state (or rather a projection onto an eigenvalue) and collapses the rest of the superposition, meaning we cannot read all 2^n results. That is, while a single unitary gate will act linearly on a superposition, we can't do multiple distinct calculations (on the eigenstates) at once because measurement destroys the superposition.

- Superposition therefore supplies an exponentially large computation space, but by itself gives no way to extract a useful answer from it. For our answer, we look to our next section, interference.

3.5.2 Interference

- The next property we will examine looks within the superposition, the complex amplitudes of each eigenstate. Because our amplitudes are complex, when computational paths recombine (like with cNOTs), their amplitudes add, not their probabilities. This may lead to paths that reinforce (constructive interference) or cancel (destructive interference), unlike classical probabilities, which only ever add.

- This becomes the engine behind our quantum speedups. The VERY general algorithmic design philosophy behind any quantum algorithm is: prepare a superposition, act specific unitary operators so amplitudes interfere as we desire, measure, then repeat.
- In fact, we will discuss this very property when we take a closer look at the Deutsch-Josza, Grover, and Shor algorithms.
- Without interference, superpositions are computationally inert, since we would have no mechanism by which to bias amplitudes toward a correct "answer." At that point, we might as well randomly guess.

3.5.3 Entanglement

- A system of n unentangled qubits is a product state, $|\psi\rangle = |\psi_1\rangle \otimes |\psi_2\rangle \otimes \cdots \otimes |\psi_n\rangle$. Even though this state lives in a 2^n-dimensional Hilbert space, it is fully specified by only $O(n)$ real parameters (2 complex numbers, 4 real, for each qubit's superposition), and a classical computer can simulate it efficiently by tracking each qubit independently.
- This is because the set of separable states is a very small subset of the total Hilbert space. Introducing inseparable states, or entanglement, gives us access to the rest of the Hilbert space.
- A generic entangled state within the Hilbert space requires the full 2^n-dimensional amplitude vector to describe.
- The Jozsa-Linden theorem shows that pure-state quantum computations with sufficient entanglement cannot be simulated in polynomial time, which is precisely why entanglement is a necessary pillar of super-polynomial speedups.

4 A First Look at Quantum Algorithms and Quantum Information Theory

4.1 Classical Computations on a Quantum Computer

- Let us ask if quantum computing is Turing-complete: Can we replicate any classical logic circuit using a quantum computer? Absolutely. In fact, if quantum mechanics can be used to explain all of physical reality, as physicists claim, we better hope it should.
- We can easily replicate a standard bit by simply considering only the eigenstates of the computational basis ($|0\rangle = 0,\ |1\rangle = 1$) and try avoiding gates that map eigenstates to superposition states (e.g. Hadamard gates).
- In fact, it suffices to show that we can create a NAND gate, since it is a universal gate. To do so, we will use a Toffoli gate. While construction often uses T-gates or Hadamard gates, the gate maps eigenstates to eigenstates. The schematic and truth table are as follows:

Inputs			Outputs		
a	b	c	a'	b'	c'
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

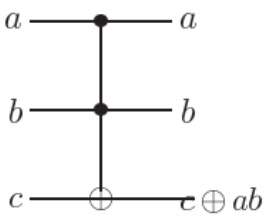


Figure 7: Truth Table for the Toffoli Gate and its Circuit Representation

- If we were to look only at the values for which $c = 1$, and use a and b as our binary inputs, we can see that $c' = c \oplus ab = 1 \oplus ab = \neg(ab)$, which is precisely the output of a NAND gate.
- To simulate a non-deterministic classical computer, we need a way to simulate randomness. To noone's surprise, quantum computing is great at this. We can use a sequence of Hadamard gates to simulate a sequence of 50/50's (which can be used to generate any finite number).

4.2 Quantum Parallelism and the n-Hadamard

- **Quantum Parallelism** is a subject that we have discussed in our quantum logic section, and it is a feature of many (virtually every) quantum algorithms. This property allows quantum computers to evaluate a function for many different input values simultaneously.
- Suppose that we have a function $f(x) : \{0, 1\} \to \{0, 1\}$; it has a one-bit domain and range. With an appropriate sequence of logic gates, we can transform a state $|x, y\rangle$ into $|x, y \oplus f(x)\rangle$, calling this transformation U_f. Note: if $y = 0$, then $y' = f(x)$.
- If we input the state $x = (1/\sqrt{2})(|0\rangle + |1\rangle), y = 0$, then the output state $|\psi\rangle = U_f\,|x, y\rangle = (1/\sqrt{2})(|0, f(0)\rangle + |1, f(1)\rangle)$. Thus, with a single use of a circuit, we can evaluate the function f(x) on two states, $|0\rangle, |1\rangle$, at once. Again, as a reminder, we will only be able to see the output of one of these mappings if we were to measure (since it is in a superposition state).
- This procedure can be easily generalized to a function of an arbitrary number of bits by using an operation called the **n-Hadamard transform**. This is simply n Hadamard gates acting in parallel on n-qubits. This (if acted on a set of n-qubits initialized to $|0\rangle$) yields the following state of the system: $|\psi'\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$, where $|x\rangle$ iterates all eigenstates of our combined Hilbert space. We write $H^{\otimes n}$ to denote this action.

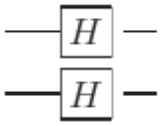


Figure 8: The Hadamard Transform on Two Qubits

- We can generalize our two-state parallelism for n qubits with n+1 total qubits. Prepare the system in the state $|0\rangle^{\otimes n}|0\rangle$. Then, act the n-Hadamard transform on the first n qubits, followed by acting U_f on the first n qubits as the first input and the last qubit as the second input. This will produce the state in the last qubit: $(1/\sqrt{2^n})\sum_x |x\rangle |f(x)\rangle$
- Thus, we have evaluated the function f(x) on n items at once. You may be wondering: Why should we care? If measuring will collapse the superposition to a single value, don't we lose $n-1$ pieces of information? We will begin to unravel this question soon with the Deutsch-Jozsa Algorithm.

4.3 The Deutsch-Jozsa Algorithm

- A simple alteration to the system in the previous section will demonstrate precisely how quantum systems can outperform classical systems by implementing the Deutsch-Jozsa algorithm. This algorithm combines parallelism with interference.
- Problem: Alice, in New York City, selects a number from 0 to 2^{n-1}, and mails it to Bob in Cambridge. Bob calculates some function f(x) and replies with the result 0 or 1. Bob promised to use an **oracle** function f, which is either constant (acts the same for all values of x) or balanced (half gives one, the other half gives 0), but doesn't tell Alice. Alice wants to determine with certainty which function he is using.
- Note: we call certain unitary gates/functions **black boxes** or **oracles** when we don't know or care about how such a gate is implemented. We just care about its action on our state.
- In the classical case, Alice may only send Bob one value of x in each message, so in the worst case, she needs to send $2^n/2+1$ messages/queries. Note: in each message, Alice sends Bob n bits of information (since there are $\geq log_2(2^n)$ bits of information needed to send a number between 0 and 2^n-1).
- If Bob and Alice decided to exchange qubits instead, and Bob agreed to turn the function $f(x)$ into a unitary operator U_f, which returns the transformed qubit. Then, we can figure out the type of oracle function with a single message exchange.
- Suppose we had the following circuit, starting with the state of the system being $|\psi_0\rangle = |0\rangle^{\otimes n}|1\rangle$

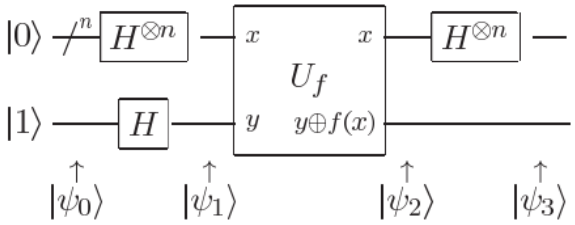


Figure 9: Deutsch-Josza Algorithm Circuit

- After the first set of Hadamards, we have, by basic calculation, the state of the system:

$$|\psi\rangle_1 = \sum_{x\in\{0,1\}^n} \frac{|x\rangle}{\sqrt{2^n}}[\frac{|0\rangle - |1\rangle}{\sqrt{2}}]$$

- We are now in a superposition of all values of the state, and the answer register is an evenly weighted superposition of 0 and 1. Next, Alice sends this state to Bob to throw into his unitary oracle function. He gets the state

$$|\psi_2\rangle = \sum_x \frac{(-1)^{f(x)}|x\rangle}{\sqrt{2^n}}[\frac{|0\rangle - |1\rangle}{\sqrt{2}}]$$

- Now, Bob sends this superposition (without measurement) to Alice. She now has a set of qubits with the result of Bob's function evaluation stored in the amplitude of the superposition. She now applies a Hadamard to the first n qubits. The notation becomes really messy here, so let us look at a single case of the state $|x\rangle = |1\rangle$ or $|0\rangle$. We see that $H|x\rangle = \sum_z (-1)^{xz}|z\rangle/\sqrt{2}$. Thus, for

$$H^{\otimes n}|x\rangle = \frac{\sum_z (-1)^{x\cdot z}|z\rangle}{\sqrt{2^n}}$$

- Here, $x \cdot z$ is the dot product of the string x and z modulo two (even or odd overlap). So, we have

$$|\psi_3\rangle = \sum_z \sum_x \frac{(-1)^{x\cdot z + f(x)}|z\rangle}{2^n}[\frac{|0\rangle - |1\rangle}{\sqrt{2}}]$$

- Now, we measure the state. We will get one of two cases. If f is constant, the amplitude for $|0\rangle^{\otimes n}$ is either 1 (or -1, depending if its constantly 0 or 1). Since the state must have unit norm, this means all other amplitudes must be zero, so a constant oracle will measure $|0\rangle$ on the first n qubits. If f is balanced, then the positive and negative contributions for $|0\rangle^{\otimes n}$ cancel. Thus, for the state to have unit norm, we have that at least one (of the first n) qubits will measure $|1\rangle$.

- In summary, we measure the first n qubits. If there is a qubit that measures $|1\rangle$, f must be balanced. Otherwise, f must be constant.

- Therefore, we have determined a quantum algorithm that provides an exponential speedup to solve a given problem. Despite how convoluted this is, this algorithm provides the foundations to build more impressive quantum algorithms.

4.3.1 The Bernstein-Vazirani Algorithm

- This algorithm is an example of a use case of the Deutsch-Josza algorithm (though still a toy example), and it is only a slight deviation from the logic.

- Suppose someone secretly chooses an n-bit string: $s = s_1 s_2 ... s_n$. Here, our oracle function, instead of being either constant or balanced, evaluates $f(x) = s \cdot x \ (mod\ 2) = (s_1 x_1 + s_2 x_2 + ... s_n x_n)(mod\ 2)$. Using this oracle, we want to determine s.

- To recover every bit of s, we would ask for $f(10...0) = s_1, f(010...0) = s_2, ...$ So we would expect to query the oracle n times. Quantum computers can evaluate the oracle on every input simultaneously using superposition, allowing the algorithm to discover the code word with only one oracle call.

- We follow the EXACT same circuit of Deutsch-Josza, but simply swap the oracle functions: Initialize $|0\rangle^{\otimes n}|1\rangle$, Hadamard each qubit, Access the Oracle ($|x, y\rangle \mapsto |x, y \oplus f(x)\rangle$, Apply Hadamards to the first n qubits.

- The math traces:

  1. $|0\rangle^{\otimes n}|1\rangle$
  2. $\frac{1}{\sqrt{2}}\sum_x |x\rangle|-\rangle$
  3. $|x, y \oplus f(x)\rangle \implies |-\rangle \mapsto (-1)^{f(x)}|-\rangle$, so we have the total system state is $\frac{1}{\sqrt{2}}\sum_x (-1)^{s\cdot x}|x\rangle|-\rangle$
  4. $H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}}\sum_y (-1)^{x\cdot y}|y\rangle$, so we get the state of the first n-qubits is $\frac{1}{2^n}\sum_y \sum_x (-1)^{x\cdot(s\oplus y)}|y\rangle$ (we don't care about the last state because it remains in the $|-\rangle$ state)

5. The key identity here is that $\sum_x (-1)^{x \cdot z} = 2^n$ if $z = 0$ and $\sum_x (-1)^{x \cdot z} = 0$ otherwise. Here, $z = s \oplus y$, so only the $y = s$ state remains since every other y cancels by destructive interference. Thus, measuring our state will always yield the output string.

- The algorithm exploits the fact that Hadamard transforms map phase information into computational basis information. (X to Z basis)

4.4 A Summary of Quantum Algorithms

- Broadly, three classes of quantum algorithms provide a known speedup compared to classical computers: Quantum-Fourier-Based, Quantum Search, and Simulation Algorithms. The Deutsch-Jozsa algorithm falls into the first category, as well as Shor's algorithm for factoring.

- **Quantum Fourier Transforms**:

  - The discrete Fourier transform is a famous result in mathematics: it transforms a set of N complex numbers $\{x_0, ..., x_{N-1}\}$ into another set of N complex numbers $\{y_0, ..., y_{N-1}\}$ defined by:

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{(2\pi i)jk/N} x_j$$

  - This has a wide variety of use cases across physics, electronics, technology, and beyond. The Hadamard transform of Deutsch-Jozsa is actually an example of this generalized class of Fourier transforms. We lift the classical Fourier transform to a quantum version using the operator U on n qubits by its action on the computational basis:

$$|j\rangle \mapsto U|j\rangle = \frac{1}{\sqrt{2^n}} \sum_{k=0}^{2^n-1} e^{(2\pi i)jk/2^n} |k\rangle$$

  - This is a unitary operation (and implementable as a quantum circuit). Additionally, written in its action on superpositions, we see that it corresponds to a vector notation for the classical Fourier transform when N=2^n:

$$\sum_{j=0}^{2^n-1} x_j |j\rangle = \sum_{k=0}^{2^n-1} y_k |k\rangle$$

  - A classical Fourier transform requires $Nlog(N) = n2^n$ steps, where as the QFT requires $log^2(N) = n^2$ steps.
  - The QFT is fundamentally important because it translates hidden information, such as periodicities and phases embedded in quantum states, into measurable amplitudes. This allows quantum algorithms to solve complex problems exponentially faster than classical computers.

- **Quantum Search Algorithms**:

  - The representative quantum algorithm in this category is Grover's Algorithm. It gives a quadratic speedup to search through a non-sorted database. Classically, it would require $O(N)$ steps, but we get that down to only $O(\sqrt{N})$. While not an exponential speedup, it is still substantial.
  - We maintain interest in this algorithm as it has a wide range of applications across multiple disciplines.

- **Quantum Simulations**:

  - As we mentioned earlier in this paper, quantum computing was first developed as a method to simulate quantum systems. Classical computers have immense difficulty in examining quantum behaviors, as the information storage required to model a quantum system grows exponentially, so we should use a quantum system to model a quantum system.

- Note: A crucial step in developing a useful simulation is finding a way to efficiently extract desired answers. This is very difficult and only partially understood since with any measurement, we collapse our wave function.
- Despite this, quantum simulation remains one of the most important applications of quantum computers. Building faster and more accurate quantum simulations will enable advances in fields whose understanding is bound by quantum phenomena.
- Quantum simulations give rise to an intriguing (and optimistic) corollary to **Moore's Law**. Classically, this law states that the power of classical computers will double once every two or so years, for constant cost. This means that for a quantum computer to outpace the storage of a classical computer, we simply need to be able to add one qubit less than every two years.

4.5 Complexity Classes

- The question still floating in the air is: How strong are quantum computers? Truthfully, we don't know. This doesn't mean you should treat your day-trading uncle's hyperbolic answer with the same validity as a quantum researcher, but we don't have a boundary for computational complexity.
- **Computational Complexity Theory** studies classifying the difficulty of many computational problems, both classical and quantum. The most basic idea is that of a **complexity class**. We can think of it as a collection of computational problems that share a feature of the computational resources needed to solve these problems.
- Two such classes are **P** and **NP** problems: **P** is the class of computational problems which can be solved in polynomial time on a classical computer, and **NP** is the class of computational problems which can be verified in polynomial time on a classical computer. An example that highlights this distinction is the question of factoring numbers. It is very easy to check that two numbers are factors of a number (**NP**), but as we currently know, there is no way to factor a number in polynomial time (not **P**).
- P must be a subset of **NP**, but we don't know if it is proper containment or not. This is one of the most important unsolved problems in all of theoretical computer science. Many researchers believe it must be the case, but without solid evidence.
- Additionally, it is not known where quantum computers can be used to quickly solve every **NP** problem. While it can obviously do a subset (like factoring, which people believe is **NP** but not **P**), we don't know if there exists such an algorithm for the entire computational class.
- An **NP-hard problem** is a type of computational problem that is "at least as hard" as the hardest problems in NP (It doesn't have to be in NP, just that it has to be as hard as the hardest NP problems). It means there is no known fast (polynomial-time) method to guarantee a perfect solution, and it is highly likely that no such perfect method will ever be found. We say a problem is **NP-complete** if it is among the hardest problems within the NP class (it is NP-Hard and in the NP class).
- Another class of problems is the **PSPACE** class: set of problems that consists of problems that can be solved in finite space, but can take "a long time" to solve. **PSPACE** is believed to be a bigger class than both **P** and **NP** intuitively, but, again, it hasn't been proven.
- But what about quantum complexity classes? We call **BQP** the class of all computational problems that can be efficiently solved by a quantum computer. Where exactly **BQP** falls in relation to **PSPACE**, **P**, and **NP**. We have already proven earlier that all quantum computers can run classical algorithms in polynomial time ($\mathbf{P} \subset \mathbf{BQP}$), but we don't know where it falls between **PSPACE** and **NP**. This is highlighted by the drawing below:
- This ends our discussion, for now, on computational complexity. While not the most concretely driven section, it is nice to see how the problems we are aiming to solve relate to one another.

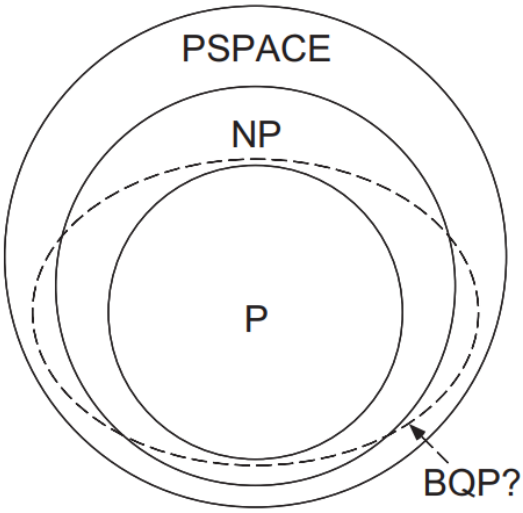


Figure 10: Relationship of Computational Complexity Classes

5 Quantum Circuits

- We have already gotten a good overview of what quantum computing is and what it entails. Now, we begin our deep dive into the weeds and details.

5.1 Summary of Quantum Operations, Thus Far

- We stated that there are two main categories of operations (gates) in quantum computing: single-qubit operations and multi-qubit operations. Let this section serve as a refresher.

5.1.1 Single Qubit Gates

- As a summary of single-qubit gates:
- A single qubit is a vector parameterized by two complex numbers, related to the probability of measuring $|0\rangle$ and $|1\rangle$ by the Born rule.
- Operations on a qubit must preserve norm and are thus unitary. Since we are restricting, temporarily, our gaze to single-qubit operations, they are represented by 2×2 matrices. In general, for n-qubit systems, they are $2^n \times 2^n$ matrices.
- Of these, there are six "main" single-qubit gates we concern ourselves with (two of which we are new to our discussion):

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix};\ Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix};\ Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix};\ S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix};\ T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

- The phase gate (or sometimes called the S gate) and the $\pi/8$ gate (often called the T gate). These are simply rotations by $\pi/2$ and $\pi/4$ around the Z-axis, respectively. You may be curious as to why we call the T gate the $\pi/8$ gate if it's a rotation of $\pi/4$. The answer is that we can factor out a global phase of $e^{i\pi/8}$; the same can be said for the S gate.
- There are a few algebraic identities, namely: $H = \frac{X+Z}{\sqrt{2}};\ S = T^2;\ Z = S^2$.

5.1.2 Controlled Gates (Multiqubit Gates)

- The controlled operation boils down to an if statement: "If A is true, then do B." We have already introduced the cNOT, a two-qubit gate, with one control (A) and one target (what we do B on). We define its action as $|c\rangle |t\rangle \mapsto |c\rangle |t \oplus c\rangle$. The matrix representation is

$$cNOT = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

- We can generalize this to an arbitrary single-qubit unitary operation, say U. The controlled-U operation has the action $|c\rangle |t\rangle \mapsto |c\rangle U^c |t\rangle$. We can easily recover the cNOT gate if $U = X$. Written as a block matrix (I is the 2×2 identity, and 0 is the 2×2 zero matrix), the controlled U gate is:

$$\textit{controlled-}U = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix}$$

- The **Toffoli gate** is a 3-qubit operator; it is like a cNOT gate in its behavior, but it has two control qubits (both need to be one to flip) and a single target. It has the following action: $|c_1\rangle |c_2\rangle |t\rangle \mapsto |c_1\rangle |c_2\rangle |t \oplus (c_1 \bigcap c_2)\rangle$. We can construct the Toffoli gate from a sequence of Hadamards, T and S gates, and cNOTS. Though the design is a bit complicated:

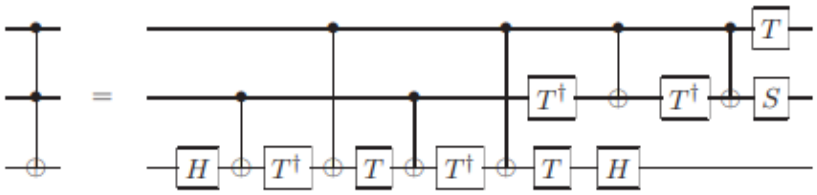


Figure 11: An Implementation of the Toffoli Gate

- A generalizable circuit for any unitary-(U)-based Toffoli is given below. Here, we say $V^2 = U$. Moreover, if we let $V = (1 - i)(I + iX)/2$, we recover a standard Toffoli gate:

- We can show every unitary operator U acting on n qubits can be exactly implemented using only single-qubit unitary gates. Let $N = 2^n$. Then U is an $N \times N$ unitary matrix.

  1. Using similar ideas to Gaussian elimination, one can successively eliminate off-diagonal entries of U. Each elimination acts nontrivially on only two computational basis states $|i\rangle, |j\rangle$, leaving the other basis vectors unchanged. Thus, $U = T_1 T_2 ... T_m$, where T_k is a two-level unitary (acts on a 2d subspace).
  2. Suppose T mixes basis states $|x\rangle, |y\rangle$. If x and y differ in several bits, apply cNOT gates to map these basis states to two states differing in only one qubit. $|x\rangle \mapsto |0...0\rangle$, $|y\rangle \mapsto |0...01\rangle$. This moves the desired T operation to act only on the final qubit, conditioned on all remaining qubits being fixed. Thus, $T = P^{-1}(C^{n-1}(V))P$, where P is a circuit of cNOTs, $V \in U(2)$ acts on one qubit, $C^{n-1}(V)$ denotes a multiply-controlled one-qubit unitary.

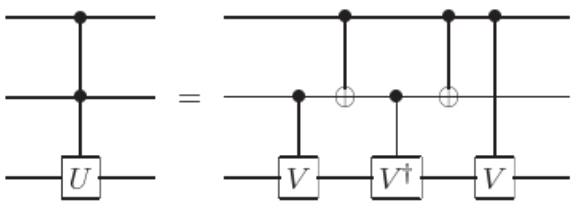


Figure 12: Generalized Unitary Toffoli Gate Constuction

3. Here, a **multiply-controlled one-qubit unitary** is a quantum logic operation where a specific single-qubit unitary transformation U is applied to a target qubit if and only if a set of multiple control qubits are all in the $|1\rangle$ state, e.g Toffoli.
4. A multiply-controlled one-qubit unitary can itself be decomposed into a finite sequence of single-qubit unitary gates and cNOTs. Through expressing arbitrary multi-qubit operations as controlled one-qubit operations (after suitable basis changes), these two ingredients are enough to build any quantum circuit.
5. Thus, we can decompose an arbitrary unitary operation on n qubits with cNOTs and unitaries.

- **A few key properties** (C=cNOT here has qubit 1 as a control and qubit 2 as a target):

$$CX_1C = X_1X_2; \quad CY_1C = Y_1X_2; \quad CZ_1C = Z_1; \quad CX_2C = X_2; \quad CY_2C = Z_1Y_2$$

$$CZ_2C = Z_1Z_2; \quad R_{z,1}(\theta)C = CR_{z,1}(\theta); \quad R_{x,2}(\theta)C = CR_{x,2}(\theta)$$

5.1.3 Measurement

- A "final" element used in every quantum circuit is measurement because it projects our quantum superposition state into the computational basis. We often denote a measurement with a meter symbol.

- **Principle of Deferred Measurement**: Any measurement performed during a quantum circuit can be postponed until the end of the computation, provided that any operations that would have depended on the measurement outcome are replaced with equivalent quantum-controlled operations. (Also, we cannot alter the state in question with a non-trivial unitary operation).

  – You may be confused as to what I mean. So let's do an example: Suppose we have a qubit $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$. Imagine we want to do the following to another qubit in our system: measure the qubit; if the outcome is 0, apply gate A; if the outcome is 1, apply gate B. After measurement, the state is projected into $|0\rangle$ with probability $|\alpha|^2$ or $|1\rangle$ with probability $|\beta|^2$, and then we classically choose (with respect to our rules) which gate to apply.

  – Instead, we can simply not measure the qubit; since it is still in superposition, we can apply a controlled operation respecting $|0\rangle \mapsto A$, $|1\rangle \mapsto B$ or as a matrix

$$U = \begin{pmatrix} A & 0 \\ 0 & B \end{pmatrix}$$

  – The resulting state is: $\alpha|0\rangle A|\phi\rangle + \beta|1\rangle B|\phi\rangle$. Only after everything is finished do we measure, and the probabilities of the outcomes are the same.

- **Principle of Implicit Measurement**: we assume, in general, any non-ending quantum wire at the end of a quantum circuit may be assumed to be measured. This principle is in place to not have any weird entanglement, pure/mixed state, or density matrix consequences from not fully measuring everything.

5.2 Universal Quantum Gates

- Picking up mid-way through the universality argument from the previous section: we already know an arbitrary $2^n \times 2^n$ can be constructed from 2×2 unitaries. But now, we want to know how to build a two-level unitary out of gates you can implement on real qubits?

- **Universal Gate Set**; a foundational collection of logic gates from which any possible computation can be constructed. Classically, we only needed one element in this set, but in the quantum world we need at least two gates: a Clifford Gate and a non-Clifford gate (we will explore further).

- Another thing to add: Universal Gate Sets are not unique. In fact, we should hope so since, as we will show below, certain gate sets are wildly inefficient.

5.2.1 Single-Qubit + cNOT Gates Form a Universal

- Setup: Let U be a two-level unitary on n qubits, acting non-trivially on basis states $|s\rangle, |t\rangle$ (with $s = s_1, ..., s_n; t = t_1, .., t_n$ being their bit strings). The non-trivial 2×2 block is $\tilde{U}$.
- A **Gray Code** from s to t is a chain of bit strings starting at s and ending at t, where each step flips exactly one bit (Go through each "bit" in the string and, at points of contention, flip one at a time). Since s,t differ in at most n places, you can always find a Gray code of length $m \leq n + 1$. E.g.,
$$|101001\rangle \mapsto |101011\rangle \mapsto |100011\rangle \mapsto |110011\rangle$$
- Now, rather than jumping straight from $|s\rangle$ to $|t\rangle$, we will convert it, step by step, using a Gray code until we get to one step away ($|s\rangle \to |g_2\rangle \to ... \to |g_{m-1}\rangle$, where $|g_{m-1}\rangle$ differs from $|g_m\rangle = |t\rangle$). We do this so we can work in a localized 2×2 block matrix. Then, apply the "real" operation at a single-bit difference ($\tilde{U}$). Then, walk back against the Gray code to return the system to only a slightly modified version of its prior self ($|g'_{m-1}\rangle \to |g_{m-1}\rangle \to ... \to |s'\rangle$).
- Imagine this like a conversation at a party: two people in separate conversations want to share gossip. Rather than shouting over the entire party, which is messy for many reasons, one of them walks to the other (Gray Code). Then, they swap gossip ($\tilde{U}$), then one of them walks back (Reverse Gray Code).
- Note: swapping out to $|g_{m-1}\rangle$ and back takes at most $2(n-1)$ controlled operations, each realizable in $O(n)$ single qubit + cNOT gates; the controlled-$\tilde{U}$ itself is an $O(n)$ gates. So one two-level unitary costs $O(n^2)$.
- This reveals that an arbitrary n-qubit unitary costs $O(n^2 4^n)$ single-qubit + cNOT gates ($O(4^n) = O(2^{2n})$ of two-level unitaries needed for an n-qubit unitary times the number of gates needed for a single two-level unitary). Brutally inefficient, but this isn't an efficiency proof, but a proof for the existence of a universal set.
- It does also mean that for quantum computing to be at all tractable, we need a better universal gate set.

5.2.2 Clifford vs. Non-Clifford Gates

- As touched upon earlier, we require both Clifford and Non-Clifford gates to span a universal gate set. Let us paint a picture as to why:
- **Clifford gates** alone aren't enough to achieve a quantum advantage. The **Clifford group** (Hadamard, S, CNOT, and Pauli gates) can all be simulated pretty well on a classical computer. This is directly stated in the **Gottesman–Knill theorem**. So any circuit built from only Clifford gates offers no speedup over classical computation, no matter how complex the circuit looks.
- **Non-Clifford Gates** provide that quantum "magic" to our computations. Adding a single non-Clifford gate (usually the T gate) to the Clifford set makes it universal! This is where quantum computational power comes from.
- The reason why we need both is due to both universality and fault tolerance:
  - Clifford gates are the foundations for fault tolerance (in fact, we can't implement an error correction code without physical and logical Clifford gates). In stabilizer-based error-correction, Clifford gates can typically be implemented transversally (individually across code blocks, not within them), making them relatively cheap and robust to implement fault-tolerantly since errors have a harder time propagating.
  - Non-Clifford gates are expensive to implement fault-tolerantly. The T gate usually can't be applied transversally in stabilizer codes, so it requires costly techniques like magic state distillation and quantum teleportation to implement reliably.
- Only together do these sets of operators allow us to achieve scalable, fault-tolerant quantum logic for quantum computing.

5.2.3 The Search for a Discrete Universal Set

- What do you mean? Why do we care about a discrete set of gates? cNOTs plus arbitrary single-qubit gates are universal, but "arbitrary single-qubit gate" is a continuum. Therefore, there is no hope of implementing every possible rotation (on the Bloch sphere) exactly and fault-tolerantly.
- Suppose for some reason that we want exactly 421 grams of flour for a recipe, but we don't have a gram scale. All we have is a set of measuring cups. Some may argue we should get as close as possible, but it's more reasonable to use an estimation. That is, as long as it's close enough that it doesn't significantly impact our recipe or the taste, it is "fine." Additionally, it wouldn't make sense to purchase a 421-gram scale (one that can only precisely measure 421 grams) because that is too costly and resource-heavy. (We will also explore the notion of fault tolerance later).
- But how do we measure "close enough"? Define the **error** between a target unitary U and the one implemented, V, as $E(U,V) = max_{|\psi\rangle}||(U-V)\,|\psi\rangle\,||$
- This isn't random; it directly lifts from measurement statistics. For any Positive-Operator-Valued-Measurement element M, if P_U, P_V are the outcome probabilities under U against V, $|P_U - P_V| \leq 2E(U,V)$. Small E implies U and V are essentially indistinguishable experimentally.
- **Chaining Inequality**: errors from a sequence of imperfect gates $(V_m, ..., V_1)$ are at most added linearly, never worse (due to the triangle inequality): $E(U_m, ..., U_1, V_m, ..., V_1) \leq \sum E(U_j, V_j)$. So for any set tolerance $\Delta/2m$, it suffices that each individual gate satisfies $E(U_j, V_j) \leq \Delta/(2m)$.
- The standard universal set: H, S, cNOT, and T (we can also use H, (S for convenience), cNOT, and Toffoli as another example set).
- Why can H and T approximate single-qubit gates?:
  - T is, up to global phase, a rotation by $\pi/4$ about $\hat{z}$ on the Bloch sphere; The product of HTH is a rotation of $\pi/4$ about $\hat{x}$. The first H rotates the X axis to the Z-axis position; T then rotates around the new Z-axis (now the X-axis); the final Hadamard puts it back in standard coordinate position.
  - Composing them $T \cdot HTH = e^{-i\pi Z/8}e^{-i\pi X/8}$ works out to a rotation of $R_{\hat{n}}(\theta)$, where $\hat{n} = (cos\frac{\pi}{8}, sin\frac{\pi}{8}, cos\frac{\pi}{8})$ and angle θ, defined by $cos(\theta/2) = cos^2\frac{\pi}{8}$.
  - Note: θ is an irrational multiple of 2π (this can be proven with complex number theory).
  - Because $\theta/2\pi$ is irrational, repeatedly applying $R_{\hat{n}}(\theta)$ generates angles that (modulo 2π) will densely pack the range $[0, 2\pi)$; this is a topological result. So $R_{\hat{n}}(\theta)^n$ can approximate a rotation by any angle α to any accuracy. This is because, we can show that $E(R_{\hat{n}}(\alpha), R_{\hat{n}}(\alpha+\beta)) = |1 - e^{i\beta/2}|$.
  - Conjugating $R_{\hat{n}}(\theta)$ by H yields $HTHT = R_{\hat{m}}(\theta)$, which is a rotation about a second, non parallel axis: $\hat{m} = (cos\frac{\pi}{8}, -sin\frac{\pi}{8}, cos\frac{\pi}{8})$. This is great.
  - A standard result in mathematics is that any rotation in SO(3) (rotations on a sphere) can be written using three rotations around two (unique) axes: $U = R_{\hat{n}}(\beta)R_{\hat{m}}(\gamma)R_{\hat{n}}(\delta)$. Since we can approximate any rotations about both axes arbitrarily well, we can approximate any single-qubit gate with only H and T.
  - Combined with the cNOT universality result, any circuit of cNOTs and single-qubit gates can be approximated only using H, cNOT, and T. (S is also usually included for fault tolerance).
- The question you may now be asking: how many gates does our approximation cost? In the worst case, if we want an accuracy of ϵ for an m-gate circuit $\Omega(m2^{m/\epsilon})$, which is exponentially intractable. The actual cost for a single gate is (on average) more accurately $\Theta(1/\epsilon)$ (for a single qubit gate), yielding $\Theta(m^2/\epsilon)$ for the entire circuit (only quadratic).
- The **Solovay-Kitaev Theorem** optimizes this. A single-qubit gate can be approximated to accuracy ϵ using just $O(log^c(1/\epsilon))$ discrete gates ($c \approx 2$). So an m-gate circuit costs $O(mlog^c(m/\epsilon))$, which is poly-logarithmic with respect to m/ϵ, and it is achievable with simply the set: H, S, cNOT, and T gates.

- Before we continue, I want to address a point: we often can't efficiently (n-polynomial gate count) implement an arbitrary unitary.
  - Using a counting argument, we may derive that for g gates, acting on at most f qubits, in m steps, we get the following relation (for an arbitrary unitary operation).

$$m = \Omega(\frac{2^n log(1/\epsilon)}{log(n)})$$

  - Thus, there exist n-qubit states (and therefore unitaries) that require exponential (in n) gates to approximate within an ϵ tolerance. This provides an upper bound for how many gates we may need, but by using the Solovay-Kitaev Theorem, we can obtain a more reasonable (but still difficult) upper bound of

$$O(n^2 4^n log^c(n^2 4^n/\epsilon))$$

  - So the "general purpose" construction is essentially "optimal," meaning we can't do better if we insist on achieving every unitary state. This, however, is an upper bound. Many of the circuits we need to implement quantum speedup (like, for example, the quantum Fourier transform) are much cheaper to build. If we want to take a closer look at this, we should compare *BQP* and *PSPACE* complexity classes once again.

5.3 Another Look at the Embedding of Computational Classes

- The complexity classes we want to examine:
  - **PSPACE** are computational problems solvable on a (classical) Turing machine using polynomial space with polynomial relation to its input size, with no limit on time.
  - **BPP** is decision problems solvable in polynomial time on a classical (probabilistic) Turing machine, with bounded error probability. For those unaware, a probabilistic Turing machine introduces randomness by making decisions based on probabilities (like flipping a coin). For the same input, it might take a different path or finish at different times.
  - **BQP** is the quantum analogue of BPP. It is a set of computational problems solvable by a polynomial-size quantum circuit with bounded error probability, rather than a classical algorithm.
- A key, and proven result is that **BQP** $\subset$ **PSPACE**. This means that anything a quantum computer can determine efficiently (polynomial space and time), a classical computer can still decide using only polynomial space (not polynomial time).
- To understand why, we can write the final-state amplitude as $\langle u| U_{p(n)}...U_1 |0\rangle$, then insert the completeness relation between each U_i. This turns it into an exponentially long sum of polynomial-space-computable terms (we discard results once added to the sum), so the total stays in polynomial space. This is intuitive since, if you are adding up a thousand different terms (without ordering), you don't care about the terms you've already added, only the ones you haven't.
- You may also be slightly concerned that if **BQP** is at least equal to **PSPACE**, why do we care about quantum computing if classical computers have the capacity to run such computations? This is because humans live very short lives; if factoring the primes associated with the encryption of a billionaire's bank account takes longer than the duration of human civilization, what's the point?
- Since we already know that **BPP** $\subset$ **BQP**, if we can show that if **BQP** $\neq$ **BPP**, then we can tell that **BPP** $\neq$ **PSPACE**. A long open computational question. Although proving that initial premise may be harder than we expect.

5.4 A Note on Quantum Simulations

- Let's finish this section by discussing one of the pillar use cases of quantum computing. Quantum simulations are a fundamental necessity for behavior analysis of quantum systems.

- The core difficulty for simulating dynamics classically usually means discretizing a differential equation (Newton's Law, Poisson's equation, Schrödinger's Equation, etc.). The issue does not stem from solving this for a single quantum/classical system, but rather with scale. For example, modelling an n-qubit system requires 2^n coupled differential equations. Physically important systems like the Hubbard and Ising model or QED/QCD have no known efficient classical simulation.

- Directly computing e^{-iHt} is intractable for a generic Hamiltonian, but most realistic Hamiltonians are decomposable into a sum of local terms. $H = \sum H_k$, each acting only on a constant (with size) of particles. Even though these operators don't tend to commute on a single particle, we can use **Trotter's formula** to approximate the time evolution of the system by alternating short evolutions under each H_k individually.

- **Trotter's Formula**: Let A and B be Hermitian operators; then for any real t:

$$\lim_{n\to\infty}(e^{iAt/n}e^{iBt/n})^n = e^{i(A+B)t}$$

  This holds even if A and B don't commute. Each $e^{-iH_k\Delta t}$ acts on a small subsystem and is efficiently implementable as a short quantum circuit. We can use higher-order symmetrized versions with the Baker-Campbell-Hausdorff formula. The resulting algorithm iterates this approximate step $|\tilde{\psi}_{j+1}\rangle = U_{\Delta t}\,|\tilde{\psi}_j\rangle$

- Let us do a very basic and solved example: Suppose we have a particle in 1D potential V(x), governed by $H = p^2/2m + V(x)$. We can split this Hamiltonian into a position representation piece (V(x)) and a momentum representation piece ($p^2/2m$), so the Hamiltonians can be applied discretely by swapping representations. Thus, we only need to use $n = O(log(N))$ qubits, where N represents the number of basis states we care to express. This is compared to tracking N amplitudes needed to run this simulation classically.

- There are limits, of course. Each step will fully replace the old state (there's no way to peek into a superposition), so we don't have an itemized history of the computation. Additionally, some Hamiltonians cannot be efficiently simulated (since that would require every unitary to be efficient as well). We also need each seperated/decomposed Hamiltonian to be tractable, else our quantum simulation becomes very inefficient.

6 The Quantum Fourier Transform and Quantum Search Algorithms

6.1 Talking About the Quantum Fourier Transform and its Applications

6.1.1 The Quantum Fourier Transform

- We have already briefly discussed the Quantum Fourier Transform earlier, an efficient quantum algorithm used to perform a Fourier transform of quantum mechanical amplitudes. It doesn't necessarily speed up the classical task of computing Fourier transforms, but it will enable us to perform a phase estimation: the approximation of eigenvalues of a unitary operator (under certain circumstances). These are both prerequisites for the period finding problem and eventually the factoring problem and Shor's Algorithm.
- We care about Fourier transforms and transforms as a whole, because it allows us to convert a difficult-to-solve problem into a problem with a known solution. This is a skill used across math, physics, and, in a more general sense, our everyday life.
- One such transform is the discrete Fourier transform that maps vectors of complex numbers $(x_0, ..., x_{N-1})$ (of length N) to another vector of complex numbers $(y_0, ..., y_{N-1})$ given by:

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{(2\pi i)jk/N}$$

- The quantum Fourier transform simply lifts this definition to the probability amplitudes of a quantum superposition, where the relation between x_j and y_k is described above:

$$\sum_{j=0}^{N-1} x_j \left|j\right\rangle \longrightarrow \sum_{k=0}^{N-1} y_k \left|k\right\rangle$$

- For some intuition, if a quantum state is an equally-weighted superposition over positions spaced τ apart (a "comb" of period τ, out of N total positions, with N a multiple of τ), then applying the QFT produces an equally-weighted superposition over frequencies spaced N/τ apart, a comb with reciprocal spacing.
  - A cool example occurs if you consider $N = 16$ and $\left|\psi\right\rangle = \frac{1}{2}(\left|0\right\rangle + \left|4\right\rangle + \left|8\right\rangle + \left|12\right\rangle)$. Under a QFT, this state will map back directly to itself (you could probably assume so from the intuition above).
  - Since the QFT is a unitary operator (easy to show yourself, just take the Hermitian adjoint), it applies linearly to all states separated by a constant "period."
  - If we have a superposition whose period is not a divisor of N, we will observe that the QFT will "leak" probability amplitude to near multiples of N/τ, so we don't get a clean superposition, but one that is "smeared."
- We wish to show now a quantum circuit that allows us to compute the Fourier transform on an arbitrary set of inputs.
- First, let us rewrite this definition as a product, which will reveal the underlying Hadamard structure:

$$\left|j\right\rangle \longrightarrow \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{(2\pi i)jk/2^n} \left|k\right\rangle = \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} ... \sum_{k_n=0}^{1} e^{(2\pi i)j(\sum_{l=1}^{n} k_l 2^{-1})} \left|k_1 ... k_n\right\rangle$$

$$= \frac{1}{2^{n/2}} \bigotimes_{l=1}^{n} [\sum_{k_l=0}^{1} e^{(2\pi i)jk_l 2^{-l}} \left|k_l\right\rangle] = \frac{(\left|0\right\rangle + e^{2\pi i(0.j_n)} \left|1\right\rangle) ... (\left|0\right\rangle + e^{2\pi i(0.j_1 j_2 ... j_n)} \left|1\right\rangle)}{2^{n/2}}$$

Here, we let $0.j_n, ...$ represent the binary fraction with respect to the j's as the decimal digits. Please do not confuse them with decimal points, as I originally did.

- This representation makes it easy to derive an efficient circuit for the QFT. Such a circuit is shown below. Additionally, we controlled unitaries, where the unitary gate is $R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$

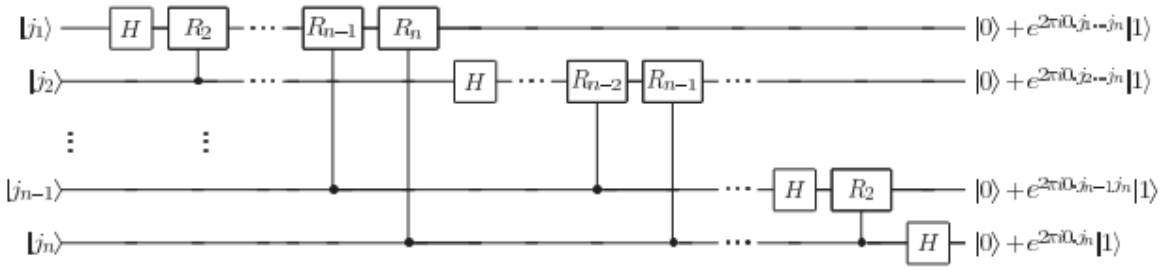


Figure 13: Quantum Circuit for a Quantum Fourier Transform

- At this point, you should have enough intuition to make sense of this circuit yourself. To satisfy my need to explain everything, I will run through a quick summary. The Hadamard gates each contribute a $1/2^{1/2}$ factor, which combines to $1/2^{n/2}$ factor, and puts each state into a superposition of $|0\rangle$ and $|1\rangle$. Moreover, the controlled R_k leaves the control and the $|0\rangle$ state of the target unaffected. It only acts on the $|1\rangle$ state by multiplying it by a relative phase.

- In total, this circuit requires n(n+1)/2 (plus n/2 swaps on average for each controlled unitary). This yields an $\Theta(n^2)$ algorithm, which is pretty fast compared to the poly-logarithmic number we were facing before. For classical algorithms like the Fast Fourier Transform on 2^n elements, we are looking at gate counts in the range of $\Theta(n2^n)$. But, unfortunately, we do not know how to use the quantum Fourier transform to achieve a speedup (since we must collapse the superposition amplitudes, which encode the transform).

6.1.2 Phase Estimation

- The quantum Fourier transform is the key to a general procedure known as phase estimation, which in turn is the key for many quantum algorithms. Suppose we have a unitary operator with some complex eigenvalue of norm one. The goal of phase estimation is to estimate the value of φ, where the eigenvalue (with respective eigenstate $|u\rangle$) is $\lambda = e^{2\pi i\varphi}$.

- To perform the estimation, we assume we have oracle functions that can produce $|u\rangle$ and execute the controlled-U^{2^j}, for $j \geq 0 \in \mathbb{Z}$. Right now, just keep in the back of your head the word "estimation" since this isn't a precise algorithm that gets a definite result but a "close-enough" answer.

- This algorithm uses two registers of qubits. The first contains t qubits initialized to the state $|0\rangle$. How we choose t depends on the digits of accuracy we want and with what probability we want the estimation to be within that range. The second register begins in the state $|u\rangle$, and holds as many qubits as is needed to store $|u\rangle$.

- There are two main parts to the procedure:

  – The first part begins with an n-Hadamard transform to the first register (we will never take the second register out of the state $|u\rangle$). This is followed by the application of controlled-U operations on the second register, with U raised to successive powers of two. The circuit, shown below, yields the final state of the first register in this part is

$$|\psi_1'\rangle = \frac{1}{2^{t/2}} \sum_{k=0}^{2^t-1} e^{(2\pi i)\varphi k} |k\rangle$$

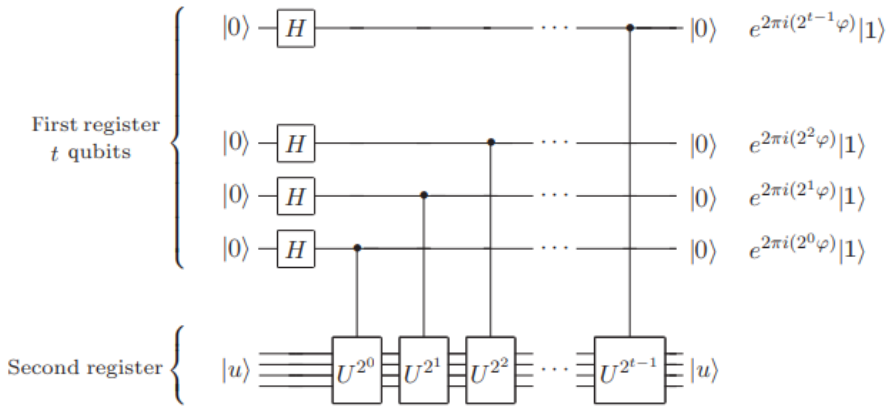


Figure 14: First Stage of the Phase Estimation Procedure

  - The second part of this procedure is to apply the inverse quantum Fourier transform on the first register. This is obtained by simply reversing the circuit (because the transformation is unitary), and it can be accomplished in $\Theta(n^2)$ steps. Then, we measure in the computational basis.

- As of right now, you may be asking: Why does this work? As we established, we let φ be precisely expressed in t bits, as $\varphi = 0.\varphi_1...\varphi_t$. Then, the state resulting from the first stage of the procedure can be written as:

$$|\psi_1'\rangle = \frac{1}{2^{t/2}}(|0\rangle + e^{2\pi i 0.\varphi_t}|1\rangle)(|0\rangle + e^{2\pi i 0.\varphi_{t-1}\varphi_t}|1\rangle)...(|0\rangle + e^{2\pi i 0.\varphi_1...\varphi_t}|1\rangle)$$

- This already takes the form of the output to a quantum Fourier transform. Therefore, we will see that the output state from the second stage is the product state $|\varphi_1...\varphi_t\rangle$. Therefore, measuring in the computational basis will yield φ exactly. Now, this assumes that we have the phase is precisely rational and expressible with t bits of information. If it is not, we get the smearing effect we described earlier from the QFT. This is why t controls both the precision and accuracy.

- In summary, the phase estimation yields the transform:

$$\frac{1}{2^{t/2}}\sum_{j=0}^{2^t-1} e^{2\pi i\varphi j}|j\rangle|u\rangle \mapsto |\tilde{\varphi}\rangle|u\rangle$$

  Here, $|\tilde{\varphi}\rangle$ is a "good" estimator for $|\varphi\rangle$.

- Reverse engineering the procedure, we get that for an n-bit accurate estimate with failure probability of at most ϵ, we should choose t as $t = n + [log(2 + 1/2\epsilon)]$. Intuitively, n bits buys you the precision you want, and the extra logarithmic factor is "insurance" that minimizes spectral leakage down to ϵ.

6.2 Period Finding and Shor's Algorithm

6.2.1 Period Finding

- The **Period-Finding algorithm** is the payoff of the section before. In fact, it was almost immediately reverse-engineered from the period intuition argument of the quantum Fourier transform.

- The problem setup is as follows: Suppose f is a periodic function such that $f(x + r) = f(x)$ for some unknown r, such that r is the smallest such value, and $0 < r < 2^L$. Given an oracle gate $U|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle$ that can be queried in superposition, we want to find the period of f, r.

- The oracle is secretly a phase estimation problem. We can define the "shift-by-one" unitary, U_1, acting as $U_1 |f(x)\rangle = |f(x+1)\rangle$. Because f is periodic with respect to r. U_1 has order r (applying it r times yields the identity transform), so its eigenvalues are the r-th roots of unity ($e^{2\pi il/r}$. For those unaware, the r-th roots of unity divide the complex unit circle into r equal slices (along the radius) with one cut always on the positive real axis.

- The eigenvectors of this "shift-by-one" operator must therefore be $|\hat{f}(l)\rangle = (1/\sqrt{r}) \sum_x e^{-2\pi ilx/r} |f(x)\rangle$. This is intuitive since applying the operator will shift $|i\rangle \mapsto e^{-2\pi il/r} |i+1\rangle$. This is effectively the same procedure for rotating a regular n-sided-polygon by $360°/n$.

- One query of the f-evaluation oracle, applied to a uniform superposition over x, implicitly realizes controlled powers of U_1 acting on these eigenstates. This is precisely the setup for phase estimation, except we get all the necessary controlled-U_1^j powers "for free" out of a single oracle call, because the counting register directly indexes x (by linearity rather than needing t separate calls.

- Now, it is time for the official algorithm (we set our precision estimate to be $t = O(L+log(1/\epsilon))$):

  1. We start in the initial state $|\psi_0\rangle = |0\rangle^t |0\rangle$.
  2. By using an n-Hadamard transform on the first t qubits, we obtain: $|\psi_1\rangle = \frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle |0\rangle$
  3. Now, we apply U with the first t qubits acting as $|x\rangle$ and the $t+1$-th qubit acting as $|y\rangle$. We can then reverse engineer $f(x)$ from our eigenstate. This will only be exact if l and r are multiples. The state is now

$$|\psi_2\rangle = \frac{1}{\sqrt{2^t}} \sum_x |x\rangle |f(x)\rangle \approx \frac{1}{\sqrt{r2^t}} \sum_{l=0}^{r-1} \sum_x^{2^t-1} e^{2\pi ilx/r} |x\rangle |\hat{f}(l)\rangle$$

  4. Applying the inverse quantum Fourier transform (couldn't be clearer) yields us:

$$|\psi_3\rangle = \frac{1}{\sqrt{r}} \sum_{l=0}^{r-1} |\sim l/r\rangle |\hat{f}(l)\rangle$$

  5. Measuring the first register (like for phase approximation), we get $\sim l/r$, which we then interpret with the continued fraction algorithm (to find the smallest period) to get an approximation of the period to be $r' \approx r$.

- As our receipt for this algorithm, we only used one oracle query (we had a great coupon) and $O(L^2)$ other gates. Then, if we want a complete itemized list, we had to run $O(L^3)$ classical post-processing with the continued fractions algorithm.

6.2.2 Order-Finding and Shor's Algorithm

- We can use our Period-Finding algorithms to find orders of groups. In fact, the problem is just a special case of our period-finding algorithm.

- Given x and N coprime to x (they share no factors other than one), find the least $r > 0$, with which $x^r \equiv 1 (mod\ N)$. This is literally the period-finding algorithm applied to the function $f(k) = x^k (mod\ N)$, which is periodic in k with period exactly r.

- Here the oracle becomes modular exponentiation: $U |z\rangle |y\rangle = |z\rangle |x^z \cdot y (mod\ N)\rangle$. We can build this oracle with $O(L^3)$ gates total (O(L) for repeated squaring times $O(L^2)$ for modular multiplication circuit).

- Here, we would need to know r to construct an eigenstate. To sidestep this, create an easy state for the output register, which is an equal superposition of each $|u_s\rangle$ (as a result of the discrete Fourier transform on a group of order r). Running phase estimation on this mixture reports a uniformly random s/r, exactly the random l/r behavior we get from period-finding.

- So what? Why is the order of a group important? Well it has to do with some group theory and number theory.
- If we recovered r is even and $x^{r/2} \neq -1(mod\ N)$, then $x^{r/2}$ is a nontrivial square root of $1(mod\ N)$, since $x^r \equiv 1$. This means $gcd(x^{r/2} - 1, N)$ or $gcd(x^{r/2} + 1, N)$ is guaranteed to be a nontrivial factor of N (simple number theory result).
- This means order finding leads to a factoring algorithm; in fact, it is THE **Shor's Algorithm**. A randomly chosen x satisfies the "r is even, not -1" condition, with a probability $p = 1-(1/2)^m$, where m is the number of distinct prime factors.
- Here is one of the pillars of quantum computing laid right in front of us. One of the most intractable (in time) procedures in all of modern computing is breakable in $O(n^3)$ gates. This is not purely a randomly constructed algorithm. In fact, this problem is deeply rooted within the language of the very nature of Algebra, and naturally emerges from the structure of a group.

6.3 The Hidden Subgroup Problem

- These problems, period and order finding, are emblematic of a broader category of question. They share a similar set of premises (the language is group theoretic, but I will do my best to break it down after):

  Let f be a function from a finitely generated group G to a finite set X such that f is constant on the cosets of a subgroup K, and distinct on each coset. Given a quantum oracle function for performing the unitary operation $U|g\rangle|h\rangle = |g\rangle|h \oplus f(g)\rangle$, with $g \in G, h \in X$, and $\oplus$ is the group operation on X, we want to find a generating set for K.

- Terminology:
  - A **group** is a set equipped with a binary operation (two inputs, one output) that satisfies the group axioms: it contains an identity and an inverse for each element, it is closed under the group operation, and the group operation is associative.
  - A **finitely generated group** has a finite subset of elements that, under a finite number of subsequent operations with other elements in the group (generated), can generate the full group.
  - A **subgroup** is a group that is embedded in another group (e.g. $1. - 1$ is a subgroup of $1, i, -1, -i$ with the group operation of multiplication).
  - **Cosets**, like in linear algebra, are defined with maximal sets of the following: $gH = \{g * h : h \in H\}$. That is, every element in the coset (represented by an element $g \in G$) differs from each other by at most an element of H, where H is a subgroup of G (we are assuming G is commutative). We say the set of all cosets is G/H.
  - "f constant on cosets of a subgroup K" means all elements of a given coset of G/K are mapped to a single element in X, but here, no two different cosets are mapped to the same element of X.
- With these definitions, let's see if we can make more sense of how our period-finding algorithm makes sense here. Immediately, we see that $G = (\mathbb{Z}, +)$, X is a finite set, and $K = \{0, r, 2r, ...\} \cong \mathbb{Z}, r \in G$. The generating set we found is simply $\{r\}$, which is simply the period.
- From this, we can generalize any algorithm of this sort (where G is abelian).
  1. Prepare a uniform superposition over G in the first register (tensored with $|0\rangle$ in the second register)
  2. Query the Oracle function
  3. Measure the second register. This collapses the first register onto a uniform superposition over a single, uniformly random coset of K.
  4. Apply the QFT over G to this coset state
  5. Measure, then repeat to ensure we got the correct answer.

- The reason why this works is deep and convoluted but not impossible to understand as an (abstract) algebra student. Nonetheless, this represents a very cool intersection of theoretical mathematics and quantum computing, revealing deep algebraic structures in quantum logic algorithms.

6.4 A Quantum Search using Grover's Algorithm

- While we are on a roll, we should understand the final pillar of quantum algorithms: Quantum Search Algorithms, and specifically Grover's Algorithm. Thus far, we have covered the Deutsch-Josza Algorithm, Quantum Simulations, and most recently Shor's Algorithm. All of these have been wildly successful results in the field, achieving exponential speed-ups. Thus, having our final example be Grover's Algorithm seems like a let-down since it is only a quadratic speed-up.

- Nonetheless, it is still widely useful across many fields and real-life applications since it answers a very generic question. It also still has its fans, after all 3Blue1Brown did make a video about it. We have already addressed the main points of a quantum search algorithm: we want to search an unstructured set of data and find some element(s).

- Simple enough. We have all faced this problem before, whether it is finding the other sock pair in a basket of laundry or searching a deck of cards to get rid of the jokers. The question remains: how are we able to cut the search speed from $O(n)$ to $0(\sqrt{n})$. After all, wouldn't we need to search each item to determine if it is or isn't what we are looking for? The answer, as always, stems from the linearity of quantum computing.

- First, let us choose our tools, or rather tool, carefully.

6.4.1 The Oracle

- As has been the case for most of our quantum algorithms, our explanations have begun by choosing an oracle.

- Suppose we wish to search through a space of N elements. We don't really care about labeling the element itself, so long as our oracle can discern the correct element. It doesn't matter if we name them after cereal brands or my friends' failed situationships. Therefore, for convenience, we will label them with numbers from 0 to $N-1$. Without loss of generality, we will assume $N = 2^n$ (since if not, we have extraneous basis states that we don't need to care about).

- Suppose we also have M solutions, with $1 \leq M \leq N$. A classical oracle we could consider lifting is $f : N \to \{0, 1\}; x \mapsto f(x)$, where if x is a solution, $f(x)$ is one; else, it is zero.

- We want a similar oracle, but obviously as a unitary operator. For this thought experiment, suppose we are given a black box, U, (whose inner workings aren't important to us yet) that acts on the computational basis as $|x\rangle |q\rangle \mapsto |x\rangle |q \oplus f(x)\rangle$. Here, x is our index register, and q is our oracle qubit (it gets flipped iff our function flags a hit). To check if $|x\rangle$ is a solution, all we need is to act U on $|x\rangle |0\rangle$ and measure the second register.

- Stealing a page out of Deutsch-Josza's book, we should initialize $|q\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. If $|x\rangle$ is a solution to f, then the $|0\rangle$ and $|1\rangle$ swap places in the superposition, picking up a negative sign. If it is not a solution, it remains in the original state. Thus, we can summarize the behavior as follows:

$$|x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right) \xrightarrow{U} (-1)^{f(x)} |x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}}\right)$$

- The state of the oracle qubit has not changed and will never change (until measurement) in this mapping. Therefore, we can exclude it from notation this far and only focus on the first register: $|x\rangle \mapsto (-1)^{f(x)} |x\rangle$. This maps linearly on all such basis states in our search space, so the amplitude of the solutions is negated each time we apply this transformation.

- It turns out that for an M-item search in a space of N entries, we only call the oracle $O(\sqrt{N/M}$ times.

- An important distinction to make is that the oracle can recognize a solution. It doesn't know the solution. I used precise language here because these semantics are at the heart of the oracle unitary. This is the difference between me saying I can recognize every Pokémon if I am presented with it, versus me being able to list every one of them off by memory.

6.4.2 How to Search

- Piggybacking over the previous section, we utilize two registers, the first housing n qubits all initialized to $|0\rangle^{\otimes n}$. The second register only houses a single qubit and is initialized to the $|1\rangle$ state. We then immediately apply a Hadamard gate to effectively initialize the state to $(|0\rangle - |1\rangle)/\sqrt{2}$. We won't ever change the second register's state, but it is an essential part of this algorithm.
- We apply an n-Hadamard transform to the first n qubits to put it into an equal superposition: $|\psi\rangle = \frac{1}{N^{1/2}} \sum_{x=0}^{N-1} |x\rangle$.
- The algorithm is summarized in repeated application of the following procedure:
  1. Apply the oracle U.
  2. Apply the Hadamard transform $H^{\otimes n}$
  3. Perform a conditional phase shift, with every computational basis state except $|x = 0\rangle$ receiving a phase shift of -1.
  4. Apply the Hadamard transform $H^{\otimes n}$
- For our reference, I have added the schematic for the quantum circuit below:

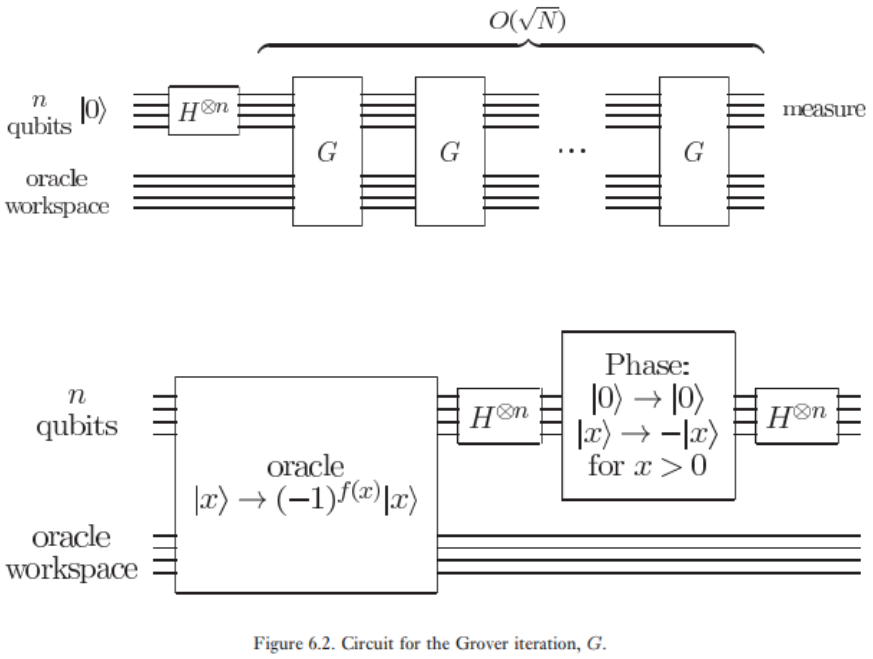


Figure 6.2. Circuit for the Grover iteration, G.

Figure 15: Summarized Schematic of Grover's Algorithm

- You may be curious as to the phase operator. We can consider it the following transformation: $2|x = 0\rangle\langle x = 0| - I$. Note: this is the indexed 0-th state; it is not referencing any state containing a 0 binary element.
- Moreover, you may be curious as to why we single out this state to not receive a phase flip in particular. It is for two reasons: if we added a negative phase to each element, it could be factored as a global phase (which doesn't do anything). Moreover, the $|x = 0\rangle$ element will be in

a search of any length. This does not pose an issue for our algorithm since, before (or after) we conduct the algorithm, we can manually check if this element is a solution in $O(1)$ complexity.

- We can combine this with steps 2 and 4 of our iteration to produce the operator:

$$H^{\otimes n}(2\left|0\right\rangle\left\langle 0\right| - I)H^{\otimes n} = 2\left|\psi\right\rangle\left\langle\psi\right| - I, \quad \left|\psi\right\rangle = \frac{1}{N^{1/2}}\sum_{x=0}^{N-1}\left|x\right\rangle$$

- This operation applied to a general superposition ($\left|\psi\right\rangle = \sum \alpha_k \left|k\right\rangle$) results in the following:

$$(2\left|\psi\right\rangle\left\langle\psi\right| - I)\sum_k \alpha_k \left|k\right\rangle = \sum_k [-\alpha_k + 2\left\langle\alpha\right\rangle]\left|k\right\rangle, \quad \left\langle\alpha\right\rangle \equiv \sum_k \alpha_k/N$$

- This operator reflects (from the minus signs) the system's state about the mean state of the system $\sum \left\langle\alpha\right\rangle \left|k\right\rangle$. The definition of this operator is self-explanatory from the mathematical definition.

- Therefore, we can summarize the entirety of the Grover iteration with the operator: $G = (2\left|\psi\right\rangle\left\langle\psi\right| - I)U$

6.4.3 Geometric Visualization

- For right now, you are simply trusting me that our circuit works as we expect, but we want to obviously know why. To do so, we must first recognize that Grover's iteration is simply a rotation of the state of our system. This rotation occurs in the plane spanned by the even superposition of all data entries ($\left|\psi\right\rangle$) and the state consisting of an even superposition of the solution vectors.

- To see this, let $\sum'$ denote the sum over solutions to the search problem, and $\sum''$ indicate the sum over all non-solutions ($\sum' + \sum'' = \sum$). We can agree that the following vectors are orthonormal:

$$\left|\alpha\right\rangle \equiv \frac{1}{\sqrt{N-M}}\sum{}'' \left|x\right\rangle; \quad \left|\beta\right\rangle \equiv \frac{1}{\sqrt{M}}\sum{}' \left|x\right\rangle$$

- Through simple algebra, we can see that we can $\left|\psi\right\rangle = \sqrt{(N-M)/N}\left|\alpha\right\rangle + \sqrt{M/N}\left|\beta\right\rangle$. As a result, we can see that the original even superposition (after we apply the first n-Hadamard to our first register) lies in the plane spanned by $\left|\alpha\right\rangle, \left|\beta\right\rangle$.

- The effect of G can be understood by decomposing its action into the oracle step and the condensed phase step. The oracle, U, performs a reflection in the plane about the vector $\left|\alpha\right\rangle$. The oracle negates the amplitudes of the answers ($O(a\left|\alpha\right\rangle + b\left|\beta\right\rangle = a\left|\alpha\right\rangle - b\left|\beta\right\rangle)$), behaving similarly to how, in two coordinates, reflections about one axis negate the coordinate of the other (e.g., reflection about the y-axis: $(x, y) \mapsto (-x, y)$).

- The condensed phase step also performs a reflection in the plane, about the vector $\left|\psi\right\rangle$ (which is itself in the plane). Therefore, we get a wonderful result: until measurement, we are always operating within the plane, regardless of the number of times we apply G.

- Let $cos\frac{\theta}{2} = \sqrt{(N-M)/N}$; we have that $\left|\psi\right\rangle = cos\frac{\theta}{2}\left|\alpha\right\rangle + sin\frac{\theta}{2}\left|\beta\right\rangle$. Below is a photo delineating the reflection procedure of a Grover's iteration:

- Therefore, $G\left|\psi\right\rangle = cos\frac{3\theta}{2}\left|\alpha\right\rangle + sin\frac{3\theta}{2}\left|\beta\right\rangle$, meaning the initial rotation angle is indeed θ. For subsequent uses of G, we can simplify the expression using trigonometric arguments, yielding:

$$G^k\left|\psi\right\rangle = cos(\frac{2k+1}{2}\theta)\left|\alpha\right\rangle + sin(\frac{2k+1}{2}\theta)\left|\beta\right\rangle$$

- We then must iterate this procedure until we get $\theta/2 \approx \pi/2$, since this will mitigate the probability of measuring an "incorrect" answer. Once we measure the system and repeat until we get a distribution, we can then interpret it for correct answers.

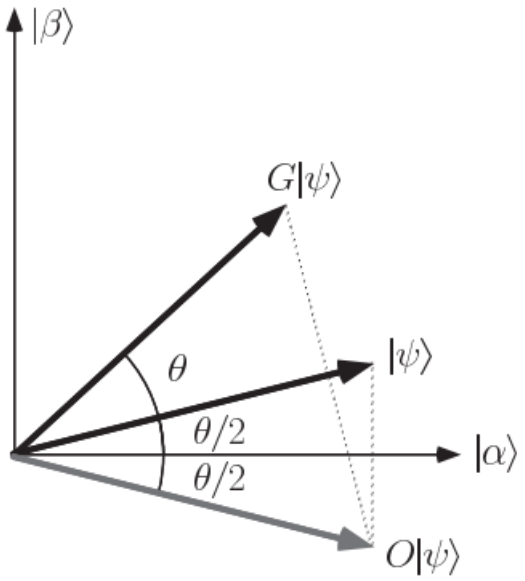


Figure 16: A Geometric Picture of Grover's Iteration

- Repeating the Grover iteration $R = [arccos\sqrt{M/N}/\theta]$ times (here, the brackets indicate rounding to the nearest natural number) will guarantee the state of our system is within an angle $\theta/2 \leq \pi/4$ of $|\beta\rangle$. This will allow us to measure answers (with equal probability) more often than non-answers. All we need to do now is repeat several times to obtain all of our answers.

- Something to observe: we prefer that the number of targets is significantly smaller (counterintuitively) than the number of elements in our data set. This will yield a finer possible accuracy, since the angular error in the final state is at most $\theta/2 \approx \sqrt{M/N}$.

- Doing basic statistical analysis from this lower bound of rotations R. Assuming $M \leq N/2$, we have that we can attain an upper bound of $R \leq \lceil \frac{\pi}{4}\sqrt{N/M} \rceil$. This means $R = O(\sqrt{N/M}$ which is a great improvement over O(N/M) classically. Oftentimes, we state that M=1, so we get the famous result that Grover's Algorithm is $O(\sqrt{N})$!

- **Theorem of Polynomial Bounds:** for problems which are described as evaluations of total functions F (as opposed to partial functions or "promise" functions), we can never achieve beyond a polynomial speedup of a classical algorithm (Hence, Grover's is optimal). This is a common theorem in quantum computing that I won't derive for the sake of brevity.

- Note: Shor's algorithm doesn't satisfy the premises of the theorem since we are given a very specific function that has periodic algebraic qualities based on the problem (promised). As opposed to the problem of Grover's, which is fundamentally an arbitrary function.

6.4.4 Constructing the Oracle and Phase Unitaries

- We have been treating our oracle and phase unitary as black boxes thus far, but for any algorithm to be implemented, we need it in terms of the universal gate set.

- **The Oracle:**

  - Before we give the blueprints of this transformation. I want to examine the givens of the oracle. The oracle encodes a verification procedure, not the solution itself. It implements a Boolean function f(x) that outputs 1 if x is valid, and 0 otherwise.
  - You must know how to recognize a correct answer, but you do not need to know which candidate is the correct one in advance.

- The oracle is constructed from the problem's rules or specification. For example, a Sudoku oracle checks whether a candidate grid satisfies all Sudoku constraints, without knowing the completed puzzle beforehand.
- If the problem is to find a specific known target (e.g., a particular database entry or an ID number), then the oracle necessarily encodes that target as part of the problem definition. It does not reveal its location in the search space, so we do not know where it is before we search.
- To actually construct the oracle, we would need to use a universal gate set. Additionally, each search task would require us to create a brand new oracle function (since no two non-identical functions would mark the same set).
- We will use **n-controlled-not gates**: controlled-not gates with n controls (in our first register) and 1 target (which is our ancilla).
- To encode the state we want to verify, we would use either an identity (indicating 1) or an X gate, which is given by HS^2H=X, (indicating 0). Before acting our entangling gate, we apply a Hadamard to the ancilla (to achieve a phase flip if the solution is correct). Then, we would act our n-controlled gate on the ancilla. Finally, reversing the initial transformation of X's, Hadamards, and identities. Repeat in series for all correct solutions.

- **The Phase Transformation**
  - In practice, we actually do the exact opposite of this operation (and factor a global phase of -1): $I - 2\left|0\right\rangle\left\langle 0\right|$. This only flips the phase of $\left|0\right\rangle$.
  - We can do this without a new ancilla: Apply X gates to each qubit; Apply an n-controlled Z gate (this is the n-controlled gate with the second register qubit flanked on both sides with Hadamards); Apply X gates again.

- This is simply a specific case of the oracle function where all answers are correct other than the zero state.

6.5 Quantum Counting

- Suppose we don't know, and want to find, the number M associated with a given oracle unitary for an N-item search problem. Classically, we would need to iterate through every single item in the list, but the quantum counting algorithm runs it significantly faster.

- The core idea is to leverage both the Grover iteration operation, G, and the phase estimation algorithm to estimate the eigenvalues of G, which encode M.

- The Grover iteration has eigenvalues $e^{i\theta}$ and $e^{i(2\pi-\theta)}$, for eigenvectors $\left|\alpha\right\rangle$ and $\left|\beta\right\rangle$ respectively. Using an augmented search space of size 2N, the rotation angle satisfies $sin^2(\theta/2) = M/2N$. Therefore, if we can find (an approximation) to the eigenvalues of G, we can reverse solve for M.

- A first register of t qubits undergoes phase estimation controlled by G^k applied to a second register of $n+1$ qubits (initialised with Hadamards). This yields θ accurate to 2^{-m} with probability of accuracy $1-\epsilon$:

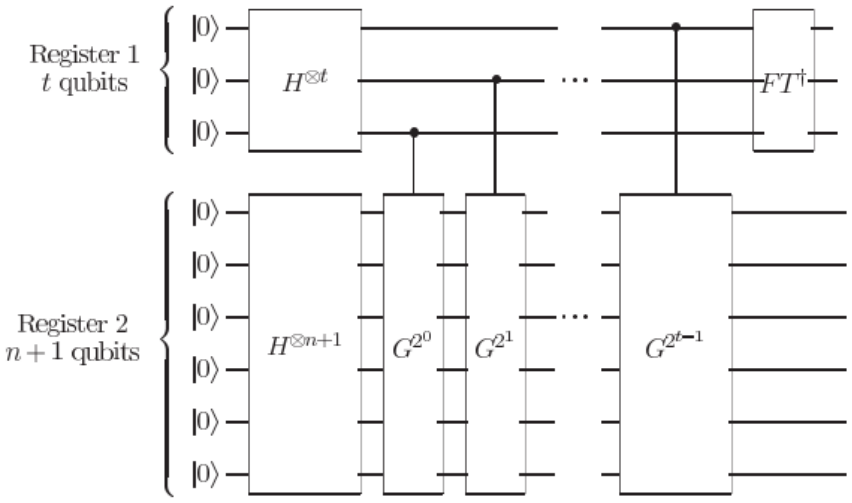


Figure 17: Quantum Counting Algorithm Circuit

- Converting the estimate of θ into an estimate of M gives an error bound $|\Delta M| < (\sqrt{2MN} + \frac{N}{2^{m+1}})2^{-m}$. With $m = \lceil n/2 \rceil + 1$, achieving $O(\sqrt{M})$ accuracy with only $\Theta(\sqrt{N})$ oracle calls, which is also a quadratic speedup!

6.6 A Few Closing Words

- From a macroscopic perspective, quantum algorithms, while crafty, leverage the very basic tenets of quantum logic and algebra to achieve more efficient algorithms. This once again corroborates the fact that the quantum world does not produce a "faster" computer, but a computer that tackles problems differently, at its most basic levels. Since interest in quantum computing has soared in recent years, I hope we can discover a wider set of BQP problems that will highlight the beauty of this logic system.

7 Physical Realizations of Quantum Computers

- Thus far, we have concerned ourselves with the software aspect of quantum computing. As far as learning the material, you didn't even really need to know anything special about quantum physics; you just needed to be good at linear algebra.
- So, let us learn what makes these computers tick, what turns this logic system from a theoretical curiosity to a real-world system.
- Before we start, though, this will be much more of an expository section, since to understand how these systems work would require entire sets of notes themselves.

7.1 What Makes a Good Quantum Computer

- Every algorithm we have spent time developing has quietly assumed an idealistic case, that qubits behave exactly as we want. Of course, in the real world, this will never be true, but we can build a good enough quantum computer to run the algorithms we want.
- However, before we can ask ourselves how to build a quantum computer, we should ask ourselves what we are looking for: we need a checklist of what "good enough" means here.
- Luckily, our work is done for us. In 2000, David DiVincenzo (IBM) assembled a series of criteria for a quantum computer, and it has become the standard yardstick in the field. The DiVincenzo Criteria are five requirements a physical system must satisfy to be a legitimate quantum computer (plus two more if you want it to be able to communicate).
- **The Five Criteria for Computation:**
  - **A Scalable Physical System With Well-Characterized Qubits**: Our qubits must be better than a simple two-level system. We need to know its Hamiltonian, its couplings, and its noise sources all well enough that I can precisely control it. Moreover, our system must be scalable, working whether I have twenty, two hundred, or two thousand qubits.
  - **The Ability to Initialize the Qubits to a "Standard" State**: Every algorithm we discussed began with a state in the $|0\rangle$ state (or close to it). Thus, we better have a way to start our qubits in a designated state. Otherwise, we are doing an algorithm on states in an arbitrary superposition, which doesn't tell us anything.
  - **Long Relevant Decoherence Times (Longer Than Gate Operation Times)**: This discussion is usually split into T_1 and T_2 measures, which tell us about energy relaxation (how $|1\rangle$ decays to $|0\rangle$) and dephasing (how long before relative phase information scrambles), respectively. The ratio of coherence time to gate time tells us how many operations can occur before our qubits forget what it is doing. It would really suck if our decoherence time is really quick and we can't actually meaningfully evolve our state without it just giving gibberish.
  - **A "Universal" Gate Set Implementation**: Recall we don't need every gate, just a finite generating set. Namely, we should usually look to finding out how we can implement the H, S, T, and cNOT gates (or equivalent sets), so we can approximate any unitary gate to an arbitrary precision.
  - **A Qubit Specific Measurement Capability**: We need to measure a specific qubit without collapsing other qubits (unless they are entangled by our system purposefully). Otherwise we can't implement very important classically controlled logical operations.
- **The Two Extra Criteria for Communication** (for a quantum network):
  - **The Ability to Interconvert Stationary Qubits and Flying Qubits**: We want to be able to take stationary information and transmit it as we want. Thus, quantum computers need a way to convert between the two if we want to communicate.
  - **The Ability to Faithfully Transmit Those Flying Qubits Between Two Specified Locations**: We want to be able to send the information without running the risk of severe decoherence or going to the wrong place.

– The only issue between these two criteria is that they are at odds with one another. A well-isolated qubit from its environment is, by construction, hard to couple to anything. Conversely, a qubit that's easy to drive and measure quickly is usually easy for the environment to disturb too. This creates a tension of isolation and control. Every physical platform we will explore tries to cheat this trade-off in different ways.

- With the checklist, let's start with the physical system that, technically speaking, underlies most of these platforms at the lowest level.

7.2 Harmonic Oscillator Quantum Computer

- Whether it's the "wobble" of an ion in a trap, the voltage oscillating in a superconducting circuit, or a standing wave of light in a cavity, the underlying math of the container holding our qubit is almost always the same: the quantum harmonic oscillator.

- Below is the quantum harmonic oscillator Hamiltonian. Here, $\hat{a}$ and $\hat{a}^\dagger$ are the lowering and raising ladder operators, satisfying $[\hat{a}, \hat{a}^\dagger] = 1$

$$\hat{H} = \hbar\omega(\hat{a}^\dagger\hat{a} + 1/2)$$

- The eigenstates are the **Fock states** $|n\rangle$, with energy $E_n = \hbar\omega(n+1/2)$. These are evenly spaced rungs on a ladder, $\hbar\omega$ apart, going forever.

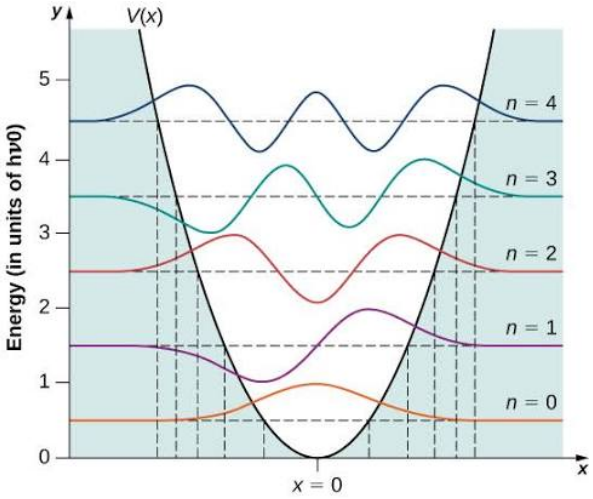


Figure 18: Harmonic Well Eigenstates

- Why is this so prevalent? It is because the QHO is one of the few quantum systems we can solve exactly, and (by Taylor expansions around a valley) any smooth potential looks approximately parabolic near its minimum. Additionally, the harmonic oscillator does not assume what causes the potential, just that there is a harmonic potential.

- A trapped ion sitting near the bottom of its confining potential, an LC circuit near equilibrium, a cavity mode: all of them are, to good approximation, harmonic oscillators. You have probably seen this before; this small-oscillation technique is taught in most introductory physics courses.

- The catch is that a bare harmonic oscillator makes a horrible qubit. If I want to isolate just the $|0\rangle$ and $|1\rangle$ and drive transitions between them with a pulse at a frequency ω. That exact frequency drives transitions for every other Fock state (since they are evenly spaced, and thus the gaps are as well).

- The way to fix this is to make the potential look not *exactly* quadratic. This introduces a small anharmonicity into the spectrum, meaning the spacing becomes unequal, so driving frequencies no longer become a problem. This is precisely what **transmons** do for superconducting systems. Now, we can selectively address just the bottom two rungs.

- But instead of treating the equal spacing as a roadblock, one entire line of quantum computing hardware treats it as a feature. It's called bosonic, or **continuous-variable encoding**, and it flips our usual approach, using the entire spectrum of Fock states to encode logical information.
- Another slightly different type of qubit is called the **Cat qubit** (named after Schrödinger's). Here, we encode logical qubits in a superposition of coherent states (often implemented with microwave cavities that are opposite phase shifts of each other): $|\alpha\rangle$ and $|-\alpha\rangle$. The nice trick is that photon loss, the main error in a cavity, changes the photon-number parity. By repeatedly measuring parity, we can detect these errors without directly measuring the encoded logical state. These will inherently bias noise and make them prime choices for implementing biased error correction models.
- Cat qubits are one example of a broader family of bosonic encodings. Another well-known example is the Gottesman–Kitaev–Preskill (GKP) code, which stores logical information in a periodic comb of oscillator states in phase space (invariant under the continuous Fourier transform). Together, these approaches are central to a subfield known as **hardware-efficient quantum error correction**.
- There's always a catch, though: a purely harmonic Hamiltonian only ever generates linear transformations in phase space (displacements, rotations, and squeezing). No matter how cleverly we chain these together, Gaussian operations alone are not universal for quantum computation. In circuit QED, the missing ingredient is supplied by a nonlinear superconducting transmon qubit, which is dispersively coupled to a high-quality 3D microwave cavity. The transmon provides the nonlinearity needed to implement non-Gaussian operations on the cavity state.
- This design philosophy is generalized in most systems we want to analyze. That is, we take a naturally occurring quantum system, then bolt on just enough control and nonlinearity to make it computationally universal. We will play this out in our next section, optical photon quantum computing.

7.3 Optical Photon Quantum Computer

- In the first section, we highlighted a tension at the center of quantum hardware: isolation vs. control. Photons sit at the first extreme of that spectrum. Light barely interacts with its environment and with each other. Thus, the photon is nature's best isolated qubit, and quite frustratingly, one of the hardest to entangle.
- Two flavors of optical qubit show up in practice, and both are just the Fock-state machinery from before wearing a different costume:
  - Discrete-Variable (DV) Encoding: a single photon is the qubit, and we encode the states in the two modes it occupies: $|0\rangle_L \equiv |1,0\rangle$, $|1\rangle_L \equiv |0,1\rangle$. It is one photon total, split across two polarizations. Simply speaking, it is a single excitation shared between two of the oscillator modes from the last section. The zero state is horizontally polarized light, and the one state is vertically polarized light.
  - Continuous-variable (CV) encoding: we avoid using single photons entirely and encode the logical qubit directly in the quadratures (amplitude/phase) of a squeezed light mode. This uses the GKP machinery above except now, the oscillator is a literal beam of light, not a microwave cavity.
- Luckily, single-qubit gates are the easy part with this form of computing. Beamsplitters and phase shifters implement arbitrary unitary rotations on a dual-rail qubit deterministically, and with high fidelity. This is just optical physics.
- Unluckily, two-qubit gates are the hard part. That photons are so well isolated, there's no natural photon-photon interaction to exploit. I can't just bring two photons close together the way I can with two atoms.

- In 2001, Knill, Laflamme, and Millburn showed that we can do universal linear optical quantum computation only with single-photon sources, beamsplitters, phase shifters, and photon-number-resolving detectors. The trick is something called **measurement-induced nonlinearity**.
- The modern descendant of this idea is fusion-based quantum computation: we build small entangled photonic resource states offline. Then we "fuse" pairs of them together via probabilistic Bell-type measurements. Instead of fighting the probabilistic nature of photonic gates, the architecture is designed around it from the start.
- More simply speaking, imagine we want to cross a river by building a bridge, but every attempt has a small chance of succeeding. Instead of making the traveler wait while you repeatedly rebuild the bridge, you can just keep trying ahead of time until a bridge works. Once you have a successful bridge, the traveler simply crosses it, reaching the other side without experiencing any of the failed attempts.
- Linear optical quantum computing works the same way: it repeatedly prepares a gate offline using extra photons and post-selection, then uses quantum teleportation to send the qubits through the finished "bridge," making the overall computation much more reliable.
- Because light won't really interact with its environment, it rarely decoheres. In fact, the main error is loss of photons. For a dual-rail qubit, a lost photon isn't a bit or a phase flip; it is leakage out of the qubit subspace entirely, which is MUCH harder to correct for.
- While this is a major downside. Photonic qubits are the only major platform that can run at room temperature without dilution refrigerators or ultra-powerful vacuums. They also work pretty well with existing fiber optic telecom infrastructure and can be fabricated with the same (CMOS-compatible) silicon photonics processes used for ordinary computer chips.
- Two main quantum computing companies use photonic quantum computing: **PsiQuantum** (which builds dual-rail qubits with a fusion-based architecture on silicon photonic chips) and **Xanadu** (which builds squeezed light and on-chip GKP state generation for error correction).

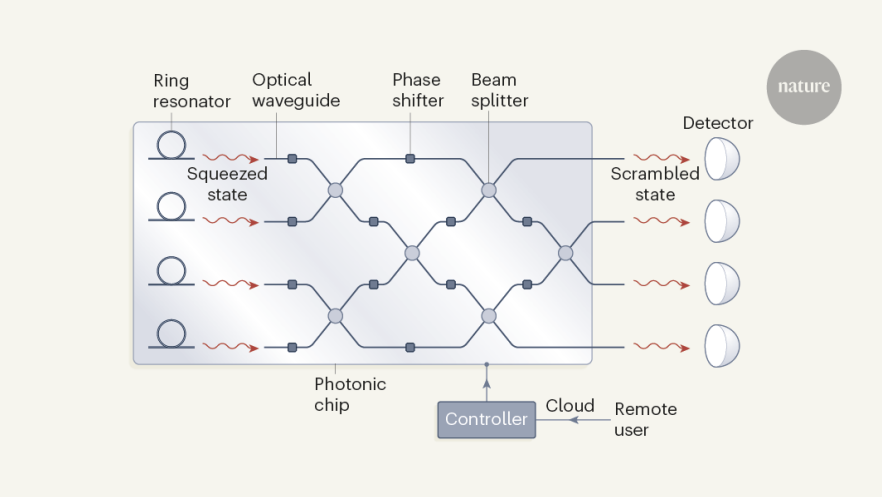


Figure 19: Programmable Continuous Variable Photonic Quantum Processor

- Above is an example of a programmable continuous-variable photonic quantum processor. The idea is that instead of moving electrons through transistors (as in a classical chip), photons are moved through an optical circuit, where beam splitters and phase shifters manipulate their quantum states.
  - You can think about this form of quantum computing like a giant maze filled with water waves instead of light.

- The ring resonator gives us carefully shaped ripples at the left side of the maze. The wave guides (blue wires) are canals that carry these waves through the circuit.
- Beam splitters are junctions where each wave divides into two paths, so every ripple explores many routes through the network simultaneously.
- Phase shifters slightly delay some of the waves so that when they meet again they interfere; some waves reinforce while others cancel out.
- The detectors measure the output modes, collapsing the quantum state to a classical read-out, which we then interpret as the computation's result.
- Each beam splitter mixes the quantum amplitudes of two squeezed states, causing their quantum uncertainties to become correlated so that the optical modes can no longer be described independently. Thus, they become entangled. Repeating this many times creates a large-entangled CV cluster state that serves as the resource for quantum computations.
- This isn't exactly how it works, but it provides a clean heuristic to use.

- As a general summary, photons make excellent messengers but very bad conversationalists. In the next section, we will look for a system that entangles easily. We turn to a system with the opposite personality, ion trap qubits.

7.4 Ion Trap Qubits

- The physical qubit here is refreshingly literal: two internal electronic levels of a single, laser-cooled, trapped ion. We don't have to deal with any continuous parameters or light polarization; it is the classical notion of an energy eigenstate.

- The idea of a trapped ion is subtler than you may think. **Earnshaw's theorem** states that no *static* arrangement of electric fields can trap a charged particle in stable 3D equilibrium (it will find a way to slip out). The **Paul trap** gets around this with an oscillating (radiofrequency) electric field instead of a static one, creating a time-averaged Coulomb interaction and, once laser-cooled close to their motional ground state, they naturally self-arrange into an orderly linear chain!

- Qubit encoding occurs (usually) in one of two common varieties:
  - **Hyperfine Qubits**: we exploit two long-lived sublevels (e.g. in a hydrogen atom, keeping n consistent but changing l and m) of the same ground energy eigenstate. It is common in Yb^+, Ca^+, and Be^+. Since both levels are ground states, there's essentially no spontaneous emission between them. Coherence times can stretch to seconds or longer, which is surprisingly good for quantum hardware.
  - **Optical Qubits**: we use the ground state and a long-lived metastable excited state (common in Ca^+, Sr^+, and Ba^+.) These all have narrower transitions, controlled with lasers rather than microwaves.

- Running quickly through the DiVincenzo checklist for this platform:
  - **Initialization**: optical pumping repeatedly drives one qubit state into a short-lived excited level that preferentially decays into the ground qubit state. That is, if we have a qubit basis $\{|0\rangle, |1\rangle\}$. You use a laser that only excites the first state to a temporary, short-lived state $|e\rangle$. The excited state then decays (with very high probability) to the ground state. Keep doing this process until a sufficient population of $|1\rangle$ states is gone.
  - **Single-Qubit Gates**: resonant laser or microwave pulses drive Rabi oscillations between the two levels. This is the Bloch sphere rotation picture from nuclear magnetic resonance (NMR) or spin precession.
  - **Measurement**: its role is cast by state-dependent fluorescence. Shine a laser resonant with a cycling transition from only one qubit state. If the ion is in that state, it is bright; if it's in the other state, it stays dark. This gives some of the highest single-shot readout precision of any qubit technology.

– **Two-Qubit Gates**: this is where ion traps get clever. The ions sit very close together with no direct interaction between their internal states. So instead, entanglement is mediated through the shared motional (phonon) mode of the whole chain. That is, the relative oscillations of the trapped ions (think of a line of masses oscillating with springs in between them, where the springs here are Coulomb potentials). These oscillations between two qubits form their own quantum harmonic oscillator, just made of jiggling atoms.

  * The standard implementation is the Mølmer-Sørensen gate: a bichromatic (two-tone) laser field, detuned symmetrically above and below the qubit transition, exerts a spin-dependent force on each ion's coupling to the shared normal mode. Choose the detuning and pulse duration so the phonon mode traces a closed loop in phase space and returns exactly to where it started. Therefore, the ions end up disentangled due to their oscillatory motion itself, but rather entangled by the geometric phase caused by a loop in phase space!
  * This should look familiar. Just like the quantum system plus ancilla set up earlier, an oscillator is doing the entangling work, and it takes a driven, engineered interaction (not the bare harmonic Hamiltonian) to make it happen.

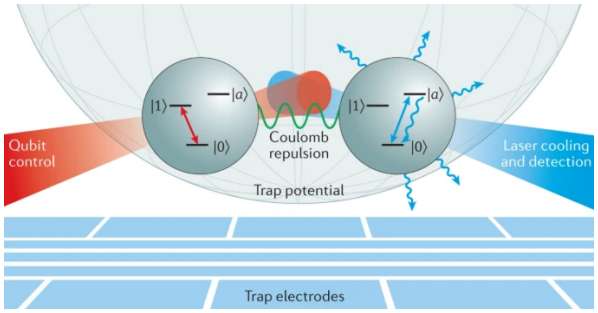


Figure 20: Segment of an Ion Trap Chain

- So how well did trapped-ion qubits score on its report card? Since ions of the same species are, by nature, perfectly identical, there is no manufacturing variance. Combined with the deep isolation of hyperfine states, this gives ion traps some of the best gate fidelities available.
- The only issue is the run time. Because of relying on laser-driven transitions rather than fast microwave pulses. Two-qubit gates are often several orders of magnitude slower than a superconducting cNOT gate.
- Moreover, scaling runs into a wall with a single chain. More ions means a denser, harder-to-resolve normal-mode spectrum, tighter laser-addressing crosstalk between neighbors, and harder cooling. In sane people's words, it becomes cramped.
- The fix was published in 2002: the Quantum Charge-Coupled Device. It shuttles ions between separate "memory" and "gate" zones using time-varying voltages, so entangling occurs locally. This platform is the backbone of Quantinuum's H-series and IonQ's commercial systems today.
- A complementary architecture turns separate ion traps into a *quantum network* by linking them with photonic interconnects. Each trap emits photons that interfere in a Bell-state measurement, heralding entanglement between ions that may be physically far apart.
- This satisfies DiVincenzo's communication criteria from before. We have that stationary ion qubits store and process information, while flying photonic qubits distribute entanglement between distant nodes. It also provides a natural bridge back to the photonic platform discussed in Section 7.3, where photons serve as the carriers of quantum information over long distances.

- Overall, trapped ions are very cool platforms and offer some of the most direct applications of the things we discussed in our previous quantum mechanics notes. Many of the criteria are satisfied by "natural" quantum mechanical principles, but the next platform, superconducting qubits, engineers everything down to the qubit itself.

7.5 Superconducting Qubits and Planar Hardware

- We made a very important complaint about the quantum harmonic oscillator: a bare harmonic oscillator has perfectly evenly spaced levels, which makes it impossible to selectively address our computational basis. Superconducting qubits are, at heart, the direct engineering fix to that exact complaint.

- Start with an LC circuit, an oscillator made of a capacitor and an inductor, which is just the harmonic oscillator realized in wires instead of light or motion.

- Now, we swap the ordinary linear inductor for a **Josephson Junction**: two superconductors seperated by a nanometers-thin insulating barrier, thin enough to allow for **Cooper pair tunneling**.

- **Cooper pairs** are pairs of electrons bound together at low temperatures through interactions with a material's crystal lattice, enabling the flow of electricity with zero resistance (very low energy loss) known as superconductivity. Hence, we get the name of **superconducting quantum computing**.

- Unlike a normal inductor, a Josephson junction is non-linear, breaking the even-level spacing of the QHO. Therefore, we can finally drive just two levels and get a genuine qubit out of a quantum oscillator.

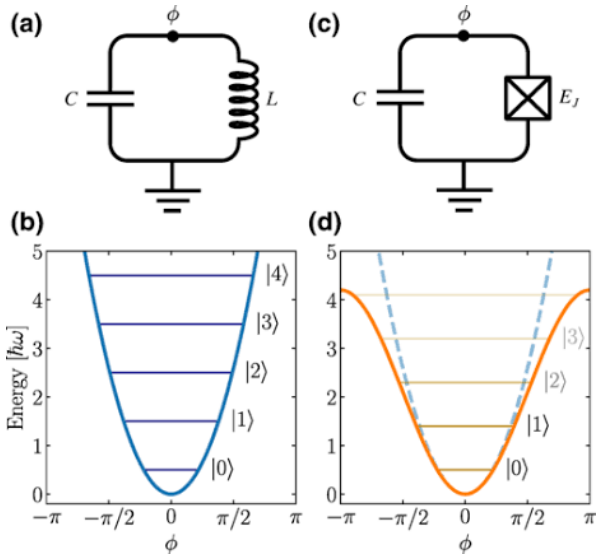


Figure 21: Circuit of a Superconducting Qubit

- The most common design is the **transmon**. It's built to maximize the ratio of Josephson energy to charging energy: $E_J/E_C >> 1$, which makes the qubit's frequency almost completely insensitive to stray charge noise on the capacitor (won't spike the energy eigenstates). This resolves a big headache for early superconducting designs.

- There is a tradeoff, though. This shrinks the anharmonicity down to a few hundred MHz so gate pulses have to be carefully shaped to avoid accidentally leaking population into higher energy states.

- Why do we need this system to be cold? Well, superconductivity itself requires it; Cooper pairs only form and flow resistance-free below a critical temperature. Devices live inside a dilution refrigerator at around 10–20 millikelvin, colder than deep space, to keep k_BT far below the superconducting gap and the qubit's own energy.
- This is the converse side of the isolation/control tradeoff. We have to engineer isolation through the form of very cold refrigeration, rather than for free the way an ion's hyperfine states do.
- These circuits can be patterned with standard photolithography, aluminum or niobium traces on silicon or sapphire lattices, onto a flat chip. This is why we often refer to it as "planar hardware." It borrows directly from decades of semiconductor manufacturing infrastructure. Also, this lattice-style architecture is leveraged in the most prominent quantum error-correction code: **the surface code**.
- As usual, let's run our go-to quantum platform checklist:
  - **Initialization**: this happens mostly for free, since the energy eigenstates are of scale significantly higher than the ambient temperature ($\hbar\omega >> k_BT$). Thus, the qubit thermalizes almost entirely into $|0\rangle$ on its own (since $|1\rangle$ is thermodynamically unfavorable). We can use some active reset protocols to clean up the rest.
  - **Single-Qubit Gates**: We use microwave pulses at the qubit frequency delivered through a capacitively coupled drive line. This is very fast, occurring within $10 - 30\ ns$.
  - **Measurement**: We read out our measurements using a technique called **dispersive readout**. Couple the qubit to a separate readout resonator; the resonator's frequency shifts slightly depending on the qubit's state (Jaynes-Cummings interaction, off-resonantly detuned). Then, we probe the resonator with a microwave pulse and read the reflected phase/amplitude to infer the qubit state. This is relatively fast, and we can multitask this across many qubits on the same feedline.
  - **Two-Qubit Gates**: Two competing philosophies exist here:
    - **Fixed-Frequency Transmons + Cross-Resonance Gates**: This is the approach led by IBM. We drive one qubit at its neighbor's frequency through their fixed mutual coupling. No tunable elements means no flux noise to worry about (making it simpler and quieter) at the cost of slower, weaker entangling gates.
    - **Flux-Tunable Qubits or Tunable Couplers**: This approach is led by Google. We briefly tune two qubits' frequencies into resonance to swap excitations directly (this produces CZ-type gates). What this means for us is that this causes faster, stronger interactions, at the cost of reintroducing sensitivity to magnetic flux noise.

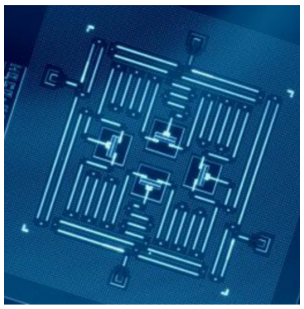


Figure 22: Transmon Qubits on a Quantum Chip

- Unlike in an ion chain's shared normal modes or a reconfigurable photonic network. Chips are laid out as 2D grids, almost always with only nearest-neighbor coupling. Two qubits on opposite corners of the chip simply can't talk directly. For two qubits to talk, we would need to route a chain of SWAP gates to get them close enough to be neighbors.
- This isn't just an inconvenience. It is the reason why the surface code is the natural error-correcting code for this platform. The topological nature of the code was built for this layout.

- Today, we have an energy relaxation time in the hundreds of microseconds (leading devices are pushing past the millisecond mark), gate times of around 20-60 ns, single-qubit fidelities above 99.9% and two-qubit fidelities above 99.5%.
- That last figure is very important since surface-code error correction generally needs the physical two-qubit error rate below roughly 0.5–1% to even be worth doing (otherwise it does more harm than good).
- A cool concrete anchor: Google's Willow chip (105 qubits as of December 2024) was the first to demonstrate surface-code error correction operating below that threshold.
- Additionally, IBM, with its fixed-frequency transmons and cross-resonance gates across its Heron and Condor generations, is aiming its Nighthawk and forthcoming Kookaburra chips at more efficient quantum low-density parity-check (qLDPC) codes rather than the surface code. Though generally, these would require non-local connectivity.
- Superconducting qubits are speedy and "easy"-to-manufacture by engineering everything, whether it is the anharmonicity, the isolation, the connectivity; all of it is from scratch.
- However, this platform, while very reliable, creates a very rigid constraint for our computing algorithms: our system is effectively locked down. We can't rearrange our planar hardware in any shape or configuration we want, and to "move" qubits we need SWAP routing, which is very costly. This is precisely the challenge that neutral atom platforms sought to address.

7.6 Neutral Atoms and Rydberg Arrays

- The qubit at the heart of Rydberg Arrays is an individual neutral atom. It usually takes the form of an alkali atom (like rubidium-87 or cesium-133) or an alkali-earth-metal like ytterbium-171 or strontium-88.
- Quantum control is given to us through optical tweezers. These tweezers aren't for plucking hairs, but rather they are laser beams focused small enough that the resulting dipole potential traps a single atom at the focus. Using spatial light modulators (or acousto-optic deflectors), we can split a single laser into thousands of independent foci, allowing us to create arrays of atoms in any one-, two-, or three-dimensional arrangement we want.
- Pulling an atom from a laser-cooled cloud into any given tweezer only succeeds about half the time. We actually preload the array, see what laser traps caught an atom, then physically drag atoms out of the overfilled traps into empty ones. We want to assemble a defect-free *before* the computation even begins.
- This shape control is a very powerful highlight of this computation platform. Also, this feels very similar to ion traps, and much of the design philosophy is very similar.
- Qubit states are encoded in long-lived hyperfine (or, in the case of alkali-earth metals, nuclear-spin) states. These states are driven with microwave or Raman laser pulses for single-qubit rotations and read out via our state-dependent fluorescence (from before).
- The main difference point emerges in a trick called **Rydberg blockades**. We briefly excite an atom's outer electron to what is known as a **Rydberg state**, one with a very large principal quantum number, so the electron orbits absurdly far from the nucleus. This gives the atom a huge effective dipole moment, and the strength of interaction between two nearby Rydberg atoms scales at a factor of n^{11}.
- This is strong enough such that exciting one atom shifts its neighbor's Rydberg energy level significantly out of resonance with the same laser. So if atom A gets excited to a Rydberg state, atom B can't follow it to the same excitation. Therefore, we can achieve extremely precise control over the elements of the array.

- A standard entangling gate turns this phenomenon into a controlled-phase gate. We drive both atoms with a pulse that would carry an atom all the way to a Rydberg state and back down (a full loop in phase space). However, if both states we are acting on are in the state coupled to the Rydberg state, the blockade prevents one atom from completing a full phase space loop, so it accumulates a different phase than it would have unblocked.
- This different phase is precisely what causes the phase of the second atom to be entangled to the first. Note: the interaction only reaches a few microns, so the two atoms need to be moved (very) close together first, which isn't that big of a deal since that's the whole gimmick of this platform: moving atoms precisely.
- This process is precisely what gives it a significant advantage over trapped ion qubits. This reconfigurability strips away most constraints of the QCCD. Instead of shuttling ions through a fixed network of etched trap zones, we can move entire rows of atoms mid-circuit! This flexibility offers one of the greatest challenges to the reliability of superconducting qubits.
- As always, there are disadvantages to mention. Rydberg gates are very slow; it often takes a few orders of magnitude longer to implement than a gate on a superconducting circuit.
- Additionally, atoms are sometimes lost from a trap outright (background-gas collisions, heating). This offers a heralded "erasure" error (since we can tell by imaging if a state gets measured or lost early), which is often easier to correct for as opposed to bit or phase flip errors. Note, this platform may still have bit or phase flip errors as well.
- Nevertheless, the reconfigurability becomes significantly more useful when dealing with error correction. Most very good codes like qLDPC codes are locked behind the need for long-range connectivity for planar hardware (which doesn't currently exist), so Rydberg arrays allow us to perform efficient error correction.
- Overall, while not as reliable as superconducting qubits, the sheer flexibility of the platform qualifies it to be one of the most discussed hardware platforms.

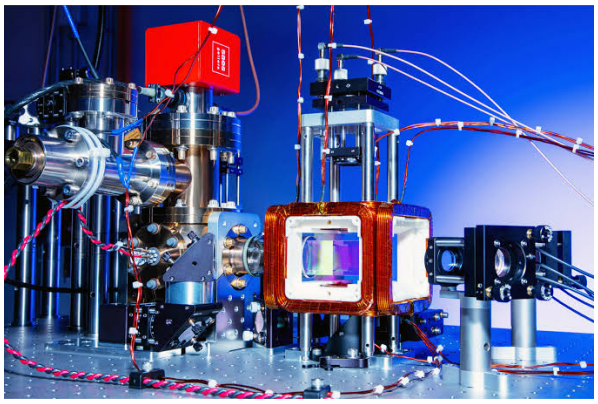


Figure 23: Neutral Atom Computation Platform

7.7 A Farewell to Quantum Hardware

- At this point in our discussion, it should be apparent that there exists no "best" quantum computing platform. Hence, no singular style is universal across private and public sector research. Each leverages a unique physical or engineering solution to remedy different issues with quantum computing.
- The biggest take away should be that the challenge of quantum hardware is not discovering quantum mechanics (per say), but learning how to exploit it while mitigating decoherence. Again, I will reiterate the tension between isolation/robustness and control.

- As the field develops, it may become the case that multiple platforms coexist, filling the role it performs best. Even still, discoveries and improvements are being pursued everyday. However, a sufficiently robust quantum computer is not enough to do quantum computations. We need a way to correct errors if they occur. We need **quantum error correction**.

8 Quantum Error Correction

- Thus far, we have been operating with a very strong assumption: every gate, every qubit, and every exchange of information has been perfect. However, to implement a true quantum computer, we can't hold onto such a luxury. The moment we store or send quantum information, the environment can "poke" at it, and the state we have been trying so hard to maintain gets changed. This makes sense since the quantum world is "small" (with a few caveats), so any packet of energy (as we have seen with perturbation theory) has a chance to screw over our system royally.

- Therefore, one of the biggest subfields within quantum computing has been quantum error correction. This section will detail the basics of QEC and prepare you with the ability to parse the field.

8.1 Understanding Quantum Channels and Single Qubit Errors

- Before we begin with understanding Quantum Error Correction, we must first understand Quantum Errors. In this section, we will go over the types of quantum noise, certain well-fitted assumptions we can make about said noise, and finally see how noise manifests into errors. This discussion must first start with quantum channels.

8.1.1 Quantum Channels

- Picture our classic two quantum friends, Alice and Bob. Alice wants to get a qubit to Bob. Both of them have flawless (quantum) computers, but the link between them (whether that is a fiber-optic cable, open air, a line of qubits being swapped down a chain, or a phone call) is not perfect, as anyone with shoddy cell service can tell you. We call that link between (quantum) computers, errors and all, the (quantum) **channel**.

- We can also think of classical channels simply as very noisy quantum channels, because if quantum mechanics is a complete theory of physics, all classical phenomena can be explained by quantum mechanics. However, this is just a fun thought experiment, and we don't gain much from thinking of it as such.

- Our first assumption regards how we model our quantum channel: its output depends only on the input (including the current state of the environment), not on some earlier history of the system. When this is true, we call our noise **Markovian**. It's exactly this that lets us collapse "everything the environment does during the transmission" into one single, well-defined map.

- **Density Matrix**: an operator that completely describes the state of a quantum system, including both pure states and statistical mixtures. It is Hermitian, with all non-negative eigenvalues, and unit trace. We can encode quantum pure states as $\rho_q = |\psi\rangle \langle\psi|$. Classical information is encoded as "if-then statements." For example, suppose we want to encode that "if we flip a coin and get heads, then we prepare the 0 eigenstate; else, prepare the 1 eigenstate." This is not a superposition state; this is classical probabilities, so we can encode this as a density matrix that takes an input and outputs a state with its probability: $\rho = \frac{1}{2} |0\rangle \langle 0| + \frac{1}{2} |1\rangle \langle 1|$.

- From our Markovian assumption, we can mathematically model any **quantum channel** as a completely positive (all eigenvalues are positive $\implies$ positive states are mapped to positive states), trace-preserving (the trace of the density matrix is maintained throughout the Markovian chain) map of density matrices.

- This is a mouthful to say, let alone explain. What this effectively does for us is two things: we are ensured that probabilities sum to one and that we always get a valid density matrix even if we have entanglements between states (inside and outside the quantum channel). The second condition is largely derived from the "completely positive" property of our quantum channel and is stricter than you may believe. An alternative we may propose is the transpose map; it sends valid density matrices to valid density matrices all by itself, but fails horribly the moment you apply it to half of an entangled pair. Therefore, we require our "completely positive" assumption.

- Every channel admits (has a) Kraus Decomposition: $\varepsilon(\rho) = \sum_k A_k \rho A_k^\dagger$, $\sum_k A_k^\dagger A_k = I$. This can be read as: error A_k happens with probability $tr(A_k \rho A_k^\dagger)$. Two things worth remembering: the probability depends on ρ itself and the set $\{A_k\}$ is not unique (a physical channel can be expressed by more than one Kraus decomposition).
- A simple example is a single Kraus operator, $\varepsilon(\rho) = A\rho A^\dagger$. By completeness, $A^\dagger A = I \implies A$ is unitary. So every error is fully coherent and reversible (simply an unwanted error).
- The basic vocabulary of single-qubit errors is the Pauli matrices, which we have already met as desired gates. Now, they exist as unwanted alterations to our density matrix. As errors (rather than gates), we interpret X as a bit-flip error, Z as a phase-flip error, and Y is both at once. By virtue of the "non-uniqueness" of Kraus decompositions, we can obtain the significant fact that any single-qubit error, no matter how exotic, can be decomposed into I, X, Y, and Z errors.
- **Dephasing Channels** offer our next step up in difficulty: $R_p(\rho) = (1-p)\rho + pZ\rho Z$. This simply says that with probability 1-p nothing happens, but with probability p our qubit receives a phase flip. Let's write an arbitrary density matrix and see how it transforms:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \mapsto \begin{pmatrix} a & (1-2p)b \\ (1-2p)c & d \end{pmatrix}$$

- As we can see, a dephasing channel shrinks the off-diagonal ("coherence") terms of the density matrix and leaves the populations alone. On the Bloch sphere, this squashes the sphere into an ellipsoid that's flattened along the equator while the poles stay fixed.



Figure 24: A Visual Representation of a Dephasing Channel

- When $p = 1/2$, we get a **completely dephasing channel**. You may be asking why not a probability greater than 1/2; this is because these are simply related to probabilities of less than 1/2 by Z-conjugation. When we have a completely dephasing channel, all our information (if we look at the mapping above) collapses to be on the diagonals, so we get simply a classical bit of statistical information.
- Physically, dephasing is what happens whenever the environment "learns" the qubit's state in the computational basis without otherwise disturbing it. Processes such as spin-orbit coupling (if we consider spin states) will "scramble" the relative phases between the eigenstates over time. We also saw this in the spin precession experiment, where the probability of measuring a state in a different basis fluctuated with respect to time.
- **Depolarizing Channel** is the next step up: $D_p(\rho) = (1-p)\rho + \frac{p}{3}(X\rho X + Y\rho Y + Z\rho Z)$. Here, an X, Y, or Z error is equally likely, each with probability p/3. Using $tr(\rho) = 1$, we can rewrite this as: $D_p(\rho) = (1 - \frac{4p}{3})\rho + \frac{4p}{3}\frac{I}{2}$. This means we keep the state the same with probability 1-4p/3. However, with probability 4p/3, we disregard the state entirely and it is replaced by the maximally mixed state (p=3/4 is the **completely depolarizing channel**, which simply replaces our state with a maximally mixed state).
- What makes depolarizing noise the theorist's favorite is its symmetry: unlike dephasing, it treats the X, Y, Z errors identically, and it's actually invariant under any unitary rotation, not just the ones that permute the Paulis. On the Bloch sphere, it shrinks the sphere uniformly toward the center, collapsing to a single point (the maximally mixed state) in the fully depolarizing limit. It's rarely an exact match for real hardware noise, but its symmetry makes it enormously convenient for analyzing codes in the abstract.



Figure 25: A Visual Representation of a Depolarizing Channel

- Both the dephasing and depolarizing channels are simply special cases of the more general **Pauli Channel**: $\varepsilon(\rho) = p_I\rho + p_X X\rho X + p_y Y\rho Y + \rho_Z Z\rho Z,\ p_I + p_X + p_Y + p_Z = 1$. A general Pauli channel shrinks the Bloch sphere into an ellipsoid still centered at the origin, but with no particular symmetry among the axes; dephasing and depolarizing are just the two extremes of all the weight on one axis versus spread perfectly evenly.
- Note: not every real channel (non-single qubit) is a Pauli channel (amplitude damping, which models a qubit spontaneously decaying to its ground state, is a common example that isn't). But Pauli errors turn out to be exactly the right building blocks for the codes we're about to build, for reasons that will become clear as soon as we generalize to more than one qubit below.

8.1.2 Multi-Qubit Noise and t-Qubit Errors

- A single qubit of quantum information isn't much use; a real error-correcting code needs to spread information across a whole register of n qubits, and we need to understand how noise acts on that register as a whole.
- **n-Qubit Pauli Channel**: $\varepsilon(\rho) = \sum_P p_P P\rho P^\dagger$, where P ranges over every 4^n tensor products of I, X, Y, Z across the n qubits, and $\sum_P p_P = 1$. This is a simple lift of the definition from the section before (and we can recover it by setting $n = 1$). This is about as close as quantum mechanics gets to a classical noisy channel: a definite, discrete error P occurs with a definite probability p_P.
- Naturally, the easiest way to build a multi-qubit channel is to just let each qubit be subject to its own noise independently.
- **Independent (Memoryless) Channel**: $\mathcal{E} = \bigotimes_{i=1}^n \mathcal{E}_i$, where each $\mathcal{E}_i$ acts on its own qubit and the qubits don't influence each other's errors at all.
- Let's do a simple sanity check. Take n=3 copies of the dephasing channel (R_p). There are $2^3 = 8$ possible combinations of phase-flip/no-phase-flip, with probabilities obtained by simply multiplying independent events. If we write out a table, we get:

Probability	Errors
$(1-p)^3$	$I \otimes I \otimes I$
$p(1-p)^2$	$Z \otimes I \otimes I,\ I \otimes Z \otimes I,\ I \otimes I \otimes Z$
$p^2(1-p)$	$Z \otimes Z \otimes I,\ Z \otimes I \otimes Z,\ I \otimes Z \otimes Z$
p^3	$Z \otimes Z \otimes Z$

- The chance of some qubit going qoing wrong is bigger $(1-(1-p)^3)$ than the single-qubit error rate p. This seems obvious: more qubits means more chances for something bad to happen, but for small p, the zero- or one-error rows dominate completely. This imbalance is exactly what error correction is built to exploit, fixing many small mistakes over a few big ones.
- To make "how many things went wrong" more precise, we need a notion of size for an error.
- **Weight**: An operator A acting on n qubits has weight, t, if it acts as the identity on all but t qubits. The qubits it acts non-trivially on are its **support**.

- **t-Qubit Error**: An n-qubit operator B is a t-qubit error if it can be written as a sum of terms each of weight at most t (the individual terms need not share the same support). A channel is a **t-qubit error** channel if every one of its Kraus operators is a t-qubit error.

- Why bother with this? Designing a code that provably corrects any error of weight at most t is a tractable problem, but a fully-general n-qubit CPTP map lives in a Hilbert space that is exponential in size and is hopeless to reason about directly. A question naturally arises: is restricting our attention to bounded-weight errors tunnel-visioning us too much?

- It turns out the answer is no (when the noise is roughly an independent channel). If every single-qubit channel making up an n-qubit independent channel is within some small "distance" ϵ of the identity, then the whole channel can be approximated by a t-qubit error channel, with an approximation error that shrinks roughly like $\binom{n}{t+1}(c\epsilon)^{t+1}$ for some smallish constant c. The proof is very long and involved so I will skip it here. However, the intuition is very similar to the $n = 3$ case we did above.

- The key takeaway is that a code built to handle any weight of at most t error will, for free, handle realistic independent noise with tiny per-qubit error rates extremely well, even though it was never designed with that specific channel in mind. This is exactly why every code we meet from here on will be evaluated with a single number: the largest t for which it can correct all errors of weight at most t. This is called the **code distance** of a quantum error correction code.

8.2 Classical Error Correction Codes

- Before we begin talking about a quantum topic, as we often do, let's talk about its classical counterpart. This is especially important since, as we will soon see, classical codes become the building blocks of quantum codes. Therefore, a sizable amount of terminology transfers from classical error correction unchanged (and the parts that don't transfer give us a target for quantum error correction).

- The **Repetition Code** is the simplest example of an error correction code that I can think of. Instead of having a single piece of information stored in a single bit, we encode each bit as several copies of itself: $0 \mapsto 000,\ 1 \mapsto 111$. It's effectively storing backup copies to recover original messages.

- Let's look at what happens when we send a three-bit codeword (111) through a channel that flips at most one bit. We can always recover the original information by majority vote. Though this argument breaks down when we introduce channels that could flip more than 1 bit. However, we get an estimate for how often such an error will manifest: 2 or 3 errors have a probability of $3p^2(1-p) + p^3 = O(p^2)$, so the probability that an error occurs scales with p^2.

- So long as p is small, our encoding actually works! The entire philosophy of error correction is balancing how small we can get this probability of logical error with how few extra bits (or general resources) we need.

- Now, let's introduce a laundry list of vocabulary.

  - Physical (Qu)bits: All bits required for an error-correction model. E.g., in the repetition code, each 1 and 0 in the string of (for example) ...111000111... This is all the data needed to encode a piece of unencoded information.
  - Logical (Qu)bits/Information: This is the actual information that we are sending. Physical qubits encode them. E.g., for the repetition string 111000111000, the logical information encoded is 1010.
  - Error: Some deviation from the original code string.
  - Syndrome: a set of measurement outcomes that indicates the presence and type of error in an error correction code. This is also referred to as a "correctable" error.
  - Logical Error: An "uncorrectable error" that may also go unnoticed ($111 \mapsto 001$ or $000 \mapsto 111$).

- Hamming Distance: For two bit strings (of the same length). The Hamming distance is the number of positions in which they differ.
- Minimum distance for a code: Given a set of codewords ("111" and "000" for the repetition code), the minimum distance of a code is the minimum Hamming distance between any two codewords. (3 for the repetition code).

8.2.1 First Look at Coding Distance

- The question arises: Why does code distance matter? A code with minimum distance d can *detect* any error of weight up to d-1 (an error that light can never turn one codeword into another), and can *correct* any error of weight up to $t = \lfloor (d-1)/2 \rfloor$ because an error of weight $\leq t$ moves a codeword less than halfway to any other codeword, so "nearest-codeword decoding" recovers the right answer unambiguously. For the repetition code, we saw the distance was d=3 and t=1.
- This notion will reappear very similar to the definition of quantum code distance. We will also see why we define quantum code distance as we did above.
- We use the $[n, k, d]$ notation to describe a code that encodes k logical qubits into n physical bits with minimum distance d. The repetition code is a [3,1,3] code.

8.2.2 Syndrome Decoding

- Rather than comparing a received string against every codeword by brute force, we can repackage a set of parity checks into a **parity-check matrix** H. For the repetition code,

$$H = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix} (mod\ 2)$$

- Given a received substring r (a three-bit chunk of our string), we compute the syndrome (or the error we can detect) as $s = Hr$. This will give you a 2×1 matrix, where
  - $(0\ \ 0)^T$ means we do nothing
  - $(0\ \ 1)^T$ means we flip the third bit
  - $(1\ \ 0)^T$ means we flip the first bit
  - $(1\ \ 1)^T$ means we flip the second bit
- Thus, by reading off s, we know what bit to flip in the substring without ever needing to know the original message bit. This doesn't really matter classically; it's just a concise heuristic. In the quantum error correction world, however, it becomes the guiding rule since reading data collapses the superposition.

8.2.3 General Linear Codes

- Classical error correction codes are more than simply the repetition code. A linear $[n, k, d]$ code is any k-dimensional subspace of $\{0, 1\}^n$ (all code words are $\mathbb{F}_2$ linear combination fo k basis code words). Certain codes far exceed the repetition code in terms of performance. We will hold onto these, but we won't go into them here. A version of exactly this idea, the CSS construction, is how we'll build efficient quantum codes out of pairs of good classical codes.
- Regardless of the code, the general idea is that we encode states with redundancy, diagnose errors via syndromes (without needing to know the message), and characterize a code with its distance. A natural question arises: can we do the same thing to qubits?

8.3 First Look at a Quantum Error Correction Code: the Shor Code

- Why can't we simply do the repetition code but with quantum states? E.g., $|\psi\rangle \mapsto |\psi\rangle|\psi\rangle|\psi\rangle$. The answer is the No-cloning Theorem of Quantum Computing (we can't copy and paste states).
- A smart idea is to encode $|0\rangle \mapsto |000\rangle$ and $|1\rangle \mapsto |111\rangle$, by linearity $\alpha|0\rangle+\beta|1\rangle \mapsto \alpha|000\rangle+\beta|111\rangle$. This is an entangled state, not the cloned state. So "encode" cannot mean "copy" for an unknown superposition, but it *can* mean "copy the basis states and let linearity do whatever it wants to the superposition." What linearity wants to do is entangle them. This turns out to be a loophole we can exploit!
- Something we should be worried about is measurement and how this affects the superposition we are trying to protect. Classically, decoding a majority vote means just reading the three bits. Reading three qubits to see "which one looks different" would collapse any superposition between the encoded branches, destroying the quantum information we were trying to save. Therefore, we need to extract only "is there an error and where", but not anything regarding the information on the bit itself.
- Something else we should worry about is that, classically, we only have to deal with bit flips. Quantum channels have to deal with some combination of I, X, Y, Z. A code that only guards against X doesn't protect against phase flips (Z). So by adding more qubits to protect against X flips (without protecting for Z), we get that there is a greater chance for phase flips to land.
- None of this is fatal; we simply need to replace the majority rules method with something smarter. Let's build the fix one layer at a time, culminating in the first genuine quantum error-correcting code: Peter Shor's 9-qubit code.

8.3.1 The 9-Qubit Code

- We said that by copying only the basis states and letting linearity handle the rest, we have the same map from above: $|0\rangle \mapsto |000\rangle$ and $|1\rangle \mapsto |111\rangle$, so a state $\alpha|0\rangle + \beta|1\rangle \mapsto \alpha|000\rangle + \beta|111\rangle$.
- Here we aren't cloning the state, simply expanding it. We can implement this as the state produced by two CNOTs, with the data qubit as control and two $|0\rangle$ states as targets.
- Now suppose an X error hits qubit 2: $X_2(\alpha|000\rangle + \beta|111\rangle) = \alpha|010\rangle + \beta|101\rangle$. In both terms of the superposition, qubit 2 disagrees with qubits 1 and 3. The qubit that disagrees is not a property of the state itself but the error.
- So instead of measuring the qubits directly, we measure only whether pairs of them agree. That is, we measure the operators Z_1Z_2 and Z_2Z_3. Why does this not collapse the superposition? A quantum measurement only destroys a superposition if the different branches correspond to different measurement outcomes.
- The measurement value of Z_1Z_2 and Z_2Z_3 are +1 if the pair agrees and -1 if it disagrees. Since they both output the same eigenstate when acting on $\alpha|000\rangle + \beta|111\rangle) = \alpha|010\rangle + \beta|101\rangle$, we don't collapse the superposition.
- To get this overlap information and ensure we are not measuring the data qubits directly, we use ancilla qubits as "scratch paper."
- Suppose we want to encode the state $\alpha|000\rangle + \beta|111\rangle$; we do not want to measure the data qubits themselves, because that would reveal the logical state.
  - Instead, we create two ancillas $|0\rangle_{a1}$ and $|0\rangle_{a2}$. To record the overlap on the ancilla (one for the overlap on q1 and q2 and the other for q2 and q3), we apply two cNOTs (one with control on q1 and the other with control on q2, both have target on the ancilla). We do something similar (but on q2 and q3 respectively) for the second ancilla.
  - If the ancilla is 0, we have that the bits overlap (1 if they don't overlap), corresponding to eigenvalues +1 and -1, respectively. So measuring a_1 and a_2 gives the same syndrome table as the repetition code.

  - Thus, we know the overlaps of the bits without collapsing the superposition. If an ancilla ever measures 1, we know where the error is and what the error is (X).

- Thus, we have created an X-correcting error correction code. Now, to ensure we get protection from both X and Z errors, we need to figure out how to implement a Z-correcting error correction code.

- We do so by simply repeating the same procedure, but rotated:

  - Recall that the Hadamard transform swaps the roles of X and Z, meaning X acts as a phase flip on the $|+\rangle$ and $|-\rangle$ basis while Z acts like the bit flip in this basis.
  - So take the entire 3-qubit bit flip code and conjugate every qubit by H: $|0\rangle \mapsto |+++\rangle$, $|1\rangle \mapsto |---\rangle$. A phase flip error shows up exactly like a bit flip error did before (all that changes is that we are in the rotated Hadamard basis). Thus, the identical syndrome-extraction trick (ancilla CNOTs now conjugated with Hadamards) locates and fixes single Z errors, again without touching the encoded amplitudes.

- To get Shor's code, we simply nest one code inside the other. We have two codes: one immune to X errors, one immune to Z errors; neither handles both alone. Shor's insight was to concatenate them:

$$|0\rangle_L = \frac{1}{2\sqrt{2}}(|000\rangle+|111\rangle)(|000\rangle+|111\rangle)(|000\rangle+|111\rangle); \;\; |1\rangle_L = \frac{1}{2\sqrt{2}}(|000\rangle-|111\rangle)(|000\rangle-|111\rangle)(|000\rangle-|111\rangle)$$

- This extends linearly to superpositions, but can we convince ourselves that this is both X and Z-resistant (and therefore also $Y = iXZ$ resistant)?

- **Bit flips** (X errors): A single X error lands inside exactly one block, and the parity checks within that block locate and fix it: 2 ancilla checks per block, 6 total. E.g., $Z_0Z_1, Z_1Z_2, Z_3Z_4, ...$ It is easy then to decipher the syndrome. (For now, I use zero-based indexing to correspond to the circuit diagram)

  - Suppose we measure $a_0, a_1, ..., a_5$ and group them as $(a_0, a_1), (a_2, a_3), (a_4, a_5)$. A single bit flip will only house zeros in exactly one of the three pairs (corresponding to q_0 to q_2, q_3 to q_5, and q_6 to q_8 respectively).
  - If there exists only a single "one" measurement in an ancilla pair (either the first or the second element of the pair), then it corresponds to the first or third element of the qubit triplet, respectively. E.g., measuring (010000) means there is a bit flip in the third qubit (or q_2). If there are two ones in a pair, then it corresponds to the middle element of the triplet. E.g., measuring (000011) means there is a bit flip on the eighth qubit (or q_7)

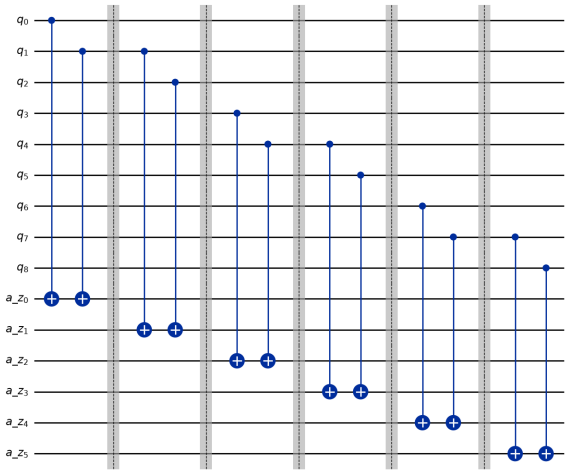


Figure 26: Bit-Flip Ancilla Measurement

- **Phase flips** (Z errors): A single Z error flips the overall sign of exactly one block. We can't Hadamard-and-compare qubit-by-qubit anymore (the sign now lives collectively across all three qubits of a block, not on any single one). However, we can compare blocks to each other (i.e. block 1: $X_0X_1X_2X_3X_4X_5$ and block 2: $X_3X_4X_5X_6X_7X_8$). This reveals which single block, if any, carries the "wrong" sign. Then, applying Z to any one qubit in that block fixes it, since a "block's" collective sign depends only on how many of its three qubits have had a Z (mod 2). We only need 2 ancillas for this; we simply controlled-Z all the supported qubits onto the ancillas. (Again, I use zero-based indexing to correspond to the circuit diagram)
  - To read off the syndrome: read the measured ancillas as a_0, a_1.
  - We use the same pair rules as above: If there exists only a single "one" measurement in an ancilla pair (either the first or the second element of the pair), then it corresponds to the first or third element of the qubit, respectively, but if there are two ones in a pair, then it corresponds to the middle qubit.

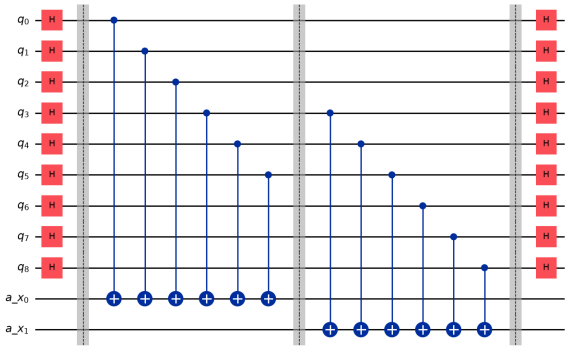


Figure 27: Phase-Flip Ancilla Measurement Circuit

- Here, we can describe the X logical operator as $X_L = Z^{\otimes 9}$ and the Z logical operator as $Z_L = X^{\otimes 9}$. The H logical operator is simply transversal: $H_L = H^{\otimes 9}$. We can similarly define any logical operator for our Clifford Operators. The task becomes much harder with non-Clifford gates, which we need to use Magic State Distillation for. More on this subject later.

8.3.2 Understanding Code Distance

- We now have, for the first time, a genuine quantum error-correcting code: 9 physical qubits encoding 1 logical qubit, correcting an arbitrary error on any single one of those 9 qubits. Following the classical [n,k,d] notation from the classical case, we write [[n,k,d]] for a quantum code. The Shor code is a [[9,1,3]] code. Why is d=3?
- From above, we can easily correct a single bit flip or phase flip syndrome. So $d \geq 3$.
- We can show that d is exactly 3 with a three-Z error, with one error in each code block, such as $Z_1Z_4Z_3$. This precisely transforms the $|0\rangle_L$ to $|1\rangle_L$. Hence, $d = 3$ exactly.
- This is worth sitting with, because the quantum Shor code is simply two copies of the classical repetition code [3,1,3]. This is precisely what motivates CSS codes.
- While you may be amazed by our construction of this error correction code, this code is pretty bad: 9 qubits, for one logical, and the code distance is 3. We can do way better. In fact, this is a main goal of QEC.

8.4 Quantum Error Correction and CSS Codes

- We built the Shor code entirely by hand and a bit of educated guessing, checking by brute force that it survives every single-qubit Pauli error. This is a fine way to discover *a* code, but it's a

terrible way to discover *good* codes. Before building anything fancier, let's write down the actual, general conditions a codespace needs to satisfy so we can correct a given set of errors. Then we can use those conditions to systematically build out error-correction codes rather than by trial and error.

- **The Knill–Laflamme Conditions:** Let $\mathcal{C}$ be our codespace (space of states with no measurable errors) with a projection operator $P : \mathcal{H} \to C$ onto it, and let $\{E_a\}$ be a set of error operators we'd like to correct (likely the Pauli errors or possibly non-unitary Kraus operators, our conditions/theorems doesn't care). Then, our codespace is *correctable* against $\{E_a\}$ if and only if there exists a Hermitian matrix α_{ab} such that

$$PE_a^\dagger E_b P = \alpha_{ab}\, P \qquad \text{for all } a, b.$$

- Two things are being asked of us here, and both should feel familiar from our Shor code construction:
  - *The diagonal part* ($a = b$): $PE_a^\dagger E_a P = \alpha_{aa} P$ says that every error acts on the codespace by the "same amount," α_{aa}, regardless of which encoded state it hit. Colloquially speaking, this simply says that the error never depends on the logical state. This aligns with the Shor code, as we could only ever measure parity on the ancilla but never the logical amplitudes.
  - *The off-diagonal part* ($a \neq b$): either E_a and E_b send the codespace to orthogonal subspaces (so a syndrome measurement can tell them apart perfectly), or they act identically on the codespace (α_{ab} is the same as if $E_a = E_b$, in which case there's nothing to distinguish; correcting one automatically "corrects" the other, because they were never different errors as far as the encoded information is concerned).

- This is a rigorous, but also more generalized version of the argument from the Shor code: it's precisely because $\{I, X, Y, Z\}$ satisfies these conditions on the Shor code that measuring the syndrome forces any error onto one of those four discrete outcomes.

- Now, the Knill–Laflamme tells us when a code *works*, but it doesn't tell us how to build one. For that, we steal a page (or more accurately two) from classical coding theory and glue two classical codes together. This forms a set of codes known as **Calderbank–Shor–Steane (CSS) Codes**.

- **The CSS construction.** Take two classical linear codes over F_2, $C_2 \subset C_1$, with parameters $[n, k_1]$ and $[n, k_2]$ respectively (so $k_2 < k_1$). That is, one code is contained within another, with the number of logicals encoded following the embedding. For each coset $x + C_2$ of C_2 inside C_1 (or each element of the quotient space C_1/C_2), build the quantum state

$$|x + C_2\rangle \;=\; \frac{1}{\sqrt{|C_2|}} \sum_{y \in C_2} |x + y\rangle\,.$$

- There are $2^{k_1 - k_2}$ distinct cosets, and one can check that these states are orthonormal, so they span a $2^{k_1-k_2}$-dimensional codespace: this encodes $k = k_1 - k_2$ logical qubits into n physical qubits. Call this code $\mathrm{CSS}(C_1, C_2)$. (The Shor code chooses another basis for the codespace, specifically the Hadamard transform of these states.)

- Now, we have to verify that this actually is an error correction code:
  - **Correcting bit flips.** An X-type error, X^e for some binary error vector e, shifts $|x + C_2\rangle \mapsto |x + e + C_2\rangle$. If e has weight small enough for C_1's classical decoder to identify it, we recover e from the syndrome, using C_1's parity-check matrix, and undo it. This part is literally classical error correction, run on the amplitudes' labels.
  - **Correcting phase flips.** Recall the single-qubit identity $H|0\rangle = |+\rangle\,, H|1\rangle = |-\rangle$ generalizes, for n qubits, to a discrete Fourier transform over F_2^n: $H^{\otimes n}|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{z \in F_2^n} (-1)^{x \cdot z} |z\rangle$. Applying $H^{\otimes n}$ to $|x + C_2\rangle$ turns it into a superposition supported on the *dual* code $C_2^\perp$, with phases depending on x. A Z-type error becomes, after this transform, formally identical to an X-type error on this dual-code state — so it's corrected the same way, using the classical decoder for $C_2^\perp$.

- Decoupled decoding: because errors split cleanly into X-type and Z-type, you measure the Z-stabilizers (from H_Z) to get a syndrome for X (bit-flip) errors, and the X-stabilizers (from H_X) to get a syndrome for Z (phase-flip) errors — each decoded independently with an ordinary classical decoder, rather than needing one joint quantum decoder.

- Putting the two halves together: CSS(C_1, C_2) corrects up to $t = \min(t_1, t_2)$ errors, where t_1 is the number of errors C_1 can correct and t_2 is the number $C_2^\perp$ can correct. Two independent classical decoders, glued together, give one quantum decoder — and crucially, we get to reuse every tool from classical coding theory (distance, syndrome decoding, the Hamming bound) essentially for free.

8.4.1 Revisiting the Shor Code

- Let us now discuss the Shor Code with the language of the CSS codes. Now, you may be concerned with our statement that this code is a product of two 3-bit repetition codes. While this is a strong notion, it's not completely illuminating.

- Here, when we say a "product of two codes," we refer to Shor's code being a concatenation: we take an inner code and an outer code, and encode each logical bit of the outer code using a full copy of the inner code. This mirrors how we defined multiplication in the third grade: for $n \times m$, add m to itself n times. For this analogy, n is the outer code, and m is the inner code.

- Shor's code, like we mentioned, is a $CSS(C_1, C_2)$, with $n = 9, k = 1$. The two classical codes are:

  - C_1 : the $[9, 3]$ code. Its parity check matrix is exactly the six "compare-adjacent-bit" checks inside each block. A word satisfies all six checks if and only if each block is constant, so $C_1 = \{(a, a, a, b, b, b, c, c, c) : a, b, c \in F_2\}$. This is just three copies of the repetition code.
  - C_2 : the $[9, 2]$ subcode. Take (A, B, C) as "superbits" (e.g., $A = a, a, a$) and impose that $a + b + c = 0$: $C_2 = \{(a, a, a, b, b, b, c, c, c) : a + b + c = 0\}$. The two non-trivial generator vectors are 111111000 and 000111111, which means we are comparing block one and block two to block two and block three.

- Since $k_1 - k_2 = 3 - 2 = 1$, there are 2 cosets of C_2 over C_1, either the sum of a,b,c is either 0 or 1, giving $|+\rangle_L$ and $|-\rangle_L$ respectively:

$$|+\rangle_L = \frac{1}{2}(|000000000\rangle + |111111000\rangle + |000111111\rangle + |111000111\rangle)$$

$$|-\rangle_L = \frac{1}{2}(|111000000\rangle + |000111000\rangle + |000000111\rangle + |111111111\rangle$$

- As per the CSS code, we define $X'_L = X^{\otimes 9}$, $Z'_L = Z^{\otimes 9}$, and $H_L = H^{\otimes 9}$. (However, this isn't the logical operators we tend to define for the Shor code.)

- Now, the Shor code isn't precisely the construction given by our CSS instructions. (In fact, as I noted, it is the transversal Hadamard of it). However, it is isomorphic to a CSS code, which means it is effectively an "equivalent" construction. We define for the Shor code (which are the logical states that we are used to):

$$|0\rangle_L = \frac{1}{\sqrt{2}}(|+\rangle_L + |-\rangle_L), \quad |1\rangle_L = \frac{1}{\sqrt{2}}(|+\rangle_L - |-\rangle_L)$$

- Additionally, we write that $Z_L = X'_L$ and $X_L = Z'_L$ to remain consistent with our desired notation. Now, we need to verify if the CSS construction for identifying syndromes is actually consistent:

  - We see that trivially, Bit-flip error detection is lifted from the $[9, 3]$ code exactly, as the construction dictated.

- Phase-flip correction comes from applying a Logical Hadamard to the $|+\rangle_L, |-\rangle_L$ state (which is simply the $|0\rangle_L$ and $|1\rangle_L$ respectively). Then, the parity check matrix of C_2 is simply given by a 2×9 matrix comprised of the two vectors: 111111000 and 000111111. Since at most one physical qubit suffers a Z error, at most one block is "phase-flipped," and this outer 3-block comparison is just the classical 3-bit repetition code running once more, now at the level of blocks instead of individual qubits.

- Therefore, it's safe to say we haven't completely lost our minds!

8.4.2 The Steane Code

- By virtue of constructing the Shor code before constructing the more general CSS codes, we saw that the Shor code didn't exactly abide by the rules we set forth. However, henceforth, we can follow exactly the rules that we had outlined in our construction.

- The flagship CSS example is the Steane code, and it uses two "copies" of the $[7, 4, 3]$ Hamming code, with a 3×7 parity-check matrix

$$H = \begin{pmatrix} 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

- The columns are the nonzero binary strings $1, \ldots, 7$ in order. The Hamming code has a nice property: its dual, given simply by the conjugate, $C^\perp$ (the $[7, 3, 4]$ code generated by the rows of H) satisfies $C^\perp \subset C$, so we may take C_1 as the Hamming code and $C_2 = C_1^\perp$ in the CSS construction above.

- This gives a quantum code where there is one logical qubit protected against any single-qubit error, and encoded in seven physical qubits. This is an immediate improvement on the Shor code's $[[9, 1, 3]]$, and for the same code distance: $[[\, n,\, k_1 - k_2,\, d\,]] = [[\, 7,\, 4-3,\, 3\,]] = [[7, 1, 3]]$

- Because C_1's and $C_2^\perp = C_1$'s parity checks both use the H matrix, the very same rows of H serve as both the X-syndrome checks and the Z-syndrome checks. Thus, the Steane code has an unusually clean, symmetric structure between its bit-flip and phase-flip codes.

- Because the Hamming code contains the all-ones vector, the Steane code's logical operators can be implemented transversally, $\bar{X} = X^{\otimes 7}$ and $\bar{Z} = Z^{\otimes 7}$, which means applying the same physical gate to each of the seven qubits independently performs a logical operation, which turns out to be exactly the kind of structure that keeps errors from spreading during fault-tolerant computation.

- A transversal gate applies the same operation independently to every physical qubit, so it can only implement a logical gate if it maps every stabilizer (defined by the parity-check matrix) back into the stabilizer group. The one-column vector ensures that the phases and Pauli operators combine correctly under this transformation, allowing the transversal operation to act as a valid logical gate.

- To construct the Hamming code zero logical qubit, we must evaluate the span given by the dual of $C^\perp$ (since the logical qubits must be invariant under X stabilizers). Since the Hamming code is self-orthogonal, $C^\perp = span(r_1, r_2, r_3)$. This spans eight vectors, since in F_2 there are 2^3 choices of coefficients. As specified by the rules of CSS construction, the zero logical must therefore be:

$$|0\rangle_L = \frac{1}{\sqrt{8}}(|0000000\rangle) + |0001111\rangle + |0110011\rangle + |1010101\rangle + |0111100\rangle + |1011010\rangle + |1100110\rangle + |110101\rangle)$$

- Rather than explicitly spelling it out, we know we could obtain the one logical by simply taking the transversal X gate on this string of seven qubits. This produces a nice symmetry where the zero logical contains all the Hamming code words with an even number of ones, and the one logical contains all the Hamming code words with an odd number of ones. This clean construction is not shared amongst most CSS-type-codes.

- To define our bit and phase flip checks, because our logicals are simply the states corresponding to possible codewords of the Hamming code, a single bit or phase flip is corrected by two copies of the exact same parity check matrix.

8.4.3 CSS Parity Check Matrices

- A CSS code is defined, as we obviously know, by a pair of binary parity check matrices: H_X ($m_X \times n$) and H_Z ($m_Z \times n$). Rows of H_X generate the X-type stabilizers and the rows of H_Z generate the Z-type stabilizers.
- For all stabilizer generators to commute (for any and all stabilizer codes), we require $H_X H_Z^T = 0$ (mod 2). That is, every row of the X matrix must be orthogonal over GF(2) to every row of the Z matrix.
- This works because an X-string and a Z-string anticommute exactly when their supports overlap on an odd number of qubits, so the mod-2 inner product of the rows must vanish.
- With n physical qubits, the number of encoded logical qubits is $k = n - rank(H_X) - rank(H_Z)$
- We can develop a matrix representation for the quantum CSS code formed from this:
  - The General Symplectic Representation: any n-qubit Pauli operator (up to a phase) can be written as $X^a Z^b$ for binary vectors $a, b \in F_2^n$. A set of m stabilizer generators is then an $m \times 2n$ binary matrix $M = [A|B]$, where each row $(a_i|b_i)$ is one generator.
  - We can then "pad" these classical check matrices together using block diagonal matrices:

$$M = \begin{pmatrix} H_X & 0 \\ 0 & H_Z \end{pmatrix}$$

  - The top m_X rows are purely X generators, and the bottom m_Z rows are pure Z generators. The result is a single $(m_X + m_Z) \times 2n$ binary matrix representing the CSS code in symplectic form.

8.4.4 Quantum Hamming Bound

- **How good can we do?** The *quantum Hamming bound* caps how efficient any nondegenerate $[[n, k, d]]$ code (one correcting $t = \lfloor (d-1)/2 \rfloor$ errors) can possibly be:

$$2^k \sum_{j=0}^{t} 3^j \binom{n}{j} \;\leq\; 2^n$$

- The intuition is a direct quantum analogue of the classical sphere-packing bound: each of the 2^k logical basis states, hit by any of the $\sum_j 3^j \binom{n}{j}$ possible weight-$\leq t$ errors (three choices, X, Y, or Z, per affected qubit), must land somewhere distinguishable inside the 2^n-dimensional physical Hilbert space, and there's only so much room. Both the Shor and Steane codes sit comfortably inside this bound with room to spare — which is exactly why we should expect codes even more efficient than $[[7, 1, 3]]$ to exist. Finding them systematically is what the next section is for.

8.5 Stabilizer Codes

- Every code we've built so far, whether it was the 3-qubit bit- and phase-flip codes, the Shor code, the Steane code, or the CSS codes, turns out to be a special case of one unifying structure: the Stabilizer Formalism. It's the framework essentially all of modern QEC is built on, and a review of QEC would be crappy without it.
- **The Pauli group:** Let $\mathcal{P}_n$ be the group of n-fold tensor products of $\{I, X, Y, Z\}$, allowed an overall phase of ± 1 or $\pm i$. Any two elements of $\mathcal{P}_n$ either commute or anticommute with each other. This is the single algebraic fact the entire stabilizer formalism runs on.
- **Stabilizer Groups and Codespaces:** A *stabilizer group* S is an abelian subgroup of $\mathcal{P}_n$ that does not contain $-I$. (Excluding $-I$ matters: if $-I \in S$, then S demands $|\psi\rangle = -|\psi\rangle$ for every codeword, which no state satisfies other than the zero vector; the codespace would be empty.) If S has $n - k$ independent generators $g_1, \ldots, g_{n-k}$, define the codespace as their simultaneous $+1$-eigenspace:

$$\mathcal{C} = \left\{ |\psi\rangle : g_i |\psi\rangle = |\psi\rangle \text{ for all } i \right\}$$

- Each independent generator halves the dimension of the space it's imposed on, so this codespace is $2^n/2^{n-k} = 2^k$-dimensional: an $[[n, k, d]]$ code, encoding k logical qubits in n physical ones, purely by specifying $n - k$ commuting Pauli operators.
- **Syndrome Extraction** is just measuring the generators. This is the payoff for all the group-theoretic bookkeeping. Suppose an error $E \in \mathcal{P}_n$ hits an encoded state. For each generator g_i:
  - If E commutes with g_i, measuring g_i still returns $+1$: this generator's syndrome bit is silent.
  - If E anticommutes with g_i, then $g_i(E|\psi\rangle) = -Eg_i$; $|\psi\rangle = -E|\psi\rangle$, so measuring g_i now returns -1. This generator's syndrome bit flips.
- The full pattern of $+1$'s and -1's across all $n-k$ generators *is* the error syndrome, and (exactly as we have seen with all of our QEC codes) reading it off never requires learning anything about the encoded logical state, since every $g_i|\psi\rangle = |\psi\rangle$ regardless of which logical state $|\psi\rangle$ we've encoded.
- Looking back at Shor's Code: the parity checks Z_1Z_2, Z_2Z_3 (and their counterparts in the other two blocks), together with $X_1X_2X_3X_4X_5X_6$ and $X_4X_5X_6X_7X_8X_9$, are our set of stabilizer generators for the Shor code! In fact, when this code was first developed, we had no idea of the notion of a stabilizer and how universal it would be.
- A good question to be asking at this point is: how do we know if a syndrome is fully described by our stabilizer measurement? Two errors $E_a, E_b \in \mathcal{P}_n$ are jointly correctable exactly when $E_a^\dagger E_b$ falls into one of two safe categories:
  - $E_a^\dagger E_b \in S$: the two errors act identically on the codespace (they differ only by a stabilizer element, which acts as the identity there), so there was never anything to distinguish;
  - $E_a^\dagger E_b$ anticommutes with at least one generator: the syndrome catches the difference outright.
- The dangerous case (the one that goes by undetected) is when $E_a^\dagger E_b$ commutes with every generator but is *not* itself in S. This operator would be invisible to every syndrome measurement (every generator returns $+1$, exactly as if no error at all had occurred) while still acting nontrivially on the encoded logical information.
- This is precisely what we saw happen with the Shor Code: inside one block of the Shor code, the "fix" triggered by a weight-2 error X_1X_2 was to apply X_3, so the net effect on the state was $X_1X_2X_3$: a weight-3, whole-block X-string that commutes with every stabilizer generator without being a product of generators itself.
- **Logical operators and code distance, properly defined.** Let $N(S)$ be the *Normalizer* of S in $\mathcal{P}_n$: all Pauli operators that commute with every element of S (equivalently, that map the codespace back into itself).
- Elements of S itself act as the identity on the codespace, so the operators that do something genuinely new to the encoded logical qubits are the cosets of $N(S)/S$. These are the **logical operators**. With this language, the code distance d has a clean, general definition: it's the minimum weight of any element of $N(S) \setminus S$, i.e. the lightest possible error that acts as a nontrivial logical operator while remaining completely invisible to every syndrome measurement. This should match our previous intuition of the word.
- CSS codes are stabilizer codes with unmixed generators. What does this mean? If a stabilizer group S can be generated using operators that are each *purely* X-type ($\prod_i X_i$, no Z's) or *purely* Z-type ($\prod_i Z_i$, no X's). Alternatively, the code is a CSS code if the X-type generators are rows of C_1's parity-check matrix, the Z-type generators are rows of $C_2^\perp$'s parity-check matrix, and the requirement that all generators commute translates exactly into the CSS nesting condition $C_2 \subset C_1$.
- Equivalently, the two parity-check matrices H_X, H_Z satisfy $H_X H_Z^\intercal = 0 \pmod 2$). So everything in our previous section was already stabilizer language, restricted to this unmixed special case.

- Let us look at the Shor code's stabilizer generators for the third time. The $[[9, 1, 3]]$ code is generated by 8 independent operators. It is one short of the $n = 9$ needed to pin down a single state, since $k = 1$ leaves one logical qubit's worth of freedom:

Type	Generator
bit-flip check, block 1	Z_1Z_2
bit-flip check, block 1	Z_2Z_3
bit-flip check, block 2	Z_4Z_5
bit-flip check, block 2	Z_5Z_6
bit-flip check, block 3	Z_7Z_8
bit-flip check, block 3	Z_8Z_9
phase-flip check, blocks 1–2	$X_1X_2X_3X_4X_5X_6$
phase-flip check, blocks 2–3	$X_4X_5X_6X_7X_8X_9$

- Of course, we can do the same for the Steane code's stabilizer generators. The $[[7, 1, 3]]$ code is generated by 6 operators, one X-type and one Z-type per row of the same Hamming check matrix H from section 8.4: row (0001111) gives $X_4X_5X_6X_7$ and $Z_4Z_5Z_6Z_7$; row (0110011) gives $X_2X_3X_6X_7$ and $Z_2Z_3Z_6Z_7$; row (1010101) gives $X_1X_3X_5X_7$ and $Z_1Z_3Z_5Z_7$. The logical operators, as mentioned above, are simply $\bar{X} = X^{\otimes 7}$ and $\bar{Z} = Z^{\otimes 7}$.

- Beyond unifying every code we've met, the stabilizer picture is what makes quantum error correction work at scale.

- This is also the content of the Gottesman–Knill theorem: any circuit built only from Clifford gates (H, S, CNOT) acting on stabilizer states, with Pauli measurements, can be simulated efficiently on a classical computer. We will now discuss Fault Tolerance, which takes these QECCs a step further. The whole game is designing circuits that manipulate encoded logical qubits without ever needing to peek at, or trust, an individual physical qubit.

8.6 Quantum Error Correction as Topology and Fault Tolerance

8.6.1 What Is Fault Tolerance?

- We closed the last section on the idea that fault tolerance means designing circuits that manipulate encoded qubits without trusting any single physical qubit. However, we built this on the strong assumption that the CNOTs and Hadamards doing the extracting were perfect.

- This means we need to demand **Fault Tolerance** to implement a quantum computer. This entails protecting encoding, gates, syndrome extraction, and decoding. The issue is we must do so with the same noisy tools.

- This may immediately raise red flags for many. Seeing as entangling gates propagate errors, a single faulty gate can smear errors across many more qubits. A careless circuit will turn a simple mess up into a much shitty-er mess up.

- We want a circuit that implements an encoded operation such that any single faulty component inside it introduces at most one error per code block. We feed that into a code built to handle weight-t errors, and the fault gets corrected exactly like ordinary noise.

- We have met the main tool for these: transversal gates. First introduced as logical operators in the Steane and Shor codes, these apply a gate independently on each qubit, across a block. Thus, no interaction between different qubits of the same block. A single faulty transversal gate can only ever touch one qubit per block. A single faulty gate within a transversal gate can only ever touch one qubit per block, by construction, which is exactly the isolated fault our code already knows how to fix.

- However, since we can never have anything nice in this accursed world, not every logical gate can be made transversal on a given code (**Eastin-Knill theorem**). Clifford gates usually are, but the T gate usually can't be. This is precisely why we need to perform something called **magical state distillation**.

- **Threshold Theorem:** by concatenating a code (nesting fault-tolerant gadgets inside fault-tolerant gadgets), below some threshold physical error rate, the logical error rate falls double-exponentially, with each level of concatenation. However, the overhead grows only poly-logarithmically. That is, arbitrarily long quantum computation is possible in principle.

- What's left is finding codes with favorable thresholds and gate sets that are cheap to make fault-tolerant. This is why we must introduce topological codes.

8.6.2 Why Topology?

- Every code we have built so far, the 3-qubit bit- and phase-flip codes, Shor's code, Steane's code, and CSS codes in general, was found the same way: start from some algebraic object (a classical code, or in Shor's case, pure trial and error), then check after the fact that the Knill–Laflamme conditions hold. This works, but it doesn't tell us where to go looking for a *better* code.

- There's also a very physical problem hiding in our constructions so far. They require non-local connectivity of the circuit. The Shor code's stabilizer, $X_1X_2X_3X_4X_5X_6$, is a six-body measurement. Real hardware lays qubits out in actual physical space, usually with only nearest-neighbor connectivity. Measuring a six-body operator on qubits that aren't all next to each other means routing information across the chip, which is itself slow and another source of error.

- We want a stronger goal than "the code corrects errors of weight t." We want to design a code where every stabilizer generator only ever touches a fixed number of local qubits, all while the code distance still grows as we add more physical qubits.

- This is a BIG constraint. For stabilizer codes built entirely from local checks on a 2D layout, you can imagine there is a known trade-off between the number of logical qubits k and the distance d (roughly $kd^2 = O(n)$). So if we are content with k staying small, this still permits d to grow like $O(\sqrt{n})$, using only local, few-body measurements. (This is because code distance is "linear" and n is "quadratic" in the lattice.

- A concern is raised: every single check we perform is nearsighted (by design); it can only ever look at a handful of neighboring qubits. However, we want large-scale protection from errors. How do we achieve this?

- The answer is topology. Instead of storing the logical information redundantly in the *value* of a local patch of qubits (as the repetition code does, quite literally writing the same bit three times in a row), we can store it in a *global, topological* property of the entire qubit layout, something that provably cannot be read off, or altered, by acting on any small region at all.

- The go-to analogy is to picture a rubber band looped around a donut. You can wiggle it, stretch it, nudge it a little bit at a time, locally, anywhere on the surface, and it will always still be looped around the donut hole exactly once. The only way to change that winding number is to cut the band and do something drastic to a large fraction of it all at once.

- A small, local disturbance is topologically powerless against it. This is precisely the kind of protection we want against a physical error, which, unless we get really unlucky, acts locally.

- This might sound intimidating, but I will try to provide an intuitive (and slightly mathematical) perspective of this code.

- In the example we will explore in the next part, we will place our qubits on the edges of a lattice drawn on a torus (i.e., a donut). Local stabilizer checks, a "vertex" check and a "plaquette" check, each touching only 4 neighboring qubits, will catch any small, local error immediately.

- The operations that actually flip the encoded logical qubit will correspond to strings of Pauli operators that loop all the way around the torus: *non-contractible loops*, in the language of topology. An error has to grow to be as large as "half of one trip around the torus" before it can masquerade as a full loop. This gives us a code whose distance scales with the size of the lattice, built entirely out of 4-support and geometrically local checks. This is Kitaev's **toric code**, which we build below.

- For those who are pendantic, "Topology" here really means *algebraic topology*, and specifically homology. We will see that the first Betti number of the surface the lattice lives on describes the number of encoded logical operators. For example, a torus has two (you can loop around the hole, or through it), so it stores 2 logical qubits; a sphere has none, and stores 0.
- So without further ado, let's jump into our first topological code.

8.6.3 The Toric Code

- Take an $L \times L$ square lattice drawn on a torus. That is, the edges wrap around, so the lattice has no boundary. Alternatively, you can think of it as the quotient space construction of a torus. Through basic counting, the lattice has L^2 vertices, with $2L^2$ edges. We place a qubit on every edge, so $n = 2L^2$ physical qubits total.

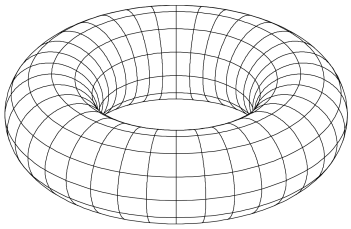


Figure 28: Example of a "Latticed" Torus

- **Stabilizer Generators**:
  - For every vertex v, define its respective vertex (star) operator as the product of X's acting on the edges touching v:

$$A_v = \prod_{i \in star(v)} X_i$$

  - Moreover, for every face (plaquette) define its respective plaquette operator as the product of Z's acting on the edges of the face:

$$B_p = \prod_{i \in \partial p} Z_i$$

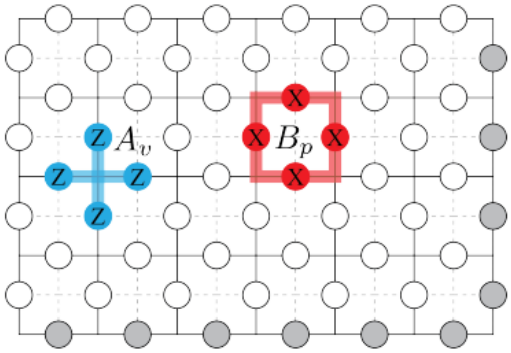


Figure 29: Toric Code Stabilizers

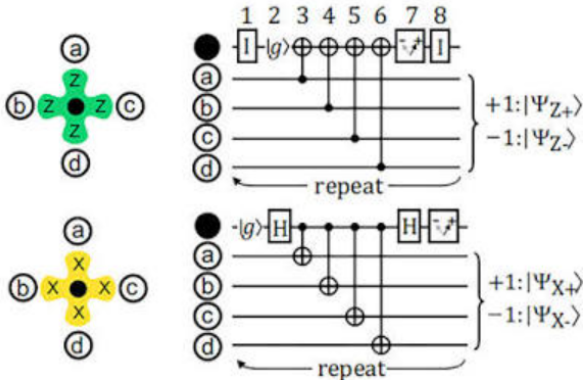


Figure 30: Ancilla Measurement Circuit

- Something important to understand is that each distinct operator here (stars and plaquettes) commutes. It is easy to see for two distinct operators of the same type. However, it also holds for overlapping stars and plaquettes even though they are composed of Z's and X's.
- The reason why is that there is always an *even* overlap of supports. That is, if a star and a plaquette overlap, they will do so on an even number of supported qubits. Thus, commutation will produce an even number of anticommutations $((-1)^{even} = 1$.
- This is actually our old CSS condition $H_X H_Z^T = 0 \ (mod\ 2)$ disguised as a geometry. In fact, if you want to consider it this way, you can imagine that the toric code is a CSS code.
- One code is given by the cycle space of the graph. Note that there are two logical qubits implemented here. One loop of the torus is the first instance, and the other loop of the torus is the second instance. Then the second code is given by the dual of this space
- With L^2 vertex checks and L^2 plaquette checks on $n = 2L^2$, we would expect that there would be no encoded logical qubits ($k = n - 2L^2$). However, there's a relation hiding in each stabilizer: by the periodicity of the lattice geometry, and the fact that each edge touches two stabilizers of each type
$$\prod_v A_v = I, \quad \prod_p B_p = I$$
- What this means is that there are only $L^2 - 1$ independent vertex generators (by definition of independent generators) and $L^2 - 1$ independent plaquette generators, leaving $k = 2$.
- This is actually a pretty cool result, since as we mentioned, this logical qubit count is directly related to the first Betti number. You can think about stabilizers and encoded logicals on physical qubits as a question of parameters and constraints. The "wasted" relations are a direct fingerprint of the surface having one "hole."
- We cannot explicitly construct a generalized logical state for the toric code, since the number of terms would grow very fast. However, using brute-force computation, orthonormality, and logical constraints, we can apply linear algebra and a computer to determine the logical states stabilized by our toric code precisely.
- **Logical Operators**:
  - We define an X-path as a sequence of adjacent X's starting and continuing along the edges of the original lattice, since the qubits live on those edges.
  - We, likewise, define a Z-path as a sequence of adjacent Z's drawn on the dual lattice (through the centers of the plaquettes), but it still acts on the same edge qubits (the ones whose edges are crossed by the dual path).

- A string of X's along a closed loop of edges automatically commutes with every A_v (since its X's meet X's), and commutes with every B_p so long as the loop *crosses* the boundary of every plaquette an even number of times (which is trivially true of any closed loop). The same logic holds with a Z string. I would also like to make note of the word cross; here, it means to enter and leave the boundary of the plaquette (not residing on it).
- The question is whether they are themselves generated by the stabilizers (and therefore act trivially on the codespace) or not: contractible loops act trivially on the space (they're a stabilizer in disguise), and non-contractible loops act non-trivially on the codespace (logical operators).
- If the loop is **contractible**, it bounds some patch of the lattice, and one can check it is exactly reproducible as a product of the $A_v's$ or $B_p's$ inside that patch. Thus, our contractible loop is a stabilizer in disguise, acting as the identity on the codespace.
- If the loop is **non-contractible**, it wraps around one of the cycles of the torus. Therefore, it is not a product of local generators (no patch of the lattice has it as a boundary, since it doesn't bound anything). This means this forms a logical operator. If the transversal loop is a Z string, then it is a Z logical, and if it is an X string, then it is an X logical operator.
- Since a torus has two independent cycles (around the hole and through it), one cycle gives X_{1L} and Z_{1L}; the other gives X_{2L} and Z_{2L}.

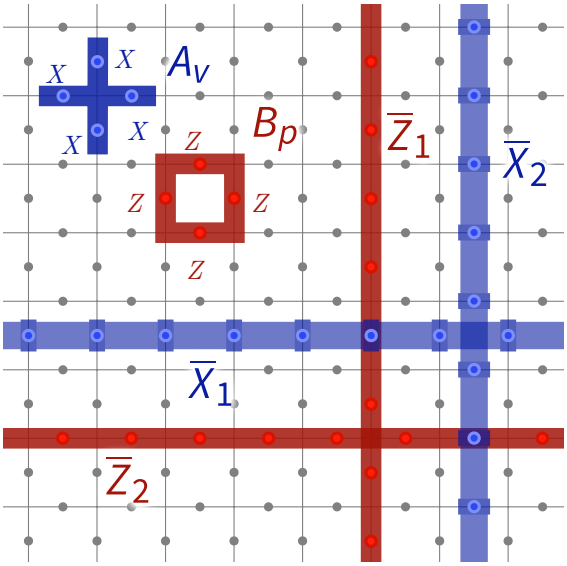


Figure 31: Toric Code Logical Operators

- Errors as Strings:
  - Suppose a Z error hits a single edge. Z anticommutes with X, so it anticommutes with the two vertex operators that edge touches, commuting with everything else. Measuring the vertex checks now returns -1 at exactly two vertices. We call each such flipped vertex an **e-type** ("electric charge") excitation.
  - In the same way, a single X error anticommutes with every plaquette operator that borders its edge, creating a pair of **m-type** ("magnetic flux") excitations.
  - A whole chain of same-type errors along a path behaves just like it did for the repetition code inside Shor's code: every plaquette along the path gets hit twice, and the flip cancels, so only the two endpoints of the error chain light up. The syndrome tells you where an error change starts and ends, never its shape or its length.
  - If an error chain is composed of X's and Z's, you can decompose it into many shorter chains of solely X's and Z's.

- You may now be a bit concerned since, if we are only given a syndrome's start and end, how can we decode this error? Given a syndrome, i.e., a set of flipped generators, the decoder's job is to guess a pairing of them and a path connecting each pair, then apply a correction.
- This is the **minimum-weight perfect matching** problem, and it can be solved efficiently. Correction succeeds if the *true* error chain and guessed correction loop trace out a contractible loop, since a contractible loop is just a stabilizer.
- It fails precisely when the true error and the guess together trace a *non-contractible* loop. That is, think of a string that covers most of a cycle, but gets closed by a mischaracterization of the decoder.
- The shortest non-contractible loop on an $L \times L$ torus has length L (it's a straight line around a cycle). An unlucky error must span about $L/2$ qubits along a straight line before this is even possible. This is how we get that the parameters of the toric code are $[[2L^2, 2, L]]$. This is exactly the trade we were promised at the start of this section: we gave up our k scaling in exchange for d that scales freely with n.
- Something we also promised was fault-tolerance. So the question naturally emerges: what happens if we have faulty components? We should see that each fault only affects a small, local set of qubits. A single faulty cNOT cannot instantly create a long error chain. So regardless of our fault (qubit noise, faulty gates, or poor measurement quality), our logical information is protected.
- Because all faults are local, repeated syndrome measurements can reveal short paths in the 3D space-time lattice. The decoder then infers the most likely combination of data errors and measurement errors to better correct errors.
- Decoding the toric code under independent noise can actually be mapped onto the statistical mechanics of a disordered Ising model, allowing quantum error correction to be analyzed as a classical phase transition.
- This mapping predicts an error threshold (roughly how much noise the code can take) of about 10-15%, depending on the noise model and decoder. Below it, larger codes exponentially suppress logical errors, while above it, increasing the code size no longer improves reliability.
- Congrats! We have finished our first topological code. Not bad, right? Right? I'm going to take your lack of a comment as a good sign rather than a sign that you fell asleep.
- The only question is now, how the actual hell do we implement a torus on a computer chip? Unfortunately, the Dunkin Donuts CEO didn't hard pivot the company to quantum hardware, so we have to find a way to implement a code like this on a flat *surface*. If only we had a Surface Code...

8.7 A Golden Standard of QEC: the Surface Code

- We ended the last section on a pretty reasonable complaint: for our code to be implementable, we need a "flat" topology, not a torus.
- The fix is pretty anticlimactic. We take the same machinery of the toric code (the same square lattice, same qubit orientation, and same vertex and plaquette operators) and just don't glue the boundaries of the lattice to get a torus.
- We take a box cutter to the torus and let the lattice have its boundaries (these are essential for how it actually works). We are left with the planar code or, as it's more well known, **the Surface Code**.
- Without exaggeration, nearly every single major superconducting quantum computing effort is trying to run some flavor of or homologous to this exact code. Just to list a few examples, we have the Tile codes, Hypergraph Product Codes, Surface Floquet Codes (LUCI), and the Color Codes.
- So, let us examine precisely what this code is.

8.7.1 Cutting the Donut

- Taking the same square lattice as we had before but now confining it to a finite rectangular patch instead of wrapping it into a torus. The periodic bounds of the lattice no longer exist; the lattice simply ends.
- The issue is not that every vertex or plaquette operator survives this cut fully intact:
  - Vertices sitting right on the perimeter are missing one of their "spokes" (edges), so they are connected to three (if on the outer edge of the lattice) or two (if on the outer corners of the lattice) edges. Therefore, these boundary A_v naturally reduce their weight/support.
  - The same complication occurs to B_p operators as well, reducing the weight to three at the boundary. However, by the geometry of the plaquette, we will never have an operator reduce its weight to two.
  - You can terminate a lattice edge in one of two different ways, and these matter significantly. The **smooth boundary** cuts the star (X-type, A_v), and the **rough boundary** cuts the plaquette (Z-type, B_p).

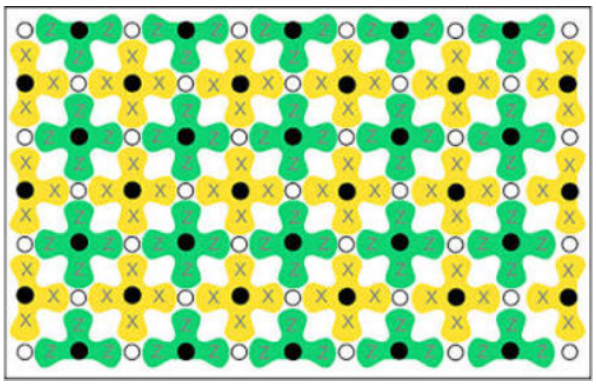


Figure 32: Depictions of Rough (Vertical Lattice Edges) and Smooth (Horizontal Lattice Edges) Boundaries

- We require two pairs of each boundary because this asymmetry is precisely what allows for logical information to be encoded. This is because a chain of X's can only pass undetected by our syndrome measurement if it starts and ends on an X-type smooth boundary. Conversely, a chain of Z's can only pass undetected if it starts and ends on a Z-type rough boundary.
- The reason why is because of a similar commuting argument as we did for the toric code. If the error string starts or ends in the bulk (the non-boundary section of the lattice), then at least one syndrome measurement will read -1. However, because an even number of anticommutations is required, if it starts and ends at its respective boundary type, there are no ancilla measurements that will read -1.
- Note, starting an X (or Z) error chain on one boundary type and ending it on the other boundary type doesn't contribute anything other than a single -1 measurement. It is not a special case.
- This is precisely how we can implement logical operators (and also logical errors). If we don't have both rough and smooth boundaries, we cannot encode logical operations.

8.7.2 The Finer, Logical Details

- In the toric code, we need loops to encode logical information around. Let's see how this manifests on the surface code.
- On the surface code, we replace this notion with an open string of operators between boundaries (as we mentioned above). We call the open string of X's between smooth boundaries the X logical. Likewise, the open string of Z's between rough boundaries is the Z logical. We call these

gate transversal since it is a sequence of gates acting on individual qubits (without letting the individual gates act within the same code block).

- We can easily see that if we write out our transversal logicals, they anticommute since the operator strings must cross an odd number of times.
- But for the mathematical pedants, how does this connect to the Betti number? The first Betti number of a finite plane is zero, so where do we get logical information from?
- We need to consider the relative version, which counts the independent open paths that connect distinct (same type) boundaries. Thus, we get that we only encode ONE logical.
- This is a step down from the toric code's two encoded logical qubits. However, this is honestly worth it to have a topological code sit flat on a plane.
- The code distance is defined by $min(d_X, d_Z)$. Here, d_X is the length of the shortest X-string spanning two smooth boundaries, and d_Z is similarly defined for Z-strings spanning two rough boundaries. Often, we will engineer the patch's proportions such that $d_X = d_Z = d$, which is a desired code distance (and also the size of one boundary edge of the square lattice).
- Forcing d_X and d_Z to agree creates an asymmetric formula for our total qubit code. This is because smooth-to-smooth and rough-to-rough spans come out one row apart in size. The parameters of the code are: $[[2d^2 - 2d + 1, 1, d]]$.
- Note this n value only describes the required physical qubits and doesn't describe that we require $2d^2 - 2d$ ancilla qubits to measure the syndrome.
- For d=3, we already see an improvement over both the Shor and Steane codes, even with the constraint of local connections.

8.7.3 Syndrome Extraction and Decoding

- Extracting the syndrome works exactly the way we described back in the stabilizer section: an ancilla per syndrome check, cNOTs to (or from) surrounding data qubits, then measure the ancilla. All that changes is that the boundary checks have one (or possibly two) fewer cNOTs than a bulk check.
- Decoding is also simply lifted from the toric code with a **Minimum Weight Perfect Matching Algorithm**, which is again implementable efficiently on a classical computer. Again, a Z error creates a pair of **e-type** excitations at the two vertex checks it has as endpoints, and the same goes for X errors and **m-type** excitations.
- The new aspect we will have to consider is the boundary itself. An excitation can also be matched directly to the nearest boundary of its own type, at a cost equal to its distance to that edge.
- Practically, this means that we give the matching algorithm one extra virtual node per excitation type to represent the boundary. This "competes" with every real excitation to determine the most reasonable error chain. This is good news; we don't have to do much to adapt our decoding for the surface code.

8.7.4 The Rotated Surface Code

- The construction above with qubits on edges, checks on vertices and faces is the "first: and most direct way to build a surface code. However, it isn't the version actually implemented on real hardware.
- Real chips actually use the **rotated surface code**, which uses fewer physical qubits for the same code distance. As we stated above, the surface code has $[[2d^2 - 2d + 1, 1, d]] + 2d^2 - 2d$ measure qubits. The rotated surface code has $[[d^2, 1, d]] + d^2 - 1$ measure qubits.

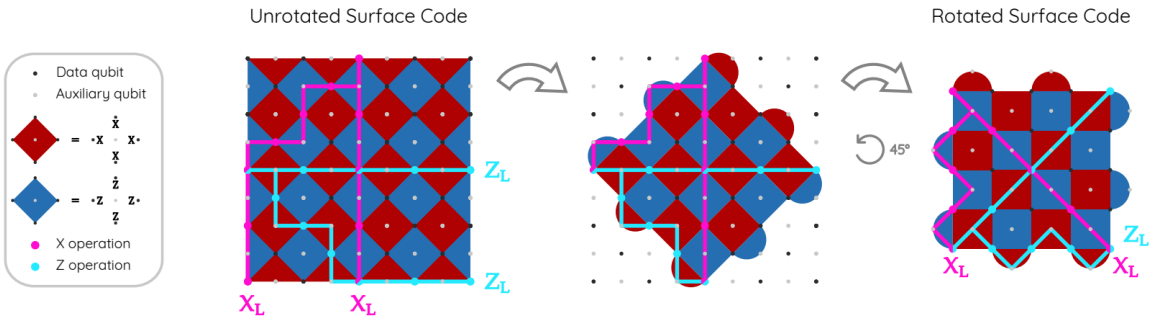


Figure 33: How to Rotate the Surface Code

- Instead of qubits on edges of the lattice, we put the $n = d^2$ data qubits directly on the vertices of the $d \times d$ grid. The stabilizers turn from a diamond pattern into a checkerboard pattern, with the couplers still going to the center of the tile.
- Now, every boundary stabilizer only gets two qubit supports. The geometric notion (picture image) of rough and smooth boundary conditions gets replaced with the "bumps" we see on the image above.
- Here, Red tiles represent X operators and Blue tiles represent Z operators. Additionally, the blue nubs represent smooth boundaries, and the red nubs represent rough boundaries.
- Other than these minor changes, everything about the rotated surface code stays the same as its unrotated counterpart. Therefore, it's a pure improvement over the surface code.

8.7.5 How Good is This?

- In the toric code section, we sourced a threshold of around 10-15%. That figure survives here relatively unchanged because locally, the toric code looks like the surface code. We only really care about what's going on in the bulk, not really at the boundary.
- This 10-15% number makes a pretty generous assumption: we have perfect syndrome measurements (the noise only gets dumped onto the gates and data qubits).
- This idealistic set of conditions is known as the **code-capacity threshold**, referring to the highest noise or error rate a quantum error-correcting code can tolerate under ideal conditions, assuming perfect, noise-free syndrome measurements.
- For noisy measurements, this figure drastically reduces all the way down to between 0.5-2.9%. Nevertheless, the fact that a topological code can tolerate a full percent of its physical operations being wrong and still operate indefinitely is incredible.
- This stopped being purely theoretical by 2025 when Google implemented the rotated surface code (with $d = 7$) on its Willow quantum computer.
- There is one question left: why haven't we discussed logical cNOTs and logical T-gates? This will be the source of our discussion in the coming two sections.

8.8 Lattice Surgery, Logical cNOTs, and the Logical Hadamard

- As of now, a single surface code patch is very effective for storing encoded logical information, but it is a very useless quantum computer. Each patch (copy of the surface code) simply protects one qubit's worth of information. To do computations, we need logical qubits to talk to each other. That is, we need to implement a logical cNOT without trusting an individual physical qubit (and obviously only local connections).

- Historically, there have been two main approaches. The first requires braiding logical information around holes (**defects**) in the lattice. To do so, we turn off stabilizer support to encode further topological information (manually increasing the Betti number), mimicking braiding of anyons.
- Conceptually, this idea is quite pretty, allowing for a clean mathematically oriented way to implement a cNOT. Practically, moving information across the QPU chip in big loops is expensive in both time and qubits. In 2012, a much less clean but a whole lot more practical solution was proposed: **lattice surgery**.
- Now, you don't need an MD to do this procedure, just a several-million-dollar quantum computer. This still requires meticulous detail, so let's break it down step-by-step.

8.8.1 Merging and then Separating Two Patches

- Before we do any surgery, we need to know how to open and close a "wound." So let's do a merge and then split off our two patches to see what happens.
- Take two patches sitting next to each other on a greater lattice, oriented so that a matching pair of boundaries (both rough or both smooth) are facing each other over a small gap. We then fill that gate with a strip of fresh physical qubits and stabilizers (initialized according to the eigenstates of the corresponding boundaries).
- These qubits participate in the new stabilizers exactly like any other bulk qubits for the time being. We then run syndrome extraction over the entire combined patch for roughly d rounds.
- We repeat this measurement several times such that a single-mid merge fault can't sneak past the code's usual distance. That is, if we only performed one round, we can't distinguish between data-qubit and measurement errors.
- From this, we get a larger surface code patch. We can then split the patches apart by measuring the strip-out on a destructive basis. We can now restart independent syndrome extraction on the individual halves (we are back to the original separation). We measure the middle physical qubits (not the ancillas) to remove them from being encoded in the logical information. Destructively measuring (measuring in an opposite basis) removes the possibility of entanglement between the code patches.

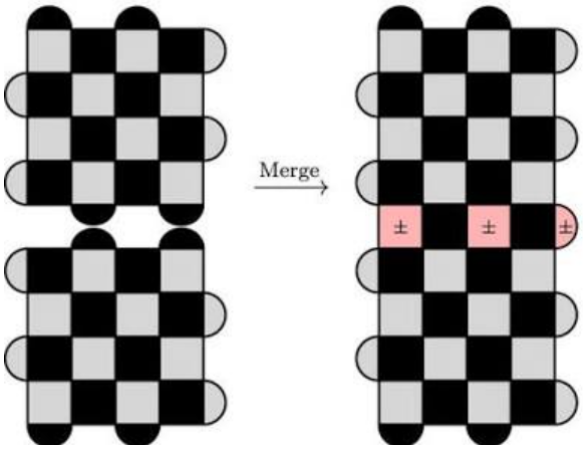


Figure 34: Lattice Grafting Procedure

- As an example, for a rough merge: we initialize the gap physical qubits as $|0\rangle$, where the Z-plaquettes of the boundary have deterministic $+1$ eigenvalues, and the X-checks will settle over repeated syndrome extraction. The converse is done for a smooth boundary merge.

- While the patches are merged, the boundary that used to separate the two patches has disappeared. To get rid of a logical qubit's worth of "freedom" has been erased, something is forced to give, you know, by the law of equivalent exchange and all. What gives when you sacrifice one logical qubit is the parity between the two original logical qubit operators. Concretely,
  - Merging along rough boundaries induces a joint measurement of $X_{1L}X_{2L}$
  - Merging along smooth boundaries induces a joint measurement of $Z_{1L}Z_{2L}$
- A natural question to have is: if a rough boundary is where the Z logical lives, shouldn't a rough merge measure the Z logicals (and vice versa)? Well, it's quite the opposite. Fusing a boundary where a Z logical used to end lets a Z logical string pass through the bridge into a neighboring patch, thus Z_{1L} and Z_{2L} are forced to agree automatically (otherwise they are no longer logicals).
- The new undetermined quantities the merge reveals are the perpendicular operators' relative value, $X_{1L}X_{2L}$. This measurement asks us if the new (longer) boundary agrees or not. This feels very familiar to Bell measurement; we are taking joint parity information from two qubits using nothing but local operations. This sketches out a path forward.

8.8.2 Building a cNOT Out of Two Measurements

- Let us see now if we can reverse engineer a cNOT from this simultaneous measurement. A single merge only tells us parity information, and it doesn't itself entangle the two logical qubits' full amplitudes the way a cNOT does. To upgrade this measurement into a gate, we will "burn" an ancilla in a resource state and let a sequence of measurements and classically controlled corrections.
- The recipe for a logical cNOT between a control patch C and a target patch T, using one ancilla patch A, is:
  1. Prepare A in the $|+\rangle_L$ state.
  2. Smooth merge C and A, measuring $\bar{Z}_C\bar{Z}_A$. Then, split the patches.
  3. Rough merge A and T, measuring $\bar{X}_A\bar{X}_T$. Then split these patches.
  4. Measure A destructively in the logical Z basis.
  5. Apply classically-controlled Pauli correction to C and/or T, depending on our three measurement outcomes.
- This procedure should remind you of something familiar: quantum teleportation. If you need more convincing, let's walk through this procedure. Let us, for the sake of simplicity, assume that we get +1 and +1 for our measurements. This is the trivial outcome, but the non-trivial outcomes differ by a classically controlled quantum gate (like in quantum teleportation).
- Starting with $|\psi\rangle_C|+\rangle_A|\phi\rangle_T$, $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, $|\phi\rangle = \gamma|0\rangle + \delta|1\rangle$.
  - Measuring $\bar{Z}_C\bar{Z}_A$ and getting +1 projects onto $(\alpha|00\rangle_{CA} + \beta|11\rangle_{CA})|\phi\rangle_T$. A has picked up a Z basis copy of C's state, exactly as a cNOT from C onto a fresh $|0\rangle$ ancilla would. Had the outcome come out to -1 instead, a single $\bar{X}_A$ correction on the ancilla puts you on this branch.
  - Measuring $\bar{X}_A\bar{X}_T$ on $(\alpha|00\rangle_{CA} + \beta|11\rangle_{CA})|\phi\rangle_T$ and getting +1, followed by reading A out in the Z basis and getting 0. We collapse the state of our system onto:
$$\alpha\gamma|00\rangle_{CT} + \alpha\delta|01\rangle_{CT} + \beta\delta|10\rangle_{CT} + \beta\gamma|11\rangle_{CT}$$
  - If we measure $\bar{X}_A\bar{X}_T$ to be -1, we would simply need a $\bar{Z}_T$ on T. Overall, the other seven measurement outcomes don't give anything new or scary; each one differs from this exact result by an extra logical on C or T. If we compare the base (trivial) case to a normal cNOT, we get the same logical state!
- For an additional note on the correction, like with teleportation circuits, you don't need to apply the corrections in real time; you can carry them forward (so long as you don't lose order) as classical annotations and "cash them in" when you want to measure or act a gate on logical information. This trick is called **Pauli frame tracking**.

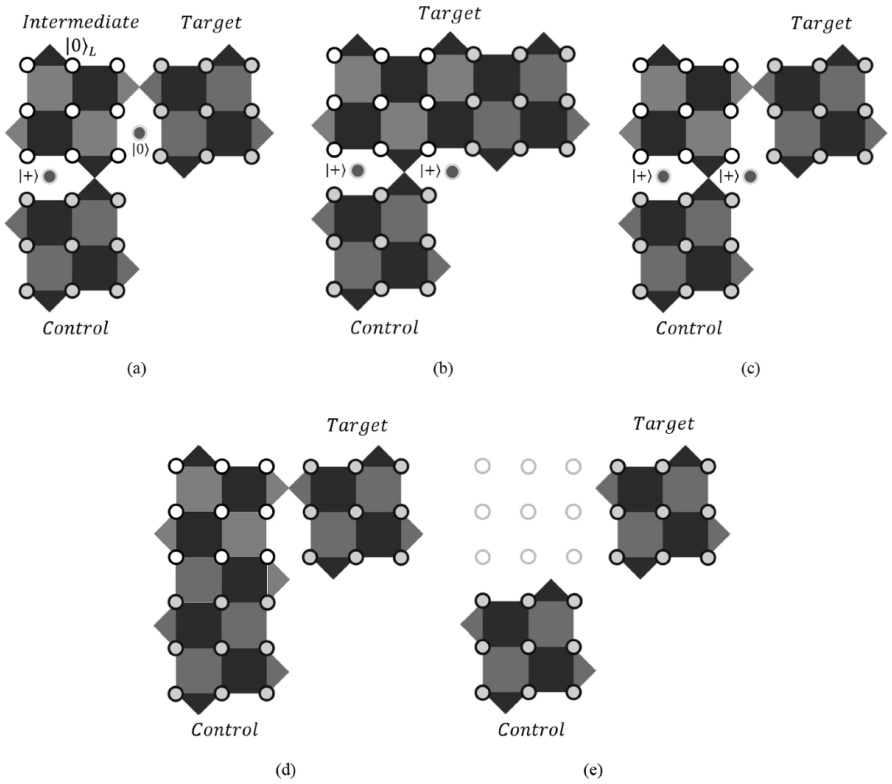


Figure 35: Overview of Lattice Surgery

8.8.3 The Cost of Lattice Surgery

- These cNOTs are not cheap. Each merge requires roughly d rounds of syndrome extraction to remain "honest," so a full lattice-surgery cNOT costs on the order of $2d$ rounds in addition to the need for an ancilla patch and the "fill-in qubits."
- Nevertheless, this cost is SIGNIFICANTLY cheaper than cNOT braiding, which costs a loop's worth of chip real estate and time.
- Something else: we can only merge patches that are local to one another on a larger chip. However, for a cNOT between two logicals that are not neighbors, you can extend a chain of extra ancilla patches to bridge the gap and merge your way down a path. These are all still local moves. (No non-locality gimmicks for us, sadly).

8.8.4 The Logical Hadamard

- Merging and splitting surface code patches cannot swap logical boundaries. To do so, we apply physical Hadamard gates to all data qubits. Then we swap the X- and Z-type stabilizer measurement circuits and apply a geometric patch rotation or deformation to restore the original boundary configuration. Over the course of a few rounds of code deformation, the smooth and rough boundaries begin to switch places.
- With $\bar{H}$ and $c\bar{N}OT$ (as well as $\bar{X}$ and $\bar{Z}$), we are beginning to fill out a Clifford set. All we need now to achieve a universal gate set (and thus a full quantum error correction code) is the S gate and T gate. There is a challenge, however; the T gate is a non-Clifford gate, and therefore can't be implemented fault tolerantly
- Therefore, we turn to some quantum wizardry, specifically Magical State Distillation, the final step in our error correction journey.

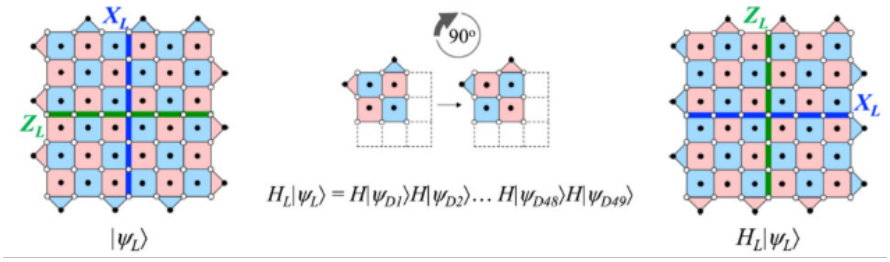


Figure 36: How to Logical Hadamard

8.9 The Logical S- and T-Gate and the Need for Magical State Distillation

8.9.1 A Close Look at the Logical S-Gate

- We ended the last section discussing the logical Hadamard gate. To implement it, we rotated the surface code patch to change its boundaries and acted with transversal Hadamards on the patch.
- The Hadamard utilizes boundary shifts because of its role as a Pauli conjugation ($X \leftrightarrow Z$). This is why we can implement it through boundary deformations. The S gate works in a similar cyclic rotation, but it maps the following cycle $X \to Y \to -X \to -Y \to X$ (given by a $\pi/2$ rotation about the Z-axis on the Bloch sphere).
- Seeing as our two-dimensional patch only has rough and smooth boundaries (and not a third "mixed" boundary), we can't rotate, or partially rotate, the code to implement an S-gate.
- Instead, we perform a process called gate teleportation. We consume a premade ancilla in the positive y-basis:
$$|+i\rangle + L = \bar{S}\,|+\rangle_L = \frac{1}{\sqrt{2}}(|0\rangle_L + i\,|1\rangle_L)$$
- Dropping the implied logical symbol, we take a logical data qubit: $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$ and a fresh copy of the $|+i\rangle_L$ sitting on an ancilla somewhere else. Our initial state is:
$$(\alpha\,|0\rangle + \beta\,|1\rangle)_D \otimes \frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle)_A = \frac{1}{\sqrt{2}}(\alpha\,|00\rangle + \beta\,|10\rangle + i\alpha\,|01\rangle + i\beta\,|11\rangle)$$
- Apply a cNOT with the data qubit as the control and the ancilla as the target (this is just lattice surgery):
$$\frac{1}{\sqrt{2}}(\alpha\,|00\rangle + \beta\,|11\rangle + i\alpha\,|01\rangle + i\beta\,|10\rangle)$$
- Now, if we measure the ancilla logical qubit. We project into either the zero or one outcome space:
  - If $|0\rangle$, we have the resulting state $(\alpha\,|00\rangle + i\beta\,|10\rangle = (S\,|\psi\rangle) \otimes |0\rangle)$. So no correction is needed.
  - If $|1\rangle$, we have the resulting state $(i\alpha\,|01\rangle + \beta\,|11\rangle = i(\alpha\,|01\rangle - i\beta\,|11\rangle) \approx Z(\alpha\,|01\rangle + i\beta\,|11\rangle) = (Z(S(|\psi\rangle)) \otimes |1\rangle)$. To recover the teleported state, we simply apply a Z logical gate to the first qubit.
- Thus, we have teleported the S-gate onto a logical state! However, this doesn't come for free. On every real hardware platform we have, it is a very noisy procedure. The procedure often suffers from decoherence, and there are many stages where errors can occur due to its length.
- Additionally, while the logical qubit supports a Y_L operator and $|+i\rangle_L$ is a valid logical state. The way to actually produce our $|+i\rangle$ state is a bit convoluted because we only support two types of geometric boundaries.

- I promised I wouldn't lie to you, but I did for the sake of your confidence. We can have an "in-between" state. We do so by implementing twist defects to modify the lattice, so mixed X-Z behavior is native to the hardware.
- This, however, is beyond the scope of the material covered. I will encourage the reader to explore this concept on their own.

8.9.2 The Logical T-Gate

- Let's take stock of where we are. We have a quantum error correction code and have fully developed a fault-tolerant Clifford set. All we need now is the T-gate, some non-Clifford "magic". Yet, as we have mentioned many times already, the Eastin-Knill theorem (prevents non-Clifford gates from being applied directly and transversally without destroying our error correction) puts a wrench in our plans.
- The T gate needs something that doesn't simply permute Pauli operators or manipulate the lattice geometry; we need to implement a state that isn't stabilized, and therefore is highly noisy.
- The key is gate teleportation, something we have learned in the section prior. We stop trying to apply the T gate directly and manufacture the effect of it elsewhere. To cultivate this effect, we use a process known as **Magic State Distillation**
- Define the logical state (we will drop the logical subscript in a moment for ease of writing): $|+T\rangle_L = \bar{T}\,|+\rangle_L = \frac{1}{\sqrt{2}}(|0\rangle_L + e^{i\pi/4}\,|1\rangle_L)$.
- All of the non-Clifford weirdness in the entire computation gets concentrated into the manufacturing of this one simple, fixed state. This process is very prone to error, and, in fact, it is one of the only things we must accept as noisy.
- Precisely as we did for the S gate, we now cNOT this logical state onto a logical data surface code patch and measure the ancilla:

$$|\psi\rangle_D \otimes |A\rangle_A = \frac{1}{\sqrt{2}}(\alpha\,|0\rangle\,(|0\rangle + e^{i\pi/4}\,|1\rangle) + \beta\,|1\rangle\,(|0\rangle + e^{i\pi/4}\,|1\rangle))$$

$$\rightarrow \frac{1}{\sqrt{2}}(\alpha\,|00\rangle + \alpha e^{i\pi/4}\,|01\rangle + \beta e^{i\pi/4}\,|10\rangle + \beta\,|11\rangle)$$

- As usual, we sort the measured outcomes and apply a classically controlled logical S gate:
  - If $|0\rangle$, we get the state $(\alpha\,|0\rangle + \beta e^{i\pi/4}\,|1\rangle) \otimes |0\rangle = (T\,|\psi\rangle) \otimes |0\rangle$. This is the outcome we want, so we don't touch this state.
  - If $|1\rangle$, we get the state $(\alpha e^{i\pi/4}\,|0\rangle + \beta\,|1\rangle) \otimes |1\rangle$. We then apply a logical S to this state, yielding $e^{i\pi/4}(\alpha\,|0\rangle + \beta e^{i\pi/4}\,|1\rangle) = e^{i\pi/4}T\,|\psi\rangle$
- Thus, we have found a way to implement a universal set of gates on the surface code! We used teleportation, lattice surgery/deformations, and the natural topology of the surface code, all (largely) fault-tolerant procedures.
- Something additional to note: when we measure the ancilla, we use it up. So every teleportation needs its own cultivated state, no refunds.
- We kinda cheated a little bit, though. We passed all the burden of the question of the fault-tolerant T-gate onto "how to create a non-Clifford state ($|+T\rangle$)?"
- The catch is, of course, we can't perfectly prepare $|+T\rangle_L$ perfectly and cheaply. So whatever we do make will be a crude approximation and noisy as hell. Our fix is we take a bunch of mass-produced (cultivated) state approximations, and we squeeze them into a better magical state. This is the subject of our next section: Magical State Distillation.

8.9.3 Magical State Distillation

- Magical State Distillation is a pretty long process, but it can be broken up into two long stride steps: **State Injection** and **State Refinement**.
  - State Injection takes a noisy singular physical qubit in the $|+T\rangle$ state and incorporates it into a surface code patch through several stages of measurement.
  - This very noisy patch, with many other similar patches, then builds an error correction code that supports transversal T gates built on top of the surface code. We run error correction on our logical $|+T\rangle_{RL}$ state, and produce a refined, high fidelity state.
  - We then decode this state into a single very strong "physical" $|+T\rangle_{L}$ ancilla patch of the surface code that we will then use as T-gate teleportation fuel.
- So let's begin exploring this process with state snjection on the surface code. **State injection** is a method for taking a physical qubit in an arbitrary state and creating a noisy, Clifford-stabilized state on the surface code. The key word here is *noisy*.
- This idea reframes our previous hard problem of protect a gate implementation to protect a state. Thus, we identify its own procedure.
- **State Injection Protocol**:
  - Prepare a single physical qubit $|\psi\rangle$ using ordinary (non-fault tolerant) gates for a T-type magic state. We want a $|+T\rangle$ state.
  - "Inject" the state in a corner of the surface code, whether through a quantum teleportation circuit other means. Initialize the rest of the data qubits in the fixed product state the boundary conditions normally demand. Note: certain implementations will inject the state at the center, and there is dispute over where to inject.
  - Turn on stabilizer measurements. Since $|\psi\rangle$ generally isn't an eigenstate of the code's check, we initially have that our corner qubit is only protected at distance $d = 1$. This is because $\hat{Z}_L$ is supported on that one qubit alone.
  - Over subsequent rounds of measurement, the stabilizers naturally and noisily diffuse the information to be supported by other qubits. The patch takes $L - 1$ rounds to fully encompass the full lattice. We can show this entangling phenomenon, but for brevity this is left to the reader to verify. For a hint: these stabilizers are effectively the same entangling circuit we used for the Shor code.
- At the end of these measurement rounds, we are left with a poor reconstruction of the initial state. This poor quality is due to these injections not being fault-tolerant, amplifying potential errors in the original state.
- The key to refining exists within building a quantum error correction code within our quantum error correction code. For that, we look to the **[[15,1,3]] quantum Reed-Muller code**. It isn't chosen because it is scalable nor because it's strong, but because it has a transversal (fault-tolerant) T-gate: $\bar{T} = T^{\otimes 15}$.
- At this point, you should be familiar enough with the parity check construction of stabilizer codes to understand a code by its parity check matrix, so I will leave these matrices for the reader to reference:

$$H_X = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}$$

$$H_Z = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}$$

- So, with this code set. Let us find a way to refine our physical surface patch. Note, here when I say physical qubits, they are actually logical patches of the surface code:
  - Start with 15 physical qubits (surface code patches) in the state $|+\rangle$ and Clifford-encode (cNOT) them into a single logical state $|+\rangle_L$ of the [[15,1,3]] code. This builds our error correction code on top of the surface code. I like to think of this as the machinery of a factory, as is often depicted.
  - We now want the logical $\bar{T}$ on the Reed-Muller code. Luckily, as we mentioned, it is simply a transversal gate $T^{\otimes 15}$. To implement it. simply T-gate teleport our noisy states onto the $|+\rangle$ states that make up our logical. Every piece of this teleportation gadget is Clifford and fault-tolerant.
  - We measure the code's stabilizers. If every single check returns its trivial outcome, accept and continue. If it fails, throw away the entire batch and restart with 15 fresh qubits. This is where the noisy copies are post-selected, so we get high-quality logically encoded $|+T\rangle_L$.
  - Now, we decode the accepted state back down using the Clifford inverse of the encoding circuit, measuring out and discarding the other 14 qubits. What's left is one remaining clean, physical (surface code patch) qubit in our desired magical state.
- It is precisely this clean state that we can then teleport onto a desired algorithm. Going back to the factory analogy, the raw ancillas act as the materials, and you can think about the refinement process as the factory line.
- As is the case for actual factories too, we create a stockpile of these high-fidelity states before we carry out any actual algorithm. This is because on-demand production is highly inefficient.
- As a few words of warning, this protocol never tries to correct anything. It only ever detects and discards bad states. The fact that our code had distance three means that we can only detect every single weight two and below errors. Therefore, our rate of error scales as: $\epsilon_{out} \propto \epsilon_{in}^3$.
- This procedure only works well below a threshold. An idealized analysis with noiseless Clifford circuitry shows that the protocol purifies states as long as the raw input error sits under 14%. In practice, noisy Clifford gates limit performance, giving an output error of about:

$$\epsilon_{out} \approx 35\epsilon_{in}^3 + 7.1\epsilon_L$$

- Here, ϵ_L gives the logical error rate of the Clifford circuitry. This creates a noise floor that can't be overcome by additional rounds of distillation. The protocol is also probabilistic, so some batches are discarded, making the success yield an important consideration.
- Certain procedures that need higher fidelity states will often layer these distillations repeatedly. Large-scale quantum algorithms commonly require two or three such levels to achieve the necessary magic-state quality.
- For codes similar to the surface code, such as the color code, a similar procedure is implemented.

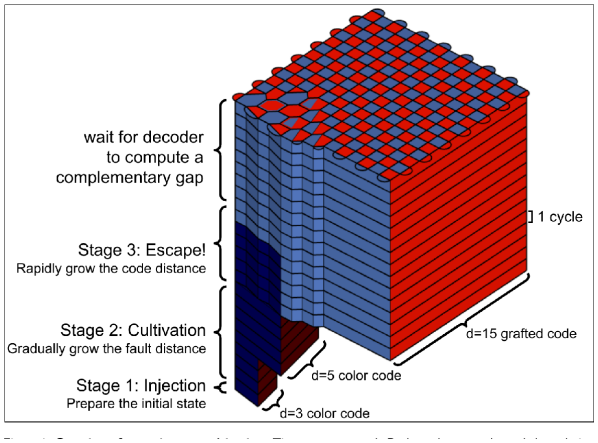


Figure 37: Magical State Distillation on the Color Code

8.10 A Farewell to Quantum Error Correction

- Quantum error correction offers us a much-needed layer of protection against noise. While no code is perfect, developing real-world implementations of these codes is a critical step in quantum supremacy.
- We have taken a look at only a handful of prominent codes: Shor's code, CSS codes, the toric code, and the surface code. However, the world of error correction is not limited to just these; some of which we have mentioned, like the color code or qLDPC codes. I hope to explore these codes and more in the next writing that I do specifically on quantum error correction.
- As of now, the theoretical side of quantum error correction far outpaces the hardware side, but through the refinement of quantum platforms, we will begin to see more codes implemented. Eventually, this will lead us to real-world large-scale quantum computation.

9 Conclusion

- It has been a pleasure exploring an introduction to quantum computing with you. This has only been a survey of the world of quantum computing. Hopefully, you have learned some useful things in this paper to aid you in your quantum journey and possibly deeper dives you may take. Moreover, I wish that you read the next paper I am in the process of developing, exploring quantum error correction. Finally, I would like to thank you for reading and wish you all the best.